The Australian National University
2600 ACT | Canberra | Australia



Australian National University

School of Computing

College of Systems and Society

One Solve, Every Step: Accelerating Actor–Critic Learning

— 24 pt research project (S1/S2 2026)

A report submitted for the course
COMP8800, Advanced Computing Research Project

By:
Hanning Deng

Supervisor:
Prof. Liang Zheng

October 2026

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the Academic Integrity Rule;
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or LMS course site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

I acknowledge that I am expected to have undertaken Academic Integrity training through the ANU Academic Integrity module/s prior to submitting an assessment, and so acknowledge that ignorance of the rules around academic integrity cannot be an excuse for any breach.

October 2026, Hanning Deng

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Prof. Liang Zheng, who first introduced me to research and has supported me every step of the way. Prof. Zheng generously gave me both guidance and the resources I needed, and whenever I ran into difficulties, I always knew I could turn to Prof. Zheng for help. This kindness and support have meant a great deal to me, both for this thesis and for my growth as a researcher. Being part of Prof. Zheng's group has also given me the chance to meet many outstanding students, from whom I have learned so much. I feel truly fortunate to have had Prof. Zheng as my supervisor.

I would also like to thank Zeyu Zhang, Xingjian Leng and Saeed Razavi, who have given me so much support, encouragement and kindness along the way. Studying together with them in the group has made my time there a real pleasure.

In addition, I am deeply thankful to Zhanhao Liang, who listened to my progress reports and discussed the project with me throughout the year. These discussions taught me to look beyond whether a method works and to ask why it works. Above all, I am grateful for Zhanhao's patience and encouragement, which made this journey so much easier.

Finally, I would like to thank my family for their love and unwavering support throughout my studies. Their encouragement and belief in me have always been a source of strength, and knowing they were behind me made every challenge easier to face. I could not have come this far without them.

I also acknowledge the use of Claude from Anthropic and ChatGPT from OpenAI to help edit and polish the writing of this thesis.

Abstract

In actor–critic reinforcement learning, the actor chooses actions and the critic estimates how good they are. When an agent learns a new skill late in a task, the earlier choices that lead to this skill become better. The critic, however, updates its estimates slowly, one step at a time. For a while it still tells the actor that these choices are worse than usual, so the agent can learn a skill and yet use it less. We call this delay *capability-to-choice lag* and study it through the signal the actor actually receives: how each action compares with the others. We propose Policy Differential Revaluation (PDR). Using a transition model, PDR computes how much each action's value has changed since a saved earlier policy, and gives the corrected values to the actor right away. It also splits the actor's signal into two parts: what the earlier values already preferred, and what has changed since. On three public benchmarks, KeyCorridor, MiniGridLoCA and DeepSea, PDR, with the weight on each part tuned per task, adapts faster than standard TD, prioritized experience replay, DisCor, 8× TD, IQL, CQL and prioritized sweeping in nearly every setting. We then use PDR to change one part of the signal at a time, and ask when the corrected signal arrives, which part of it matters, and how its two parts should be combined. We find three things. First, the critic's delay comes from the order of its updates: each one-step estimate waits for the next one, and updating later states first removes almost all of the delay's growth with distance. Second, the actor only uses differences between actions: removing 98.6% of a critic's error does not help if that error is the same for every action, yet in our experiments this is the error the critic fixes first. Third, both parts of the signal matter: removing either one stops learning in one of the benchmarks. Together, these results show that fast adaptation depends on giving the actor an accurate comparison between actions early, and that a critic should be judged by this signal, not only by its overall error.

Code will be released at https://github.com/Hanning99/COMP8800_Research_Project, and the raw results and saved checkpoints for the four benchmark experiments will be released at https://huggingface.co/datasets/denghanning/COMP8800_Research_Project to support reproducibility.

Table of Contents

1. Introduction **1**
- 1.1. Motivation . . . 1
- 1.2. Research Questions . . . 2
- 1.3. Proposed Approach . . . 2
- 1.4. Contributions . . . 3

2. Background **4**
- 2.1. Markov Decision Processes . . . 4
- 2.2. Returns and Value Functions . . . 4
- 2.3. Actor–Critic Methods and Temporal-Difference Learning . . . 5
- 2.4. Advantages and Policy Gradients . . . 5
- 2.5. Experience Replay and Planning . . . 6
- 2.6. Deep Reinforcement Learning . . . 6
- 2.7. The Actor-Facing Signal . . . 6
- 2.8. Policy Change and Critic Staleness . . . 7

3. Related Work **8**
- 3.1. Critic Correction and Regularization . . . 8
- 3.2. Advantage Estimation . . . 9
- 3.3. Policy Evaluation after a Policy Change . . . 9
- 3.4. Planning and Value Propagation . . . 10
- 3.5. Value Accuracy and Task Performance . . . 10
- 3.6. Adaptation to Local Changes . . . 11
- 3.7. Comparison with the Closest Approaches . . . 11

4. Motivating Experiment **13**
- 4.1. Task and Experimental Setup . . . 13
- 4.2. Observed Capability-to-Choice Lag . . . 14
- 4.3. Critic Repair under a Fixed Actor . . . 16
- 4.4. Exact-Signal Intervention . . . 17
- 4.5. Summary and Open Questions . . . 18

5. Methodology **19**
- 5.1. Problem Formulation . . . 19
- 5.2. Policy Evaluation Relative to a Reference . . . 20
  - 5.2.1. Value Differential and Source . . . 20

5.2.2. Fixed Point and Equivalence to Policy Evaluation . . . 21
5.3. Comparison with TD Propagation . . . 22
5.4. Learned References and Empirical Models . . . 22
5.4.1. Learned Reference . . . 23
5.4.2. Empirical Dependency Model and Bellman Closure . . . 23
5.5. Decomposition of the Actor Signal . . . 24
5.5.1. Reference and Change Components . . . 24
5.5.2. Actor Objectives . . . 25
5.5.3. Component Weighting . . . 25
5.6. Extensions . . . 26
5.7. Implementation . . . 26

6. Evaluation **29**
6.1. Benchmark Set . . . 29
6.2. Evaluated Methods . . . 30
6.3. Experimental Protocol . . . 30
6.4. Empirical Results . . . 32
6.4.1. KeyCorridor . . . 32
6.4.2. MiniGridLoCA . . . 34
6.4.3. DeepSea . . . 34
6.4.4. SecureCourier (Supplementary) . . . 35
6.5. Summary . . . 36

7. Analysis of Signal Timing **37**
7.1. Controlled Propagation Task . . . 37
7.2. Effect of Bellman Distance . . . 38
7.3. Effect of Update Order . . . 39
7.4. Partial Corrections and Signal Sign . . . 40
7.5. Effect on Actor Updates . . . 41
7.6. Relation to Neural Learners . . . 42
7.7. Summary . . . 42

8. Analysis of Signal Content **43**
8.1. Common-Shift Control . . . 43
8.2. Error Decomposition of Learning Critics . . . 45
8.3. Actor-Input Interventions . . . 46
8.4. Signal Alignment and Policy Change . . . 46
8.5. Value Accuracy versus Behaviour . . . 48
8.6. Summary . . . 48

9. Analysis of Signal Composition **49**
9.1. Component Ablation . . . 49
9.2. Coefficient Ratio and Scale . . . 51
9.3. Summary . . . 51

10. Discussion **52**
10.1. The Lag Is a Signal Problem . . . 52
10.2. PDR as Correction and Instrument . . . 52
10.3. What the Actor Needs . . . 53
10.4. When a Reweighted Signal Beats the Exact One . . . 54
10.5. Evaluating Critics by Their Actor Signal . . . 55
10.6. Limitations . . . 55

11. Concluding Remarks **56**
11.1. Conclusion . . . 56
11.2. Future Work . . . 57

A. PDR Implementation Details **59**
A.1. Graph Construction and Batch Lookup . . . 59
A.2. Indexing Example and Cached Inputs . . . 60
A.3. State Reduction, Streaming and Costs . . . 61
A.4. Factorized Query and Validation Records . . . 63
A.5. Optimizing Repeated Revaluation . . . 64
A.6. Timing Boundaries . . . 64

B. Experimental Details and Complete Results **65**
B.1. Actor, Critic and Solver Configurations . . . 65
B.2. Complete Public-Benchmark Results . . . 66
B.3. Relationships between Experimental Studies . . . 67
B.4. Protocols of the Full-Revaluation Studies . . . 68
B.5. Critic-Baseline Configurations . . . 71

C. Additional Results on SecureCourier **73**
C.1. Training Weights and Critic Repair . . . 73
C.2. Replay Support and Critic Repair . . . 74
C.3. Full-Batch Revaluation Results . . . 75
C.4. Weight and Policy-Response Diagnostics . . . 79
C.5. Critic-Training Baselines . . . 81
C.6. Gate and Delivery States . . . 83

D. Additional Results on Signal Timing **84**
D.1. Synchronous Propagation and Distance Convention . . . 84
D.2. Sensitivity to Graph Size and Response Threshold . . . 85
D.3. Actor-Response Controls . . . 85
D.4. Longer-Distance Actor Continuation . . . 86
D.5. Measurement Details . . . 86
D.6. Backup Order and Distance Dependence . . . 88

E. Additional Results on Signal Content **89**
E.1. Complete Common-Shift Control Results 89
E.2. Critic-Error Decomposition 90
E.3. Sensitivity to Reference and Transition Support 90

F. Additional Results on Signal Composition **96**
F.1. Ranking-Based Gates 96
F.2. Weight Mixtures 96

G. Computational Cost Analysis **98**
G.1. Cost Comparison with Critic Baselines 98
G.2. Baseline Retiming Protocol 100
G.3. Native versus Optimized PDR 102
G.4. Paired Benchmark Continuations 106
G.5. Benchmark Learning Curves 107
G.6. Training Timers and Benchmark Costs 107
G.7. Recorded Time Curves and Refresh Accounting 108
G.8. Independent SecureCourier Implementation Study 110
G.9. Locality, Source Discovery and Index Reuse 114
G.10. Additional Graph Computations and Scaling 114

Bibliography **117**

List of Figures **123**

List of Tables **125**

Chapter 1

Introduction

In this chapter, we describe the problem, state our research questions, outline our approach and list our contributions.

1.1. Motivation

Suppose a program can finish a task in two ways. The long way usually works. The short way often fails at a difficult step near the end, so the program learns to take the long way. Later, through practice, it learns to complete the difficult step. The short way is now the better choice, but the program keeps taking the long way for some time. We call this delay between learning a skill and using it in earlier decisions *capability-to-choice lag*. This thesis studies where the lag comes from and how to reduce it.

The program learns by *reinforcement learning*: it tries actions and learns to choose those that lead to more reward (Sutton and Barto, 2018). When the reward comes only at the end, early actions are judged by what happens later. When the program then gets better at a late step, the value of an early choice changes, even though the task has not.

We study *actor–critic* methods, which split learning into two parts. The *actor* chooses actions; its rule for doing so is the *policy*. The *critic* estimates the *value* of each action, that is, the reward expected after taking it. The critic learns by *temporal-difference* (TD) learning: it updates each estimate using the estimate of the next step (Sutton and Barto, 2018). As a result, a change near the end reaches early decisions only after the estimates in between have changed.

The actor does not use the critic's values directly. It compares each action with the policy's usual choice at the same state and favours the actions that compare well. We call this comparison the *signal* the actor receives. Only differences between actions affect the signal; an error that is the same for every action at a state has no effect on it.

Once the difficult step is learned, the short way's true signal turns positive. But the critic still holds the old estimate, so the actor keeps receiving a negative signal and keeps learning to avoid the short way. We call such a critic *stale*.

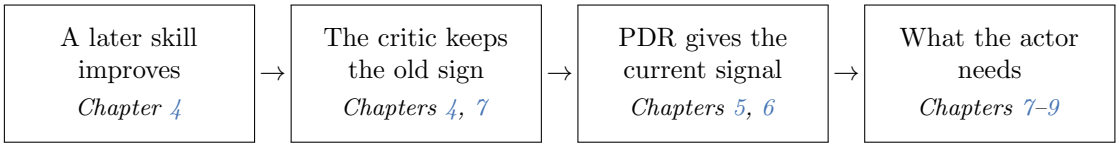


Figure 1.1.: The argument of the thesis. A later skill makes an earlier action better, but the critic keeps giving it the old sign, so the actor learns away from it. PDR gives the actor the current signal, and lets us test when it must arrive, which part matters, and how its two parts combine.

1.2. Research Questions

This thesis asks five questions:

- **RQ1 (phenomenon).** After a later skill improves, does the actor keep receiving a signal with the wrong sign, and does this slow its adaptation?
- **RQ2 (effectiveness).** Does giving the actor updated values improve adaptation across tasks and actor–critic methods?
- **RQ3 (timing).** What determines how long the corrected sign takes to reach an earlier decision?
- **RQ4 (content).** Does adaptation depend on value accuracy, or only on the signal derived from the values?
- **RQ5 (composition).** Does the actor need both the reference preference and the recent change?

1.3. Proposed Approach

To answer these questions, we propose *Policy Differential Revaluation* (PDR). PDR keeps a saved policy and its values as a *reference*. At each update, it solves for how the current policy's values differ from the reference, using a supplied transition model, and gives the updated values to the actor. The critic keeps learning. Unlike search-based methods, which plan to find better actions, PDR re-evaluates the agent's own choices after it has improved.

We prove that, with exact inputs, PDR gives exactly the values of standard policy evaluation. PDR differs only in how it writes the values, as a reference part plus a change. This form splits the actor's signal into the *reference preference*, which the saved values already supported, and the *recent change* since then.

PDR therefore plays two roles in this thesis. As a *reference correction*, it gives the actor the current policy's signal before the critic has learned it. Unlike existing methods,

which improve the critic, it changes only what the actor receives. As an *instrument*, it lets us change one part of the actor's signal at a time and observe the effect.

We compare PDR with seven critic-side methods under a common continuation protocol. PDR and prioritized sweeping use the supplied transition model; the other six baselines do not. The comparison therefore measures how far improving the critic is from correcting the actor's signal directly.

1.4. Contributions

Our analysis starts from two standard facts: a shift shared by all actions at a state leaves the actor updates we use unchanged (Lemma 1), and one-step TD carries a change back by at most one step per backup. We turn them into tools for measuring and controlling what the actor receives, and make four contributions (Figure 1.1):

- **A signal-level account of the lag** (Chapters 2 and 4). We trace capability-to-choice lag to the signal the actor receives, not to a missing skill or to the critic's overall value error. We turn the invariance above into a diagnostic that splits critic error into a common shift, which the actor objectives we use cannot see, and a relative error, which is the error in the actor's signal.
- **Policy Differential Revaluation** (Chapters 5 and 6). We propose PDR, a reference correction that changes only what the actor receives, and prove that, with exact inputs, it equals policy evaluation. Under a common protocol on three public benchmarks, PDR with per-task weights on its two parts has the highest early performance against seven critic-side methods in 10 of 11 settings.
- **Causal evidence on what the actor needs** (Chapters 7–9). Using PDR as an instrument, we show that a correction can reach an earlier action before it is large enough to flip its sign, and that in a controlled task, update order explains almost all of the delay. In the tasks we study, a TD critic first reduces the error the actor cannot see: in SecureCourier, Q error falls by 6.5% over the first 500 updates while the relative error rises by 7.5%. Changing only the actor's input from the same training state is enough to change the outcome, and each part of the signal is necessary in some task.
- **Signal-level evaluation of critics** (Chapters 8 and 10). Value error can fall while the signal stays wrong, so we propose judging a critic by its relative error, sign agreement and update alignment.

Chapter 2

Background

In this chapter, we introduce the concepts used in the thesis, define the signal the actor receives, and explain how this signal becomes stale.

2.1. Markov Decision Processes

An *agent* interacts with an *environment*. At each step, it observes a *state* s, chooses an *action* a from the allowed set $\mathcal{A}(s)$, and receives a *reward* and a next state s'. The agent's *policy* $\pi(a \mid s)$ is the probability of choosing a in s. An *episode* ends at a *terminal state*.

We assume the *Markov property*: the next outcome depends only on the current state and action. The task is then a *Markov decision process* (MDP) (Bellman, 1957),

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \tag{2.1}$$

where $P(s' \mid s, a)$ is the transition probability, $r(s, a)$ is the expected reward and γ is the discount factor.

2.2. Returns and Value Functions

The discounted *return* sums the rewards after step t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \qquad 0 \leq \gamma < 1. \tag{2.2}$$

The goal is a policy with high expected return.

The state value $V^\pi(s)$ is the expected return from s under π. The action value $Q^\pi(s, a)$ is the expected return after taking a in s and then following π (Sutton and Barto, 2018).

The state value is the policy-weighted average of the action values:

$$V^\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) Q^\pi(s, a). \tag{2.3}$$

The *Bellman equation* links an action value to the next state:

$$Q^\pi(s,a) = r(s,a) + \gamma\, \mathbb{E}_{S' \sim P(\cdot \mid s,a)} \left[m(s,a,S') V^\pi(S') \right], \tag{2.4}$$

where the *mask* m is zero at the end of an episode and one otherwise.

2.3. Actor–Critic Methods and Temporal-Difference Learning

An *actor–critic* method has two parts. The *actor* is the policy. The *critic* estimates values and uses them to train the actor (Barto et al., 1983).

The critic usually learns by *temporal-difference* (TD) learning (Sutton, 1988). For a transition (s, a, r_t, s', m_t), it forms the *target*

$$y_t = r_t + \gamma m_t \sum_{a' \in \mathcal{A}(s')} \pi(a' \mid s') \widehat{Q}(s', a'), \tag{2.5}$$

and moves its estimate towards it:

$$\widehat{Q}(s,a) \leftarrow \widehat{Q}(s,a) + \alpha \delta_t, \qquad \delta_t = y_t - \widehat{Q}(s,a), \qquad 0 < \alpha \leq 1. \tag{2.6}$$

The target uses the critic's own estimate of the next state. This is called *bootstrapping*. As a result, with this one-step target, a change at a later state moves back by at most one step per backup, through each *predecessor*: the state–action pair that leads to it. Multi-step returns, eligibility traces and function approximation can carry a change further.

2.4. Advantages and Policy Gradients

The critic performs *policy evaluation*. The actor performs *policy improvement*. To decide whether to favour an action, the actor uses its *advantage*:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s). \tag{2.7}$$

A positive advantage means the action is better than the policy's usual choice. A *policy-gradient* method changes the policy's *parameters* to increase return. It gives more weight to actions with higher advantage (Sutton et al., 1999a; Konda and Tsitsiklis, 1999). Because the policy keeps changing, the critic must keep learning its new values.

2.5. Experience Replay and Planning

An agent can store transitions in a *replay buffer* and train on them again in *mini-batches* (Lin, 1992). A stored transition keeps its reward and next state, but its TD target changes as the critic improves. A *model* predicts the reward and next state of an action. *Planning* uses a model to update values without new interaction (Sutton, 1991). Stored transitions form a graph whose nodes are state–action pairs; outcome counts give an *empirical model*.

2.6. Deep Reinforcement Learning

Large tasks need *function approximation*, such as a neural network. One update then changes predictions for many states. DQN combined neural value functions with replay (Mnih et al., 2015). We write $Q_\theta(s, a)$ for a neural critic trained on the squared TD loss $(Q_\theta(s, a) - y_t)^2$.

Soft Actor–Critic (SAC) adds a bonus for policy *entropy* (Haarnoja et al., 2018).

2.7. The Actor-Facing Signal

The actor does not use the critic's values directly. It compares each action with the others at the same state. Let $q(s, a)$ be the values given to the actor, for example $Q_\theta(s, a)$. The *actor-facing signal* is

$$A_q(s, a) = q(s, a) - \sum_{a' \in \mathcal{A}(s)} \pi(a' \mid s)\, q(s, a'). \tag{2.8}$$

With exact values, it equals the advantage. We use three actor objectives. In each, π_ϕ is the actor, q is held fixed, and $\mathrm{sg}(x)$ blocks gradients through x.

AWAC objective. Advantage-Weighted Actor–Critic trains the actor to repeat stored actions, weighted by their signal (Nair et al., 2020):

$$\begin{aligned} w(s, a) &= \min\left\{ \exp\left(\frac{A_q(s, a)}{\lambda} \right), w_{\max} \right\}, \\ L_{\mathrm{AWAC}}(\phi) &= -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\mathrm{sg}(w(s, a)) \log \pi_\phi(a \mid s) \right]. \end{aligned} \tag{2.9}$$

A positive signal gives a weight above one, a negative one a weight below one. The temperature λ sets how strongly the signal changes the weight, and $w_{\max}$ caps it.

SAC and QAC objectives. Discrete SAC minimizes $L_{\text{SAC}}(\phi) = \mathbb{E}_s \sum_a \pi_\phi(a \mid s)[\alpha_{\text{ent}} \log \pi_\phi(a \mid s) - q(s,a)]$ (Haarnoja et al., 2018). Expected-SARSA QAC minimizes $L_{\text{QAC}}(\phi) = -\mathbb{E}_s \sum_a \text{sg}(\pi_\phi(a \mid s) A_q(s,a)) \log \pi_\phi(a \mid s)$.

Lemma 1 (Invariance to statewise shifts)**.** *Let $f : \mathcal{S} \to \mathbb{R}$ be bounded and $q'(s,a) = q(s,a) + f(s)$. Under the same policy π, $A_{q'} = A_q$. Consequently, AWAC weights and the actor gradients of L_{AWAC}, L_{QAC} and L_{SAC} are unchanged when q' replaces q.*

Proof. Since $\sum_{a'} \pi(a' \mid s) = 1$, centring removes $f(s)$. AWAC and QAC use q only through A_q. For SAC, the shift adds $-f(s)$ to the loss at s, which does not depend on ϕ. □

Let $\widehat{q}$ estimate Q^π, with error $e = \widehat{q} - Q^\pi$. We split the error into the *common shift* $\bar{e}$ and the *relative error* $\tilde{e}$:

$$\bar{e}(s) = \sum_a \pi(a \mid s)\, e(s,a), \qquad \tilde{e}(s,a) = e(s,a) - \bar{e}(s) = A_{\widehat{q}}(s,a) - A^\pi(s,a). \tag{2.10}$$

Corollary 1 (Visible and invisible error)**.** *At every state,*

$$\sum_a \pi(a \mid s)\, e(s,a)^2 = \bar{e}(s)^2 + \sum_a \pi(a \mid s)\, \tilde{e}(s,a)^2.$$

The common shift $\bar{e}$ has no effect on the actor. Only the relative error $\tilde{e}$, which is the error in the actor-facing signal, can change an actor update.

Proof. The cross term $2\bar{e}(s) \sum_a \pi(a \mid s)\tilde{e}(s,a)$ is zero by the definition of $\tilde{e}$. The second statement follows from Lemma 1. □

This holds for one actor update at one state; a common shift still enters the bootstrapped targets of predecessor states.

Signal sign and ranking. We call the sign of $A_q(s,a)$ the *signal sign*, and the order of $q(s,\cdot)$ the *ranking*. Scaling q by a positive constant keeps both; for AWAC it equals dividing λ by that constant.

2.8. Policy Change and Critic Staleness

The value of an old action depends on what the agent does next, so learning a later step makes the earlier actions that lead there more valuable. Recomputing values after such a change is *revaluation* (Russek et al., 2017). Until the critic's signal reflects the change, we call the critic *stale*. Only the relative part of staleness reaches the actor (Corollary 1). It is most harmful when it gives an improved action the wrong sign or ranking; with the correct sign, it can still give the action too little weight.

Chapter 3

Related Work

In this chapter, we review methods that improve the feedback an actor learns from. We group them by what they change (Table 3.1): the critic's values, the advantage estimate, the evaluation of a changed policy, or the order of value updates. We then review work on value accuracy and on local changes, and position this thesis.

Table 3.1.: Prior work grouped by the quantity it changes, and its relation to this thesis.

Work	**Changes**	**Relation to this thesis**
PER, DisCor, TD3, Retrace, V-trace, BRACE, IQL, CQL	critic values or targets	The actor still receives the critic's relative signal; several are baselines in Chapter 6
AWR, AWAC, GAE, DAE	advantage estimate	AWAC supplies the main actor objective; PDR changes the values its weights are computed from
LSTD, LSPI, TayPO, performance difference	evaluation of a changed policy	PDR is policy evaluation written relative to a reference
Prioritized sweeping, small backups, SR-Dyna, Reanalyse	propagation and refresh schedule	Successor readiness (Chapter 7) studies why a schedule delays the signal
Action gap, value and model accuracy studies	what accuracy measures	Chapter 8 measures which part of value error reaches the actor
LoCA, LoFo, partial models	data and models after a local change	MiniGridLoCA is one of the benchmarks

3.1. Critic Correction and Regularization

Most methods improve the critic itself. **Prioritized experience replay (PER)** samples records with large TD errors more often (Schaul et al., 2016). **DisCor** gives less weight to records whose targets are likely to be wrong (Kumar et al., 2020a). **TD3**

uses the smaller of two critic estimates and updates the actor less often (Fujimoto et al., 2018). 8× TD simply takes more critic updates per actor update.

Other methods correct the targets when data come from older policies. **Retrace** uses truncated importance ratios (Munos et al., 2016). **V-trace** corrects for policy lag in IMPALA (Espeholt et al., 2018). **BRACE**, a 2026 preprint, corrects stale critics in asynchronous language-model training (Zhao et al., 2026).

Offline methods regularize the critic. **IQL** fits an upper-expectile value and trains the actor by weighted regression (Kostrikov et al., 2022). **CQL** penalizes the values of actions outside the data (Kumar et al., 2020b).

These methods change what the critic learns, but the actor still receives whatever signal the critic gives. PER, DisCor, 8× TD, IQL and CQL are our baselines.

3.2. Advantage Estimation

AWR fits the actor to recorded actions weighted by an exponential of their advantage (Peng et al., 2019). **AWAC** keeps this weighting but uses an off-policy Q critic for the current policy (Nair et al., 2020). AWAC is our main actor objective. **GAE** estimates advantages from TD errors along a trajectory (Schulman et al., 2016). **DAE** learns an advantage function constrained to have zero mean under the policy (Pan et al., 2022; Pan and Schölkopf, 2024). This constraint reflects Lemma 1: a shift shared by all actions says nothing about which action is better.

The **dueling architecture** writes $Q = V + A$ with two network streams and removes the mean of A to make the split identifiable (Wang et al., 2016). This is close to our split of critic error into a common shift and a relative error (Corollary 1), with two differences. Dueling centres A with the uniform mean over actions, whereas the actor objectives we use centre with the policy's mean, so its A is not the advantage the actor receives. More importantly, both streams are still trained on one-step TD targets for Q, so after a capability change the advantage stream is relearned by bootstrapping like any critic. The split helps the critic generalize across actions; we use it to measure which part of the error the actor sees.

3.3. Policy Evaluation after a Policy Change

When the policy changes, its values can be recomputed from stored transitions. **LSTD** solves a linear system for a linear value function (Bradtke and Barto, 1996). **LSPI** repeats this for each new greedy policy (Lagoudakis and Parr, 2003). The **performance-difference identity** relates the change in return to the old policy's advantages (Kakade and Langford, 2002). **TayPO** approximates the value change by a series in the change of the policy (Tang et al., 2020).

PDR belongs to this group. With exact inputs, it computes the same values as direct policy evaluation (Proposition 1). Like TayPO, it writes the new values as a change from an old policy, but it solves for the change exactly. This form allows us to split the signal into a reference preference and a recent change.

3.4. Planning and Value Propagation

Planning methods choose which values to update and when. **Prioritized sweeping** updates the predecessors of changed states first (Moore and Atkeson, 1993). **Small backups** update one successor at a time (van Seijen and Sutton, 2013). **Successor representations** separate future visits from rewards (Dayan, 1993), and **SR-Dyna** updates them by replay (Russek et al., 2017). **Planning as replay** recomputes earlier updates with new targets (van Seijen and Sutton, 2015), and **MuZero Reanalyse** refreshes training targets with a learned model (Schrittwieser et al., 2021). **Expert Iteration** and **AlphaZero** train the policy on action distributions found by tree search (Anthony et al., 2017; Silver et al., 2018). Chapter 7 uses this idea to explain the critic's delay. Prioritized sweeping is also a baseline; it uses the same model as PDR but learns optimal rather than current-policy values.

3.5. Value Accuracy and Task Performance

Several studies show that an accurate quantity is not always a useful one. Bellemare et al. (2016) note that a greedy policy depends only on the order of action values, and propose operators that widen the gap between actions. Fujimoto et al. (2022) show that Bellman error is a poor proxy for value error. Ilyas et al. (2020) find that deep policy-gradient methods learn even though their value baselines are far from the true values. Lambert et al. (2020) show that a model's accuracy can be weakly related to control performance. In policy-gradient theory, Sutton et al. (1999a) show that a critic with features compatible with the policy, fitted by least squares, gives an exact policy gradient even though its values differ from the true ones, and Konda and Tsitsiklis (1999) show that the critic need only estimate the projection of the action value onto the subspace spanned by the policy's score features. Corollary 1 is a simple statewise case of this idea. What we add is a measurement: after a capability change, how much of a TD critic's error lies outside the part the actor uses, and how this changes during learning (Chapter 8). **Value-aware model learning** makes the same move one level up: it measures a model's error by its effect on value backups rather than by likelihood (Farahmand et al., 2017). We measure a critic's error by its effect on the actor's signal. VAML changes the training loss; we use the measure to evaluate critics and leave training on it to future work (Section 11.2).

3.6. Adaptation to Local Changes

LoCA tests whether an agent that sees a reward change in one region changes its choices elsewhere (van Seijen et al., 2020). A revised version measures return during the local phase (Wan et al., 2022). **LoFo** replaces outdated local records while keeping the rest (Rahimi-Kalahroudi et al., 2023). **Partial Models** update only the affected part of a model (Alver et al., 2025). We use MiniGridLoCA as a benchmark.

Successor features with generalized policy improvement handle a class of such changes directly (Barreto et al., 2017). If rewards are linear in known features, $r = \phi^\top w$, each stored policy's values are ${\psi^\pi}^\top w$, so a change of w revalues every stored policy at once, and GPI acts greedily over them. This covers a reward change with fixed policies. The lag we study comes from a change of the agent's own policy, which changes ψ^π itself; $\psi^{\pi_{\text{new}}}$ is then learned by one-step TD and inherits the same delay. In MiniGridLoCA, PDR's source contains both terms: the reward change $r_B - r_A$ and the change of policy (Section B.4). The two ideas are complementary: successor features revalue a fixed policy under new rewards, and PDR revalues a new policy relative to an old one.

3.7. Comparison with the Closest Approaches

Planning that trains the policy. Expert Iteration and AlphaZero train the policy on search results, and Dyna uses a model to improve the value function (Sutton, 1991): they plan to find better actions or better values. PDR addresses a different situation. The agent has already improved, but its earlier choices are still judged by values from before. PDR does not search; it computes only the value change caused by the agent's own improvement and hands it to the actor, split into what the old values preferred and what has changed.

Direct advantage estimation. DAE estimates the advantage directly from returns, constrained to average to zero under the policy, rather than as the difference of separately learned action and state values (Pan et al., 2022). Its motivation overlaps with ours: the policy update uses the advantage, not the value level, and value targets move as the policy changes. The two approaches act at different points, however. DAE changes what the critic estimates, and after a capability change its advantage must still be relearned from new returns. We keep a standard critic, measure how its relative error evolves, and intervene on the actor's input directly. DAE's zero-mean constraint and the centring in Lemma 1 express the same property, which we use as a diagnostic rather than a training objective.

Prioritized sweeping and successor-first updates. Prioritized sweeping already orders backups so that a change reaches its predecessors quickly (Moore and Atkeson, 1993). Chapter 7 gives direct causal support for this premise: in our controlled task, update order alone explains almost all of the distance effect. It also goes further, by measuring the delay where it matters, in the actor's signal: a correction can reach an earlier action before it is large enough to flip the action's sign, and until it does, each actor update moves away from the better action. We include prioritized sweeping as a baseline. It is the strongest baseline on KeyCorridor, but it propagates optimal values, whereas the actor objectives we use expect the current policy's values.

Multi-step targets. n-step returns and eligibility traces let one backup carry a change back several steps (Sutton and Barto, 2018). They shorten the delay of Chapter 7 but keep its dependence on distance: a change still needs about h/n backups to travel h steps. Our setting adds a second obstacle. The replay was collected by older policies, so a multi-step target for the current policy needs off-policy corrections, and methods such as Retrace cut the trace where the current policy departs from the data (Munos et al., 2016), which is where a new capability makes it depart. PDR takes successors from the model rather than from sampled returns, so one solve reaches every step however the data were collected. The truncated series of Chapter 7 gives a reference point for any method that has carried a correction k steps: the actor receives roughly $D^{(k)}$, whose sign is still wrong until k is close to the full distance.

Policy evaluation and PDR. With exact inputs, PDR and direct policy evaluation have the same solution (Proposition 1) and give the same actor updates (Chapter 8). What distinguishes PDR is its form. It solves for the change rather than the values, and the change has a sparse source: it is nonzero only where the policy has changed. The solve can therefore be restricted to the predecessors of that change. With a 994-pair influence region in a graph of 994,001 pairs, selective PDR reads 1,992 entries per event instead of the whole graph, once a predecessor index exists (Appendix G.9); a local evaluator that knows the changed region does as well, so this saving comes from evaluating the change, which PDR makes explicit. Writing the values as a reference plus a change gives access to the parts of the change, which a direct solve hides: it locates where a policy change first alters value, it gives a series that can be truncated to study timing (Chapter 7), and it splits the actor's signal into a reference preference and a recent change that can be removed or reweighted separately (Chapters 8 and 9).

Positioning. Prior work improves values, advantages or update schedules. We study the signal the actor receives: when it arrives, which part of it matters, and how to correct it with policy evaluation relative to a reference.

Chapter 4

Motivating Experiment

In this chapter, we show capability-to-choice lag in a controlled task and trace it to the signal the actor receives (RQ1). We measure four things separately: the skill, the exact value of the earlier choice, the signal the actor is trained on, and the choice itself.

A courier stands at a junction with a parcel. One route goes through the corridors to the delivery box. The other is a shortcut through a security gate. The courier has a badge and a code but cannot yet open the gate, so it takes the long route. We built this task, *SecureCourier*, to see what happens once the courier learns to open the gate.

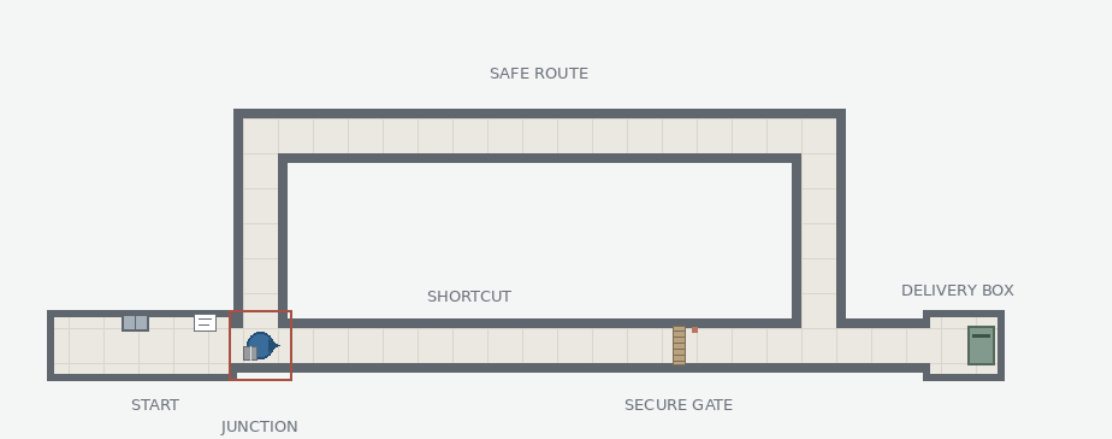


Figure 4.1.: The two routes in SecureCourier, shown with an added 2D display of the text environment. After eight actions, the courier (blue) reaches the junction (red box) with the parcel, badge and access code. Route lengths include the shared start and delivery stages. Room positions are arranged for display; several task stages share a room.

4.1. Task and Experimental Setup

The journey takes 27 actions by the safe route and 22 by the shortcut (Figure 4.1), with a budget of 40. Only the delivery earns a reward, discounted by 0.97 per step, so faster delivery earns more. The gate lies several corridors past the junction, so the courier

must choose the shortcut before it can use the new skill. Figures 4.4 and 4.5 show the shortcut and its three gate actions.

The actor encodes the observation and each candidate action separately and scores the action with a linear layer on the two encodings. This score is additive, so the observation term cancels in the softmax (Appendix A.4): the actor's preference depends on which actions are available and on their text, and is shared by every state that offers the same action. The lag therefore appears even when a single preference is shared across states. The public-benchmark and controlled experiments use actors that condition on the full state, and show the same effect of a stale signal.

4.2. Observed Capability-to-Choice Lag

We train the gate skill by guiding most journeys into the shortcut and supplying the correct gate actions. The actor and critic both learn from these records. Gate practice raises gate success from 0% to 96.6%. Yet over the next 1,024 learner cycles, the probability of choosing the shortcut falls from 18.9% to 13.5% (Figure 4.2). Each cycle collects one assisted episode and updates the critic and actor once.

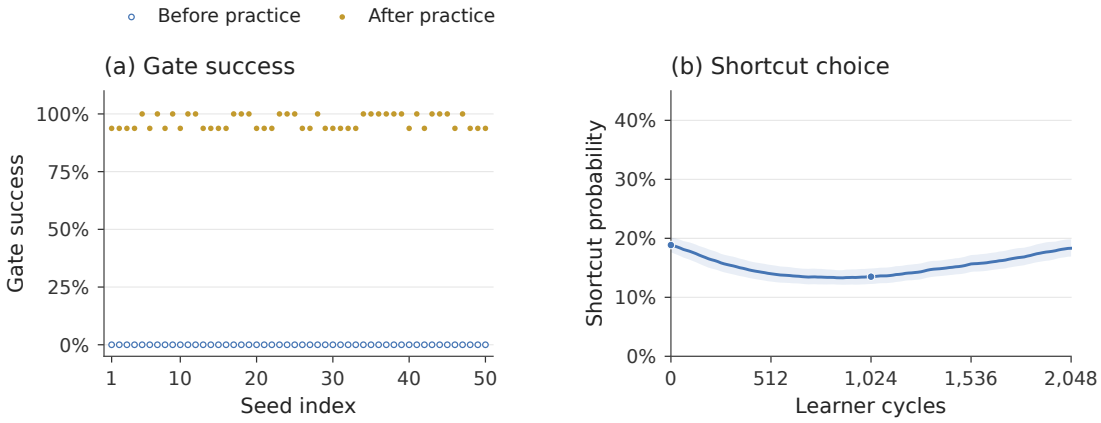


Figure 4.2.: (a) Greedy gate success on 16 text variants per seed, starting at the gate. Points show all 50 seeds; mean success rises from 0% to 96.6% after practice. (b) The line and band show the 50-seed mean and 95% bootstrap confidence interval, recorded every 16 cycles; each seed averages 16 variants. Mean shortcut probability is 18.9% at cycle zero and 13.5% at cycle 1,024. Cycle zero is the checkpoint after gate practice.

All results use 50 seeds. After gate practice, the courier can open the gate on its own, and it keeps its safe-route skill. The shortcut is now the better route. Using the task's known rules, we compute each route's exact expected return under the current policy. Gate practice raises the shortcut's return from 0.211 to 0.656, above the safe route's

0.578 (Figure 4.3). The skill is therefore already useful, yet the choice moves the other way.

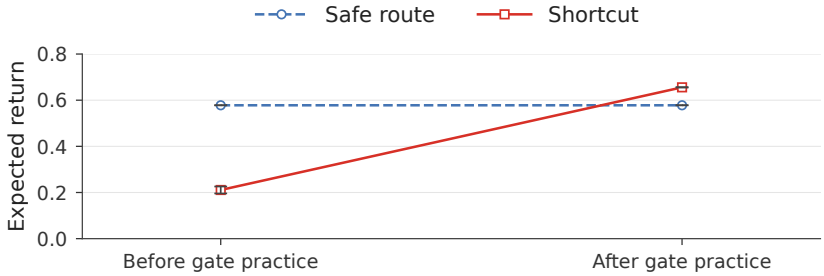


Figure 4.3.: Points show mean exact junction returns over 50 seeds: safe 0.578 at both checkpoints; shortcut 0.211 before gate practice and 0.656 after. Whiskers show 95% bootstrap confidence intervals.

To find the cause, we look at the actor's training weights. The actor is trained with AWAC (Nair et al., 2020), which weights each stored action by its signal (Section 2.7). At the checkpoint, the old shortcut records get a mean weight of 0.62. The same rule applied to exact values gives 1.24, which we call the *exact weight*. The actor therefore learns from these records with about half the weight it should.

The low weight comes from a wrong signal sign. Exact evaluation rates the shortcut above the policy's average choice in every instance, with a mean signal of +0.064. The critic rates it at or below average in every instance, with a mean signal of −0.145. The courier has learned the gate, but the signal at the junction still has the sign from before. The critic is *stale* (Section 2.8).[1] After 1,024 learner cycles, the critic's weight is still 0.861 against an exact weight of 1.300. The two meet only by cycle 2,048 (Table 4.1; Appendix C.1).

Table 4.1.: Shortcut training weights during actor–critic continuation: means and 95% bootstrap intervals over 50 seeds, each averaging 16 instances. Exact weights use the current policy at each checkpoint.

	Critic weight		Exact weight	
Learner cycle	Mean	95% CI	Mean	95% CI
0	0.621	[0.604, 0.639]	1.240	[1.233, 1.247]
1,024	0.861	[0.847, 0.875]	1.300	[1.293, 1.305]
2,048	1.287	[1.271, 1.304]	1.277	[1.270, 1.283]

[1]We count from exact evaluation rating the shortcut more than 0.02 above the policy average to the critic rating it above average. Each condition must hold for two consecutive evaluations. The median is 1,408 learner cycles across 50 seeds.

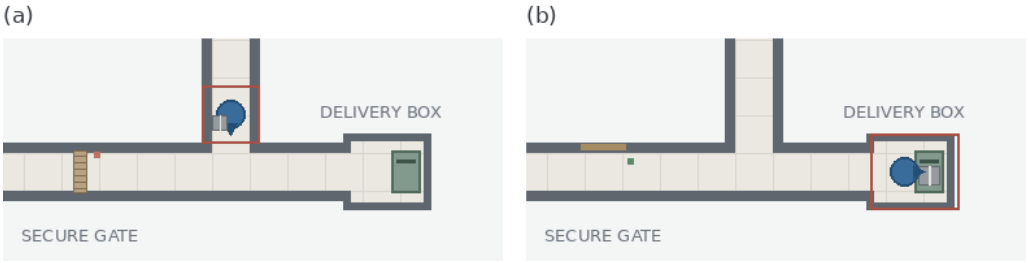


Figure 4.4.: Two journeys after the same 22 actions. (a) On the safe route, the courier still carries the parcel and has five actions left before delivery. (b) Along the shortcut, the parcel is already in the box and the courier has received the reward. Red boxes mark the two recorded positions. Both views show the same part of the map, using the environment's scripted route demonstrations.

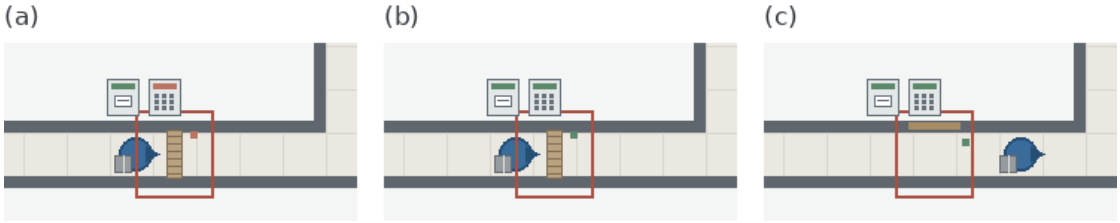


Figure 4.5.: The gate sequence, from the scripted demonstration in Figure 4.1. (a) Badge scanned, action 15. (b) Code entered, action 16. (c) Gate opened and courier through, action 17. The reader and keypad indicators turn green as their actions are completed; the gate opens only at the final action.

4.3. Critic Repair under a Fixed Actor

Why is the critic so slow? It learns by TD, so each estimate is trained towards the estimate of the next state. Taking the shortcut gives no immediate reward, so the junction's target is 0.97 times the value of the first corridor. If that value is still low, so is the junction's target. A better estimate near the gate reaches the junction only after the states in between are updated (Figure 4.6).

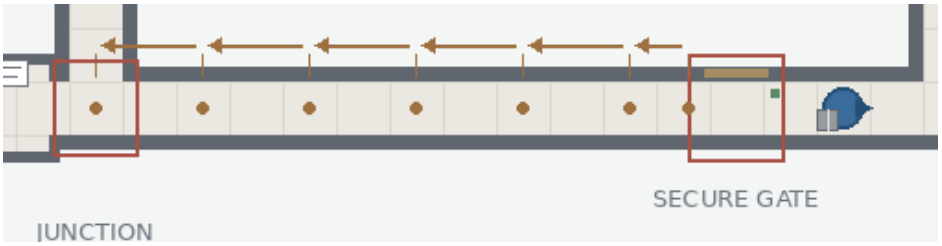


Figure 4.6.: How the next-state estimate enters an earlier TD target. The courier has passed the gate. Dots mark the states along the shortcut from the junction to the gate lobby; each brown arrow points from a successor to the state whose target uses its value. These transitions give zero reward, so each target is 0.97 times the next-state value.

To measure this delay, we freeze the actor at the checkpoint and train only the critic on the stored replay. The true value of the shortcut stays at 0.656. The critic's estimate starts at 0.139, reaches only 0.283 after 1,000 updates, and reaches 0.645 after 4,530 (Table 4.2).

Table 4.2.: Junction shortcut Q with the actor fixed: means and 95% bootstrap intervals over 50 seeds, each averaging 16 instances.

	Critic Q		Exact Q	
Critic updates	Mean	95% CI	Mean	95% CI
0	0.139	[0.126, 0.153]	0.656	[0.655, 0.657]
1,000	0.283	[0.274, 0.292]	0.656	[0.655, 0.657]
4,530	0.645	[0.640, 0.651]	0.656	[0.655, 0.657]

During the first 1,000 updates, the signal stays wrong in every instance (Section 8.2; Appendix C.2).

4.4. Exact-Signal Intervention

Does the stale signal cause the lag? *Standard* trains the actor with the critic's weights. *Corrected weights* uses exact weights for the stale shortcut records and changes nothing else. The correction runs for 1,024 cycles, followed by 1,024 ordinary cycles.

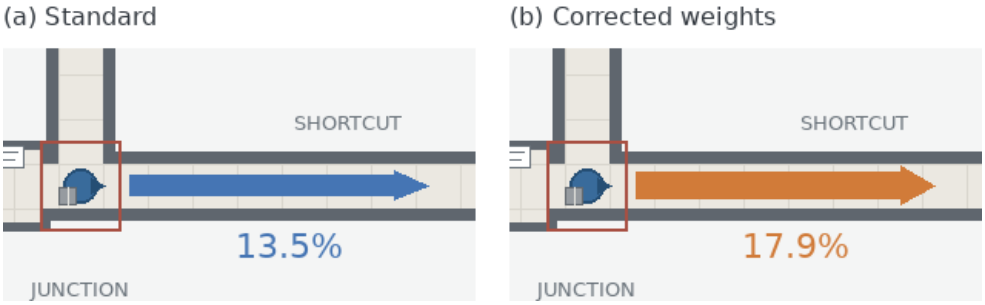


Figure 4.7.: Shortcut choice after 1,024 learner cycles: (a) Standard, 13.5%; (b) corrected weights, 17.9%. The same junction view carries the 50-seed mean probabilities; arrow widths share a linear scale. This branch leads to the 22-action route shown in Figure 4.4.

With exact weights, shortcut choice falls less (Figure 4.7), and the corrected courier earns more (Figure 4.8).

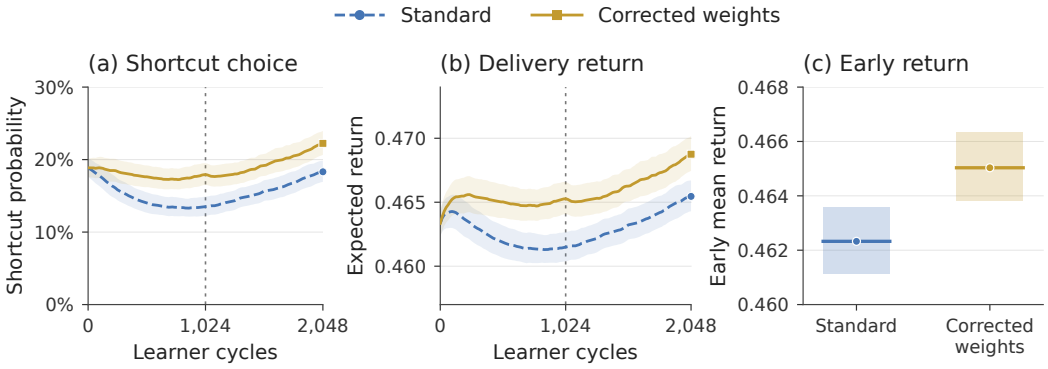


Figure 4.8.: Exact weights improve route choice and delivery return. (a,b) Means over 50 seeds with 95% bootstrap bands; the dashed line ends the correction. (c) Mean return over cycles 0–1,024 (Standard 0.46233, corrected 0.46503).

Over cycles 0–1,024, the corrected courier earns more in all 50 seeds (+0.00271, 95% CI $[0.00238, 0.00306]$). Since only the signal on the stale records changed, the stale signal is one cause of the lag.

4.5. Summary and Open Questions

This test used exact values on only some records. The rest of the thesis asks how to compute updated values for every record (Chapters 5–6), why the correction is slow (Chapter 7), which part of it the actor needs (Chapter 8), and how to balance the two parts (Chapter 9).

Chapter 5

Methodology

In this chapter, we propose Policy Differential Revaluation (PDR). PDR computes the current policy's action values as a change from a saved reference, using a supplied transition model. It gives these values to the actor while the critic keeps learning.

We prove that, with exact inputs, PDR gives the same values as standard policy evaluation (Proposition 1), so its values are correct by construction. Writing the values relative to the reference also shows where a policy change first alters value, how the change travels back to earlier decisions, and how the actor's signal splits into a reference preference and a recent change. This structure gives PDR its two roles: a correction that supplies current values (Chapter 6), and an instrument that changes one part of the signal at a time (Chapters 7–9).

Sections 5.1–5.3 derive PDR with exact inputs. Section 5.4 uses a learned reference and stored transitions. Section 5.5 defines the actor's signal and its two parts. Section 5.6 covers reward changes and soft values, and Section 5.7 describes the implementation.

5.1. Problem Formulation

When the courier learns, the task stays the same; only its choices change. Let π_{old} be a saved reference policy and π_{new} the current policy, both held fixed. For an earlier action, such as taking the shortcut, we want its change in value when the rest of the journey follows the new policy.

We use a discounted MDP

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \qquad 0 \le \gamma < 1, \tag{5.1}$$

with bounded reward $r(s, a)$ and allowed actions $\mathcal{A}(s)$. Policies are stationary and Markov. The mask $m(s, a, s')$ is zero on a terminal transition and one otherwise. For a bounded function $F(s, a)$, we define the next-step average and the Bellman operator

$$(\mathcal{P}^\pi F)(s, a) := \mathbb{E}_{S', A'} \left[m(s, a, S') F(S', A') \right], \qquad \mathcal{T}^\pi F := r + \gamma \mathcal{P}^\pi F. \tag{5.2}$$

where $S' \sim P(\cdot \mid s, a)$ and $A' \sim \pi(\cdot \mid S')$. The action value of π is the fixed point

$$Q^\pi = \mathcal{T}^\pi Q^\pi. \tag{5.3}$$

Both policies share P, r and γ. We first assume exact reference values,

$$Q_{\text{old}} := Q^{\pi_{\text{old}}}, \tag{5.4}$$

and relax this in Section 5.4.

5.2. Policy Evaluation Relative to a Reference

We first define the change that a policy update causes in each action value, and then show that solving for this change is policy evaluation.

5.2.1. Value Differential and Source

The *differential* D is the change in an action's value when the rest of the journey follows the new policy:

$$D(s, a) := Q^{\pi_{\text{new}}}(s, a) - Q_{\text{old}}(s, a). \tag{5.5}$$

Subtracting the two Bellman equations cancels the reward and gives

$$D = \gamma \left(\mathcal{P}^{\pi_{\text{new}}} - \mathcal{P}^{\pi_{\text{old}}}\right) Q_{\text{old}} + \gamma \mathcal{P}^{\pi_{\text{new}}} D. \tag{5.6}$$

The first term is known. We call it the *source*:

$$c := \gamma \left(\mathcal{P}^{\pi_{\text{new}}} - \mathcal{P}^{\pi_{\text{old}}}\right) Q_{\text{old}} = \mathcal{T}^{\pi_{\text{new}}} Q_{\text{old}} - Q_{\text{old}}. \tag{5.7}$$

It is the change made by one Bellman update that switches the policy but keeps the old values. The source starts from a local term. At each state, $b(s)$ compares the old values averaged under each policy:

$$b(s) := \mathbb{E}_{A \sim \pi_{\text{new}}(\cdot \mid s)} \left[Q_{\text{old}}(s, A)\right] - \mathbb{E}_{A \sim \pi_{\text{old}}(\cdot \mid s)} \left[Q_{\text{old}}(s, A)\right], \tag{5.8}$$

with $b(s) = 0$ at terminal states. For discrete actions,

$$b(s) = \sum_{a \in \mathcal{A}(s)} \left[\pi_{\text{new}}(a \mid s) - \pi_{\text{old}}(a \mid s)\right] Q_{\text{old}}(s, a). \tag{5.9}$$

At a gate state, $b(s)$ is positive when the new policy picks gate actions that the old values already rated highly. An earlier action receives the discounted average of b over its next states:

$$c(s, a) = \gamma \, \mathbb{E}_{S' \sim P(\cdot \mid s, a)} \left[m(s, a, S') b(S')\right]. \tag{5.10}$$

For the junction's shortcut action, c looks only one step ahead. The full change D also includes changes farther along the route.

5.2.2. Fixed Point and Equivalence to Policy Evaluation

Equation (5.6) is a fixed-point problem

$$D = c + \gamma \mathcal{P}^{\pi_{\text{new}}} D, \tag{5.11}$$

or, equivalently, the linear system

$$(I - \gamma \mathcal{P}^{\pi_{\text{new}}})\, D = c. \tag{5.12}$$

Proposition 1 (Equivalence to current-policy evaluation)**.** *Let $Q_{\text{old}} = Q^{\pi_{\text{old}}}$, use the exact masked transition operator, and let $0 \le \gamma < 1$. On the full domain, or on an exact Bellman-closed represented domain, the solution of Equation (5.12) satisfies*

$$Q_{\text{old}} + D = Q^{\pi_{\text{new}}}.$$

Proof. By Equation (5.7), $c = r + \gamma \mathcal{P}^{\pi_{\text{new}}} Q_{\text{old}} - Q_{\text{old}}$. Substituting into Equation (5.12) gives $(I - \gamma \mathcal{P}^{\pi_{\text{new}}})(Q_{\text{old}} + D) = r$, the current-policy Bellman equation. For $\gamma < 1$ it is a contraction, so its unique solution is $Q^{\pi_{\text{new}}}$. □

Full PDR is thus policy evaluation written relative to a reference. When we compare it with a live critic, both aim at the same values. Chapter 8 confirms that direct evaluation gives the same actor updates as full PDR.

The solution can be written as a series:

$$D = (I - \gamma \mathcal{P}^{\pi_{\text{new}}})^{-1} c = \sum_{k=0}^{\infty} (\gamma \mathcal{P}^{\pi_{\text{new}}})^k c, \tag{5.13}$$

$$D = c + \gamma \mathcal{P}^{\pi_{\text{new}}} c + \gamma^2 (\mathcal{P}^{\pi_{\text{new}}})^2 c + \cdots. \tag{5.14}$$

Each term looks one step further ahead. Grouping the terms by the state where the local change occurs gives

$$D(s,a) = \sum_{z \in \mathcal{S}} \text{Dep}_{\pi_{\text{new}}}((s,a), z)\, b(z), \tag{5.15}$$

$$\text{Dep}_{\pi_{\text{new}}}((s,a), z) := \sum_{d=1}^{\infty} \gamma^d \Pr_{P, \pi_{\text{new}}} \left(S_d = z,\ \prod_{j=1}^{d} m_j = 1 \,\middle|\, S_0 = s, A_0 = a \right), \tag{5.16}$$

where $m_j = m(S_{j-1}, A_{j-1}, S_j)$. A change at z affects an earlier action only along paths from that action to z. As a result, the junction's value can change even if the policy at the junction does not.

This form shows three things that a directly solved value hides: where the change starts (b and c), how it reaches earlier decisions (Equations (5.14)–(5.16)), and how the actor's signal splits into a reference preference and a recent change (Section 5.5).

5.3. Comparison with TD Propagation

TD builds up the same series one step at a time. Consider exact synchronous TD with the policy fixed, starting from the reference:

$$Q_{\mathrm{TD}}^{(0)} = Q_{\mathrm{old}}, \qquad Q_{\mathrm{TD}}^{(K+1)} = \mathcal{T}^{\pi_{\mathrm{new}}} Q_{\mathrm{TD}}^{(K)}, \qquad D_{\mathrm{TD}}^{(K)} := Q_{\mathrm{TD}}^{(K)} - Q_{\mathrm{old}}. \tag{5.17}$$

Subtracting Q_{old} from the update gives

$$D_{\mathrm{TD}}^{(K+1)} = c + \gamma \mathcal{P}^{\pi_{\mathrm{new}}} D_{\mathrm{TD}}^{(K)}, \qquad D_{\mathrm{TD}}^{(0)} = 0, \tag{5.18}$$

so by induction, after $K \geq 1$ sweeps,

$$D_{\mathrm{TD}}^{(K)} = \sum_{k=0}^{K-1} (\gamma \mathcal{P}^{\pi_{\mathrm{new}}})^k \, c. \tag{5.19}$$

With $D_{\mathrm{PDR}} := D$, the part not yet learned is

$$D_{\mathrm{PDR}} - D_{\mathrm{TD}}^{(K)} = \sum_{k=K}^{\infty} (\gamma \mathcal{P}^{\pi_{\mathrm{new}}})^k \, c = (\gamma \mathcal{P}^{\pi_{\mathrm{new}}})^K \, D_{\mathrm{PDR}}. \tag{5.20}$$

For example, on a chain where the only change is $b(z)$, d steps after an earlier pair $(s_{\mathrm{hist}}, a_{\mathrm{hist}})$,

$$D_{\mathrm{PDR}}(s_{\mathrm{hist}}, a_{\mathrm{hist}}) = \gamma^d b(z), \qquad D_{\mathrm{TD}}^{(K)}(s_{\mathrm{hist}}, a_{\mathrm{hist}}) = 0 \quad \text{for } K < d. \tag{5.21}$$

Until the correction arrives, the earlier pair keeps its old value. If that value ranked the action below the usual choice, the actor keeps receiving the old sign. Other schedules, such as sampled or prioritized replay, can be faster or slower. But all share one constraint: a predecessor cannot use a correction that its successor does not yet have.

Successor-readiness hypothesis. The delay before an earlier action gets a correct signal is set by when its successors are corrected. Updating successors first should therefore remove most of the effect of distance. Chapter 7 tests this by changing only the update order, and by truncating the series in Equation (5.14).

5.4. Learned References and Empirical Models

In practice, the reference values and the transitions are estimates. In SecureCourier, the reference is the critic saved with the old policy. In KeyCorridor and MiniGridLoCA, it is a network fitted to the old policy's values, computed by Bellman backups on the supplied graph; DeepSea uses exact values. The equations stay the same; only their inputs become estimates.

5.4.1. Learned Reference

Let $\overline{Q}_{\text{old}} \approx Q^{\pi_{\text{old}}}$ be the saved reference critic. Hats mark estimated quantities. The local change becomes

$$\widehat{b}(s) := \mathbb{E}_{A\sim\pi_{\text{new}}(\cdot|s)}\left[\overline{Q}_{\text{old}}(s,A)\right] - \mathbb{E}_{A\sim\pi_{\text{old}}(\cdot|s)}\left[\overline{Q}_{\text{old}}(s,A)\right], \tag{5.22}$$

with $\widehat{b}(s) = 0$ at terminal states.

5.4.2. Empirical Dependency Model and Bellman Closure

A finite record set $\mathcal{D}$ contains transitions. Its distinct state–action pairs form the nodes

$$\mathcal{X} = \{x_i = (s_i, a_i)\}_{i=1}^{N}. \tag{5.23}$$

Let $\widehat{P}_m(s' \mid s, a)$ be the empirical, masked next-state distribution. The current policy gives the dependency matrix

$$[\widehat{M}_{\text{new}}]_{ij} := \widehat{P}_m(s_j \mid s_i, a_i)\,\pi_{\text{new}}(a_j \mid s_j), \tag{5.24}$$

the estimated source

$$\widehat{c}_i := \gamma \sum_{s'} \widehat{P}_m(s' \mid s_i, a_i)\,\widehat{b}(s'), \tag{5.25}$$

and the system

$$\left(I_N - \gamma\widehat{M}_{\text{new}}\right)\widehat{\boldsymbol{D}} = \widehat{\boldsymbol{c}}. \tag{5.26}$$

For $\gamma < 1$, this system has a unique solution. Every next pair the policy can reach must be in the graph. We call this *Bellman closure*: for every $(s,a) \in \mathcal{X}$,

$$\left\{(s',a') : \widehat{P}_m(s' \mid s, a)\,\pi_{\text{new}}(a' \mid s') > 0\right\} \subseteq \mathcal{X}. \tag{5.27}$$

A missing pair removes its contribution. If the reference and the model are exact and closure holds, then $\widehat{D} = D$. Otherwise, their errors carry into the solution. In particular, Proposition 1 need not hold for a learned reference.

With an exact closed model, one could also evaluate the policy directly. All our experiments supply such a model: a simulator graph in SecureCourier and prepared models in the public benchmarks. PDR's role is to expose the change and deliver it to the actor while the critic keeps learning. Of the baselines, only prioritized sweeping also uses this model.

5.5. Decomposition of the Actor Signal

Adding the change to the reference gives the *revalued* values

$$Q_{\mathrm{PDR}}(s,a) := \overline{Q}_{\mathrm{old}}(s,a) + \widehat{D}(s,a), \qquad (s,a) \in \mathcal{X}. \tag{5.28}$$

The actor centres them under the current policy (Equation (2.8)). Every action the policy can take at an actor state must therefore be in $\mathcal{X}$.

5.5.1. Reference and Change Components

Centring is linear, so the signal splits into two parts:

$$A_{\mathrm{PDR}}(s,a) := Q_{\mathrm{PDR}}(s,a) - \mathbb{E}_{A\sim\pi_{\mathrm{new}}(\cdot|s)}\left[Q_{\mathrm{PDR}}(s,A)\right] = A_{\mathrm{ref}}(s,a) + \Delta A(s,a), \tag{5.29}$$

with

$$A_{\mathrm{ref}}(s,a) = \overline{Q}_{\mathrm{old}}(s,a) - \mathbb{E}_{A\sim\pi_{\mathrm{new}}}[\overline{Q}_{\mathrm{old}}(s,A)], \tag{5.30}$$

$$\Delta A(s,a) = \widehat{D}(s,a) - \mathbb{E}_{A\sim\pi_{\mathrm{new}}}[\widehat{D}(s,A)]. \tag{5.31}$$

We write the identity as

$$A_{\mathrm{full}} := A_{\mathrm{ref}} + \Delta A = A_{\mathrm{PDR}}. \tag{5.32}$$

We call A_{ref} the *reference preference*: how the saved values rank the current options. We call ΔA the *recent change*: which actions gained more value than the current average. Both use the current policy, so A_{ref} is not the old policy's own advantage.

At the level of values, let $\widehat{g}(s) := \mathbb{E}_{A\sim\pi_{\mathrm{new}}}[\widehat{D}(s,A)]$ and $\widehat{\delta} := \widehat{D} - \widehat{g}$. Then

$$Q_{\mathrm{PDR}} = \overline{Q}_{\mathrm{old}} + \widehat{g} + \widehat{\delta}, \qquad \Delta A = \widehat{\delta}. \tag{5.33}$$

By Lemma 1, the common shift $\widehat{g}$ does not affect the actor. All that revaluation adds to the actor is $\widehat{\delta}$.

This split also describes staleness. Suppose the critic still equals an exact reference, $Q_\theta = Q_{\mathrm{old}}$, but the policy has become π_{new}. Its error is $e = -D$. By Corollary 1, its invisible error is the policy average of $-D$, and its relative error is $-\Delta A$. A stale critic therefore supplies A_{ref} where the actor needs $A_{\mathrm{ref}} + \Delta A$; the harm to the actor depends on ΔA, not on the size of the common shift.

Table 5.1 lists the actor inputs used in later chapters. All use the same reference and the same solved change.

Table 5.1.: Actor inputs built from one reference and one solved change. The signal column follows from Equations (5.29)–(5.33) and Lemma 1. Direct PE equals full PDR under the conditions of Proposition 1.

Condition	Actor input q	Actor signal A_q	Used in
Live critic (TD)	Q_θ	critic advantage	all
Reference	$\overline{Q}_{\text{old}}$	A_{ref}	Ch. 8
Common-only	$\overline{Q}_{\text{old}} + \widehat{g}$	A_{ref}	Ch. 8
Relative-only	$\overline{Q}_{\text{old}} + \widehat{\delta}$	$A_{\text{ref}} + \Delta A$	Ch. 8
Reversed	$\overline{Q}_{\text{old}} + \widehat{g} - \widehat{\delta}$	$A_{\text{ref}} - \Delta A$	Ch. 8
Full PDR	$\overline{Q}_{\text{old}} + \widehat{D}$	$A_{\text{ref}} + \Delta A$	Chs. 6–8
Direct PE	$Q^{\pi_{\text{new}}}$ on the model	$A_{\text{ref}} + \Delta A$	Ch. 8
Weighted PDR	$\alpha\overline{Q}_{\text{old}} + \beta\widehat{D}$	$\alpha A_{\text{ref}} + \beta\Delta A$	Chs. 6, 9

5.5.2. Actor Objectives

PDR changes only the actor's input q, not its objective. For AWAC, the weight is

$$w_{\text{PDR}}(s,a) := \min\left\{\exp\left(\frac{A_{\text{PDR}}(s,a)}{\lambda}\right), w_{\max}\right\}, \tag{5.34}$$

and the actor loss is

$$\mathcal{L}^{\text{PDR}}_{\text{actor}}(\phi) := -\mathbb{E}_{(s,a)\sim\mu_{\mathcal{D}}}\left[\text{sg}\left(w_{\text{PDR}}(s,a)\right)\log\pi_\phi(a \mid s)\right], \tag{5.35}$$

where $\mu_{\mathcal{D}}$ is the distribution of training records. The whole weight is detached. SAC and QAC use $q = Q_{\text{PDR}}$ in the objectives of Section 2.7. The critic keeps its own TD objective; Q_{PDR} never enters a critic target. If the shortcut gained more value than the average action, ΔA is positive and raises its weight at once, without waiting for the critic.

5.5.3. Component Weighting

Since the two parts are computed separately, they can also be weighted separately:

$$A_{\text{update}} = \alpha A_{\text{ref}} + \beta\Delta A, \qquad \alpha, \beta \geq 0. \tag{5.36}$$

This is the centred form of $Q_{\text{actor}} = \alpha\overline{Q}_{\text{old}} + \beta\widehat{D}$. The choice $(\alpha, \beta) = (1, 1)$ is full revaluation. Other choices no longer estimate $Q^{\pi_{\text{new}}}$, and they come with no improvement guarantee. A common scale (k, k) keeps the sign and ranking; for AWAC it equals dividing the temperature by k:

$$\exp\left(\frac{kA}{\lambda}\right) = \exp\left(\frac{A}{\lambda/k}\right), \qquad k > 0. \tag{5.37}$$

Changing the ratio β/α changes the mix of the two parts and can change the ranking. For SAC, scaling the value term also changes its balance with entropy. Chapter 9 tests both parts by removing one at a time and by varying their ratio.

5.6. Extensions

So far, rewards and dynamics were fixed. MiniGridLoCA changes the rewards, so its source also contains $r_{\text{new}} - r_{\text{old}}$. For soft values with entropy H and coefficient α_{ent}, the source becomes

$$\begin{aligned} c_{\text{ext}} = r_{\text{new}} - r_{\text{old}} + \gamma(\mathcal{P}^{\pi_{\text{new}}} - \mathcal{P}^{\pi_{\text{old}}})Q_{\text{old}} \\ + \gamma\alpha_{\text{ent}} P_m(H_{\text{new}} - H_{\text{old}}), \end{aligned} \tag{5.38}$$

where P_m averages over next states with the mask. The same solver uses this source (array form in Equation (A.5)).

DeepSea uses $\gamma = 1$, which the contraction argument does not cover. Its graph is finite and acyclic, so $M^H = 0$ for horizon H and $(I - M)^{-1} = \sum_{j=0}^{H-1} M^j$. A single backward pass over the layers solves it.

5.7. Implementation

The implementation keeps two value sources apart: the critic, trained as usual, and PDR, recomputed from the current policy, the reference and the model (Figure 5.1). Appendix A gives the details.

Inputs. Each environment provides state keys, legal actions, network inputs and transitions (s, a, r, s', m). One data set, $\mathcal{D}_{\text{model}}$, builds the transition graph. Another, $\mathcal{D}_{\text{train}}$, supplies training batches. The graph and the reference $(\pi_{\text{old}}, \overline{Q}_{\text{old}})$ are prepared once and stay fixed. Closure is checked when the graph is built. Every batch is checked for missing states, and training stops if one is found.

Source and solve. With pairs i, parent states u, legal-action groups I_u and recorded outcomes $\mathcal{O}_i$ (Appendix A.2), the array form of Equations (5.22) and (5.25) is

$$\widehat{b}_u = \sum_{i \in I_u} (\pi_i^{\text{new}} - \pi_i^{\text{old}})\overline{q}_i, \qquad \widehat{c}_i = \gamma \sum_{(v,1,p) \in \mathcal{O}_i} p\, \widehat{b}_v, \tag{5.39}$$

Two solvers return $\widehat{D}$. For a layered graph, Algorithm 1 works backwards from the last layer; it also works at $\gamma = 1$. For a general closed graph, Algorithm 2 solves the sparse system directly. Both check the residual of Equation (A.7). Appendix A.3 gives a state-level version and a streaming kernel.

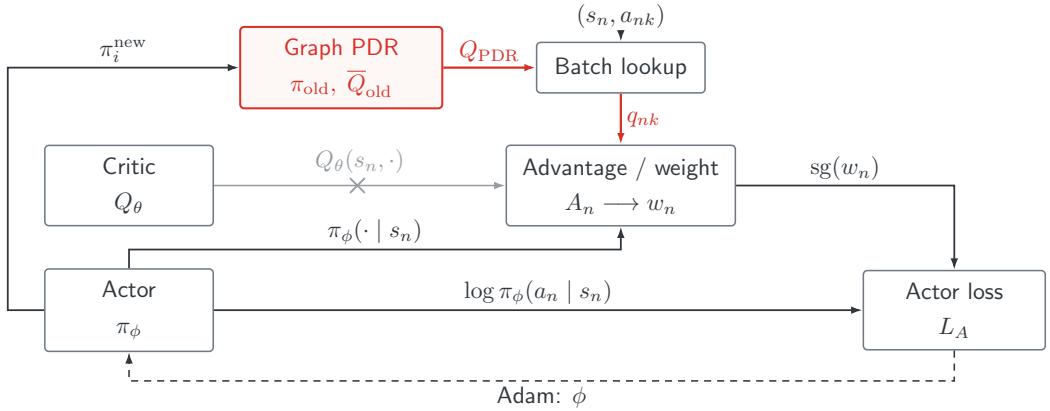


Figure 5.1.: One AWAC actor update after the critic's TD step (Algorithm 3). PDR returns Q_{PDR}; a batch lookup replaces the crossed Q_θ input to advantage and weight computation. Detached weights and the actor's log probabilities form the loss; the dashed arrow is the actor optimizer step. The graph and reference stay fixed.

Algorithm 1: Layered differential evaluation

Input: Validated deterministic DAG with groups I_u, layers h_i, h_u, successors ν_i and continuation masks m_i; aligned $\pi^{\text{new}}, \widehat{b}, \overline{q}$; $0 \le \gamma \le 1$, tolerance ϵ_D

Initialize $\widehat{D} \leftarrow 0 \in \mathbb{R}^N$, $g \leftarrow 0 \in \mathbb{R}^S$
for $h = H-1, \dots, 0$ **do**
  foreach *pair* i *with* $h_i = h$ *and* $m_i = 1$ **do**
    $\widehat{D}_i \leftarrow \gamma(\widehat{b}_{\nu_i} + g_{\nu_i})$ `// read the completed successor layer`
  foreach *state* u *with* $h_u = h$ **do**
    $g_u \leftarrow \sum_{i \in I_u} \pi_i^{\text{new}} \widehat{D}_i$
Require finite output and residual $\rho_\infty \le \epsilon_D$
return $Q_{\text{PDR}} = \overline{q} + \widehat{D}$

Algorithm 2: Sparse differential evaluation

Input: Groups I_u, outcomes $\mathcal{O}_i$, aligned $\pi^{\text{new}}, \widehat{c}, \overline{q}$; $0 \le \gamma < 1$, tolerance ϵ_D

foreach *pair* i *and continuing outcome* $(v, 1, p) \in \mathcal{O}_i$ **do**
  foreach $j \in I_v$ *with* $\pi_j^{\text{new}} > 0$ **do**
    Require covered pair j; append entry $(i, j, p\pi_j^{\text{new}})$
Assemble sparse $\widehat{M}_{\text{new}}$, summing duplicates; check nonnegative substochastic rows
$\widehat{D} \leftarrow \texttt{spsolve}(I_N - \gamma \widehat{M}_{\text{new}}, \widehat{c})$
Require finite output and residual $\rho_\infty \le \epsilon_D$
return $Q_{\text{PDR}} = \overline{q} + \widehat{D}$

Training loop. Algorithm 3 shows one training update for all three actor objectives. It updates the critic, solves for the change with the current actor, gathers the actor's input for the batch, and then updates the actor and the target critics. Each actor update uses a fresh solve.

Algorithm 3: Fixed-reference actor–critic training with PDR

Input: Actor ϕ; live critic(s) θ and targets θ^-; replay $\mathcal{D}_{\text{train}}$; validated graph; fixed reference $(\pi_{\text{old}}, \overline{Q}_{\text{old}})$; coefficients (α, β); backbone AWAC, SAC or QAC; active interval $\mathcal{I}$

for $t = 1, \ldots, T$ **do**
- Collect and insert records if online; sample a batch
- Update the live critic(s) with the backbone's native TD target (expected for AWAC/QAC; soft, twin-minimum for SAC)
- **if** $t \in \mathcal{I}$ **then**
  - Query π^{new} on the graph without gradients
  - Form $\widehat{c}$ (Equation (5.39), or Equation (A.5) with reward or entropy change)
  - Solve $\widehat{D}$ (Algorithm 1 or 2)
  - Require actor coverage; gather $q_{nk} \leftarrow \text{sg}(\alpha\overline{Q}_{\text{old}} + \beta\widehat{D})(s_n, a_{nk})$ for legal actions
- **else**
  - $q_{nk} \leftarrow \text{sg}(Q_\theta(s_n, a_{nk}))$
- Query $p_{nk} = \pi_\phi(a_{nk} \mid s_n)$; $A_{nk} \leftarrow q_{nk} - \sum_j \text{sg}(p_{nj}) q_{nj}$
- **if** *AWAC* **then**
  - $w_n \leftarrow \text{sg}[\min\{\exp(A_{n\alpha_n}/\lambda), w_{\max}\}]$; $L_A \leftarrow -B^{-1} \sum_n w_n \log p_{n\alpha_n}$
- **else if** *SAC* **then**
  - $L_A \leftarrow B^{-1} \sum_{n,k} p_{nk}(\alpha_{\text{ent}} \log p_{nk} - q_{nk})$
- **else**
  - $L_A \leftarrow -B^{-1} \sum_{n,k} \text{sg}(p_{nk} A_{nk}) \log p_{nk}$
- Update ϕ with L_A; soft-update the targets

Here α_n is the position of the recorded action in row n.

Configurations. Appendix B.1 lists the actor, reference, solver, critic and coefficients used in each task (Table B.2).

Chapter 6

Evaluation

In this chapter, we evaluate PDR as a reference correction (RQ2). All seven baselines improve the critic or its targets; PDR instead changes only the actor's input. Comparing them under a common continuation protocol, on three public benchmarks and three actor objectives, shows how far improving the critic is from correcting the actor's signal directly, here with PDR's two components weighted per task. These experiments show what PDR achieves. Chapters 7 to 9 explain why.

6.1. Benchmark Set

Every experiment continues training from saved checkpoints after a change: a new skill in SecureCourier and KeyCorridor, a reward switch in MiniGridLoCA, and a pretrained policy in DeepSea. All methods in a comparison start from the same networks, optimizer states and replay schedule. The public benchmarks train on fixed replay with a supplied closed model. SecureCourier collects one assisted episode per cycle.

- **KeyCorridor** (Chevalier-Boisvert et al., 2019). The agent must find a key, unlock a door and pick up a ball. We use S3R2, S3R3, S4R3 and S5R3 with full-state features and a fixed action mask. The metric is success probability.
- **MiniGridLoCA** (LoCA for short in tables, figures and appendices) (van Seijen et al., 2020; Rahimi-Kalahroudi et al., 2023). An 8×8 grid with two goals. After the switch, T_1 pays 1 instead of 4 and T_2 still pays 2, so the better goal changes. We use a state-ID version with 256 states. The metric is the exact 100-step discounted return under the new rewards ($\gamma = 0.99$). The starting policies still go to T_1 and score 0.918.
- **DeepSea** (Osband et al., 2020). The deterministic bsuite task at $N = 10, 20, 30, 40$. Each right move costs $0.01/N$, and the treasure pays $+1$, so the best return is 0.99. The metric is expected return.

6.2. Evaluated Methods

The seven baselines change the critic or its targets:

- **TD** trains the critic by TD; the actor uses its values.
- **PER** (Schaul et al., 2016) samples transitions with large TD error more often.
- **DisCor** (Kumar et al., 2020a) down-weights targets that are likely to be wrong.
- 8× **TD** takes eight critic updates per actor update.
- **IQL** (Kostrikov et al., 2022) uses an expectile value and a weighted-regression actor.
- **CQL** (Kumar et al., 2020b) adds a conservative penalty to the critic and uses a SAC actor.
- **PS + actor** (PS) (Moore and Atkeson, 1993) runs prioritized sweeping on the supplied model and trains the actor on the result.

TD, PER, DisCor, 8× TD and PS use the task's actor objective; IQL and CQL use their own actors. PS uses the model, like PDR, but learns optimal rather than current-policy values.

PDR uses the settings in Table B.2 (Appendix B.1). In DeepSea, PDR-SAC and CQL share the same actor and critic. They differ only in the actor's input: the critic's values or $\overline{Q}_{\text{old}} + 32\widehat{D}$. The other DeepSea baselines use AWAC actors.

6.3. Experimental Protocol

Experiment map. Table 6.1 lists the experiments in this thesis and what each is for. Only this chapter compares complete, tuned methods. The analysis chapters hold the training state fixed and change one input at a time, so each result isolates the input it changes.

Table 6.1.: Experiments in this thesis. Performance comparisons use complete, tuned methods; interventions change one input from a fixed training state.

Ch.	Setting	What changes	Purpose
4	SecureCourier	exact weights on selected records	lag and its cause
6	public benchmarks	PDR versus seven baselines	performance
7	controlled task	update order, truncation, pulse	delay mechanism
8	LoCA, SecureCourier	actor input; error split	which part matters
9	public benchmarks	weights on the two parts	role of each part
App. G	earlier configurations	implementation, timers	cost

Hyperparameter selection. The search over settings was not equal (Table B.3, Appendix B.1). Per task, TD, PER, DisCor and 8× TD tried two value scales, PS 6 settings, IQL 8–64, CQL 9–87 and PDR 7–274. PDR searched more settings than any baseline on KeyCorridor and DeepSea, and fewer than IQL and CQL on MiniGridLoCA, so its results include the benefit of a tuned weighting. On KeyCorridor and MiniGridLoCA, settings were selected on the 20 reported seeds, so the intervals are not held-out estimates; DeepSea used separate development seeds, and $N = 40$ reuses the $N = 30$ settings. The selected PDR settings mostly weight the two parts of the signal unequally (Table B.2), so this chapter evaluates the complete method, including its weighting; Chapter 9 isolates each part.

DeepSea starts. DeepSea starts come from Bootstrapped DQN pretraining. A start is eligible if pretraining produces, within its original budget of $5N^3$, a checkpoint whose expected return lies in $[0.05, 0.15]$. Eligibility is decided before any method is run, and failed attempts are not replaced: 20, 19, 13 and 13 eligible starts come from 20, 20, 20 and 100 attempts at $N = 10, 20, 30, 40$. The DeepSea results therefore hold for this population of low-return starts.

Budgets and metrics. Table 6.2 lists the budgets. The *early mean* is the area under the learning curve over the early window, divided by its length. *Final* is the value at the end. Learning curves count actor updates, not computation.

Table 6.2.: Continuation budgets, early windows and paired seeds. DeepSea seeds are the eligible starts from 20, 20, 20 and 100 prespecified attempts at $N = 10, 20, 30, 40$; failed initializations are not replaced.

Setting	Metric	Seeds	Early window	Budget
KeyCorridor	success probability	20 per task	128	512
MiniGridLoCA	Task-B discounted return	20 per actor	1,024	4,096
DeepSea	episode return	20/19/13/13	2,048	8,192

Statistical analysis. Bands and intervals are 95% seed-bootstrap intervals (10,000 resamples; 20,000 in SecureCourier). Paired contrasts resample seeds.

Information and computational cost. PDR uses the supplied model and a reference; of the baselines, only prioritized sweeping also uses the model. Full PDR with exact inputs supplies the current policy's signal (Proposition 1); the results here use PDR with task-specific component weighting, so they show what supplying a corrected signal achieves, not that PDR learns more from the same data. Computation is not matched; Appendix G reports timings.

6.4. Empirical Results

Table 6.3 summarizes the public-benchmark results; Appendix B.2 lists every method.

Table 6.3.: Early and final performance in the public-benchmark continuations. Early is normalized trapezoidal AUC over the windows in Section 6.3. KeyCorridor uses success percentages; LoCA and DeepSea use return. Best other is the largest early mean among the seven reported baselines, not an independently selected method or a significance test. Appendix B reports every method; Section 6.3 specifies cohort selection and uncertainty estimation.

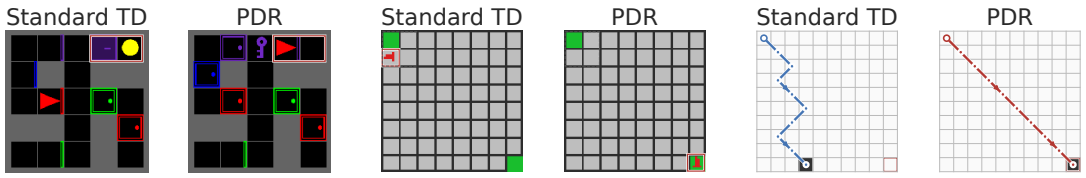
Family	Setting	n	PDR early	TD early	PDR final	Best other early
KeyCorridor	S3R2	20	19.0	18.9	75.0	IQL: 23.8
KeyCorridor	S3R3	20	65.5	47.1	97.0	PS: 58.9
KeyCorridor	S4R3	20	51.1	46.4	78.4	PS: 49.3
KeyCorridor	S5R3	20	85.2	71.2	91.5	PS: 79.1
LoCA	AWAC	20	1.724	0.100	1.787	PS: 1.440
LoCA	SAC	20	1.772	0.024	1.816	8× TD: 0.219
LoCA	QAC	20	1.772	0.023	1.816	IQL: 0.289
DeepSea	$N = 10$	20	0.963	0.530	0.990	CQL: 0.867
DeepSea	$N = 20$	19	0.969	0.622	0.990	CQL: 0.962
DeepSea	$N = 30$	13	0.966	0.429	0.990	CQL: 0.961
DeepSea	$N = 40$	13	0.965	0.366	0.990	CQL: 0.960

Two baseline patterns recur. PER and DisCor stay within 0.2 points of TD on KeyCorridor and within 0.01 in DeepSea. These sampling and weighting changes provide little improvement over TD in these experiments; 8× TD, which takes more backups, gains clearly in MiniGridLoCA with AWAC (0.925 against 0.100). CQL is below TD on KeyCorridor and in MiniGridLoCA with AWAC, although it had the largest baseline search (Table B.3). A likely reason is that its penalty lowers the values of actions that are rare in the fixed data, which here include the actions the improved policy should take more often. In DeepSea, where it shares PDR-SAC's actor and critic, CQL is the strongest baseline.

Figure 6.1 shows the difference between PDR and TD in single episodes: after the same training, the PDR agent reaches the goal while the TD agent is still on its way.

6.4.1. KeyCorridor

On S3R3, S4R3 and S5R3, PDR has the highest early mean (Figure 6.2). It beats the best baseline, PS, by 6.6 points on S3R3 ($[5.2, 8.0]$), 1.8 on S4R3 ($[0.7, 3.0]$) and 6.1 on S5R3 ($[4.1, 8.2]$). It also has the highest final success on S4R3 and S5R3, and approximately matches 8× TD on S3R3 (97.0% against 97.2%).



(a) KeyCorridor S3R3, step 51 (b) MiniGridLoCA, step 77 (c) DeepSea $N = 10$, step 9

Figure 6.1.: Standard TD (left of each pair) and PDR (right) at the same episode step after the same training, from saved AWAC policies of the earlier full-revaluation studies (Section B.4), not the configurations of Table 6.3; DeepSea uses an exact reference and AWAC rather than PDR-SAC. (a) After 64 actor updates, TD is still carrying the key (carried objects are not drawn) and the door is locked; PDR has unlocked the door, put the key down and picked up the ball. (b) After 512 updates, TD heads back to the old goal T_1; PDR reaches T_2. (c) After 1,024 updates, TD zig-zags; PDR goes straight to the treasure. Each panel is one episode, and the DeepSea episode was chosen from four recorded pairs as one that PDR completes and TD does not; over all seeds, success is 59% against 49% in (a), the probability of reaching T_2 is 0.78 against 0.01 in (b), and the treasure probability is 0.70 against 0.12 in (c).

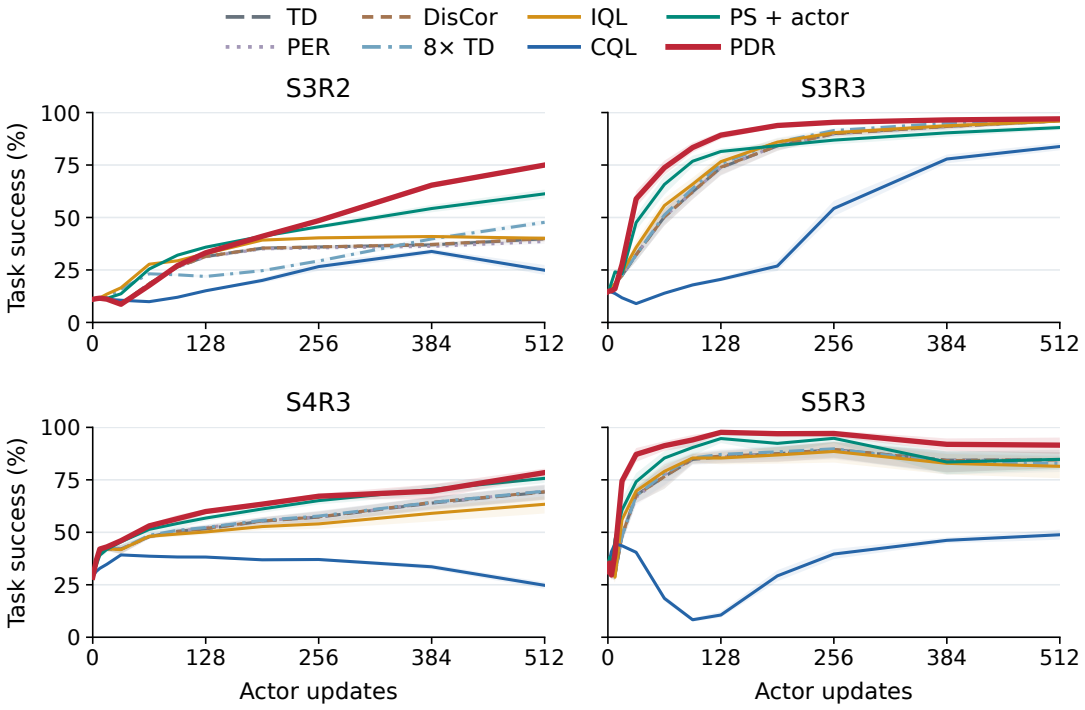


Figure 6.2.: KeyCorridor success probability over 512 actor updates, all eight methods, 20 paired seeds per task. Bands are pointwise 95% seed-bootstrap intervals.

S3R2 is the exception. Early on, PDR (19.0%) is no better than TD and below IQL (23.8%). Its gain comes later: final success is 75.0%, 13.7 points above PS ($[11.8, 15.9]$). Here PDR uses change only, $(0, 6)$, chosen for full-horizon performance.

6.4.2. MiniGridLoCA

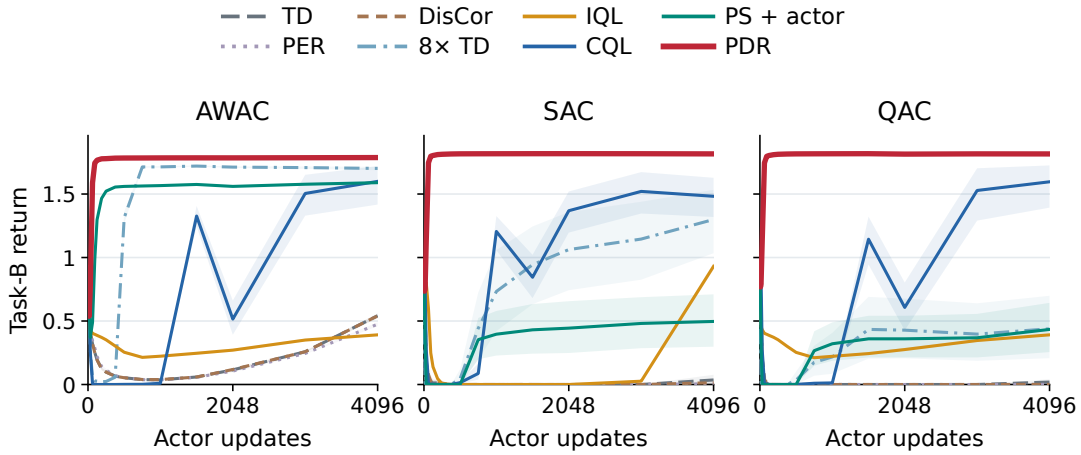


Figure 6.3.: MiniGridLoCA Task-B discounted return over 4,096 actor updates, from AWAC, SAC and QAC starting branches; all eight methods, 20 paired seeds each. PDR uses $(1, 8)$ for AWAC and $(4, 8)$ for SAC and QAC.

All methods start at 0.918 and first lose some return (Figure 6.3). The critic-only baselines then collapse. Under SAC and QAC, TD, PER and DisCor stay near zero for most of training. Under AWAC, TD falls to 0.196 by update 128. PDR recovers quickly: after 128 updates, its return is 1.767, 1.803 and 1.803 under AWAC, SAC and QAC, as the policy switches to T_2.

PDR has the highest early and final means for all three actors. It beats every baseline in all 20 seeds; the smallest gain is +0.285 over PS under AWAC ($[0.278, 0.292]$). 8× TD and CQL recover later under AWAC (final 1.702 and 1.600).

6.4.3. DeepSea

PDR-SAC nears the optimum quickly at every size (Figure 6.4). After 128 updates, its return is 0.951, 0.961, 0.952 and 0.947 for $N = 10, 20, 30, 40$. The AWAC-based baselines are much slower: at $N = 30$ and 40, they stay below 0.39 after 256 updates.

CQL is the closest baseline and the fairest comparison, since it shares PDR-SAC's actor and critic. At $N = 10$, PDR leads clearly (0.951 against 0.691 after 128 updates; early

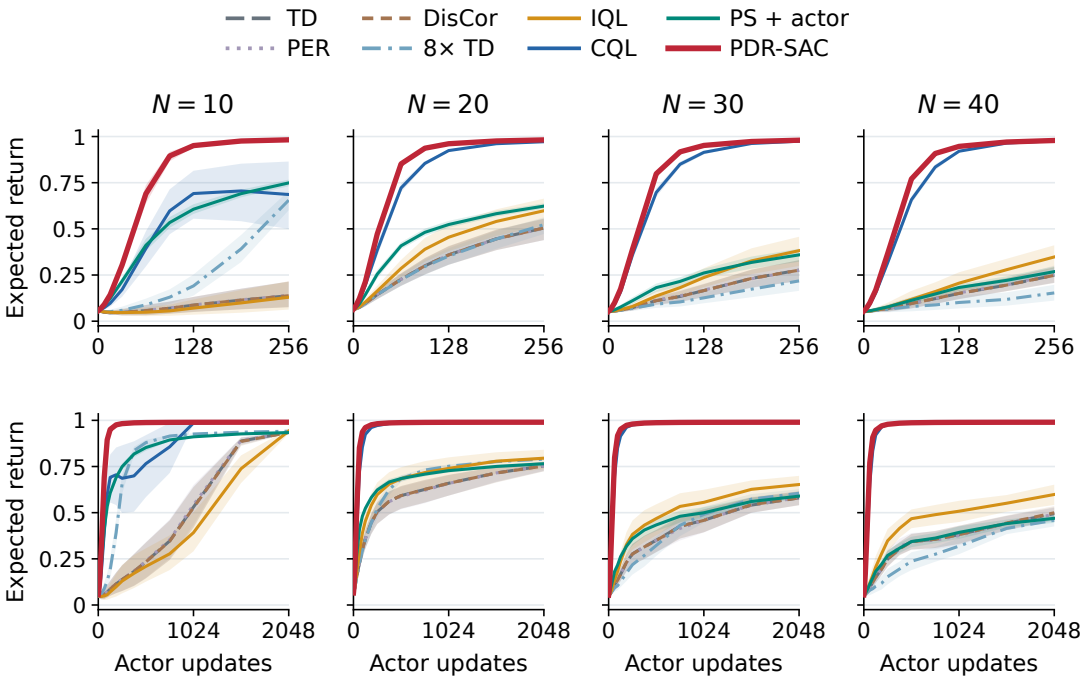


Figure 6.4.: DeepSea expected return. Top: first 256 actor updates. Bottom: early window, 2,048 updates. PDR-SAC uses an exact soft reference and $(1, 32)$; seeds are 20/19/13/13 for $N = 10/20/30/40$. The full 8,192-update view is in Appendix B.

gain +0.096, $[0.046, 0.153]$). At $N = 20, 30, 40$, the gaps are small but consistent: +0.007, +0.005 and +0.005. Both reach 0.99 by the end.

6.4.4. SecureCourier (Supplementary)

We start from the 50 checkpoints of Chapter 4. Standard TD trains the actor with the critic's weights; PDR revalues every record for cycles 1–1,024, with the policy and critic saved before gate practice as its reference. From cycle 1,025, both use the critic's weights.

By cycle 1,024, shortcut choice rises from 18.9% to 33.4% under PDR and falls to 13.5% under TD (Figure 6.5). PDR's early return is 0.47197 against 0.46233, a gain of 0.00964 (95% CI $[0.00888, 0.01049]$), positive in all 50 seeds. This is 3.6 times the gain of Corrected weights (Chapter 4).

The early gain does not carry over to final success. At cycle 2,048, greedy success differs by −0.375 points (95% CI $[-1.25, 0.25]$), so the planned non-inferiority test (lower bound above −1 point) fails. Appendix C gives further results.

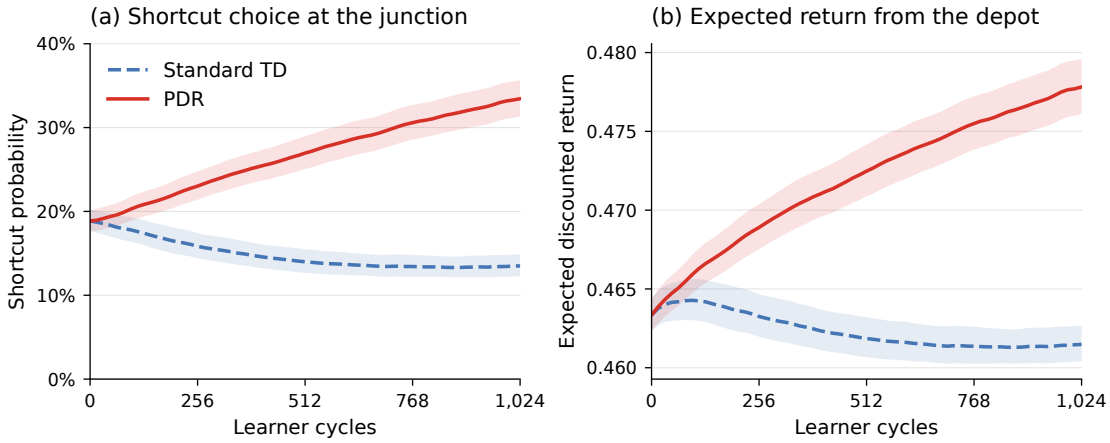


Figure 6.5.: SecureCourier: PDR versus Standard TD over learner cycles 0–1,024. (a) Shortcut probability at the junction. (b) Expected discounted return from the depot. Means over 50 paired seeds; bands are pointwise 95% bootstrap intervals.

6.5. Summary

PDR improves early adaptation (RQ2). On the public benchmarks, it has the highest early mean in 10 of 11 settings, and its final mean is never clearly below the best baseline. In SecureCourier, it also raises shortcut choice from 18.9% to 33.4%. These are comparisons of complete, tuned methods. The next three chapters ask why correcting the signal helps.

Chapter 7

Analysis of Signal Timing

In SecureCourier, the critic takes a median of 1,408 learner cycles to give the shortcut a positive signal (Chapter 4). In this chapter, we ask what sets this delay (RQ3).

Section 5.3 predicts the answer, which we call the *successor-readiness hypothesis*: a predecessor cannot use a correction until its successor has it. We test this in a controlled task where the size of the value change is fixed and only its distance varies. We then change the update order, truncate the correction, and measure the effect on a single actor update.

We measure time in two units. Section 7.2 first counts learner cycles, as in Chapter 6; the rest of the chapter counts *logical reads*, the number of stored values a method reads. Neither unit is wall-clock time, which depends on the implementation and is reported separately in Appendix G.

7.1. Controlled Propagation Task

The task is a small DeepSea-like MDP, separate from the benchmark in Chapter 6. Each probe state has two actions. *Enter* leads through a chain to a five-step skill module that ends with a reward of 1. *Safe* ends at once with reward 0.5. Learning the skill raises the correct-action probability in the module from 0.70 to 0.95. Enter's true value then rises from 0.164 to 0.765 at every probe (Figure 7.1a). Only the distance from the probe to the module differs.

Every method starts from the exact old values and uses the same graph. There are 30 runs per graph size, each with five replay orders. The main size is $N = 50$, with 1,230 probes (Appendix D.5).

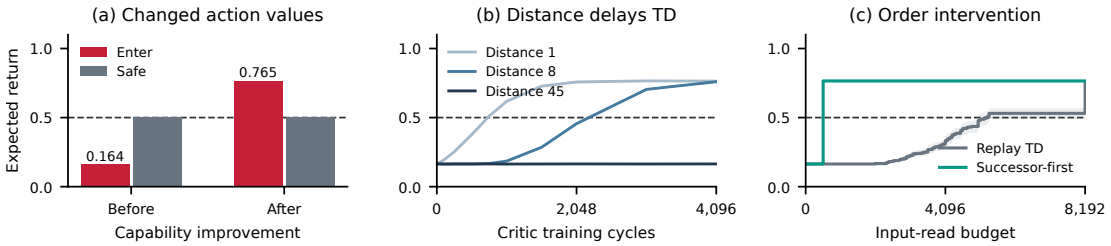


Figure 7.1.: Value correction in the controlled task. (a) The true value change is equal at every distance. (b) Cycles until the estimate of Enter crosses Safe under the tabular Adam comparison. (c) Logical reads to 90% value recovery under the step-size-one backup-order intervention of Section 7.3. Dashed lines mark the safe return of 0.5.

7.2. Effect of Bellman Distance

Tabular critics. We first give the methods of Chapter 6 tabular critics. TD, PER and DisCor take one Adam step per cycle, 8× TD takes eight, PS takes 2,048 backups, and PDR solves once.[1] We record the *correct ranking*: the share of runs that rate Enter above Safe.

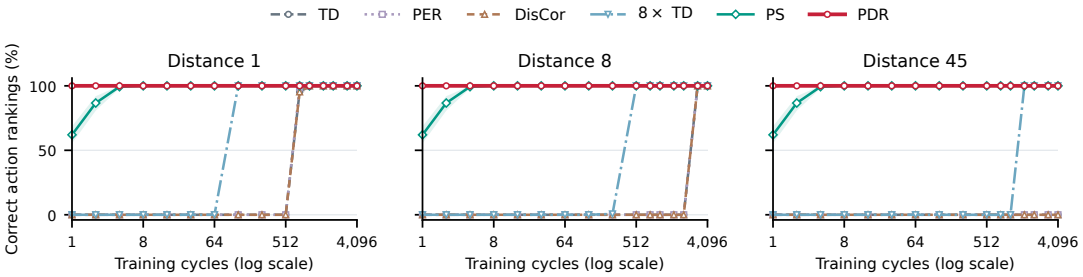


Figure 7.2.: Correct action ranking at three propagation distances. The horizontal axis is logarithmic; markers show recorded cycles. All methods start with the wrong ranking at cycle 0 (not shown).

The farther the probe, the later TD gets the ranking right (Figure 7.1b). PDR is correct in every run after one cycle (Figure 7.2). PS is also fast, reaching 100% by cycle 8 at distance 8. Extra critic updates help, but the delay still grows with distance. TD, PER and DisCor never reach 100% at distance 45.

[1] Thirty paired events at $N = 50$, each with five replay schedules; batch size 64, Adam learning rate 0.01, target update coefficient 0.005, and 4,096 cycles. Schedules are averaged within events; bands are pointwise 95% event-bootstrap intervals.

Replay TD and logical reads. The remaining experiments use replay TD with step size one and batches of 64. We measure time in *logical reads*: the number of stored values the method reads. A probe has a *value response* once it recovers 90% of the true change with the correct sign.

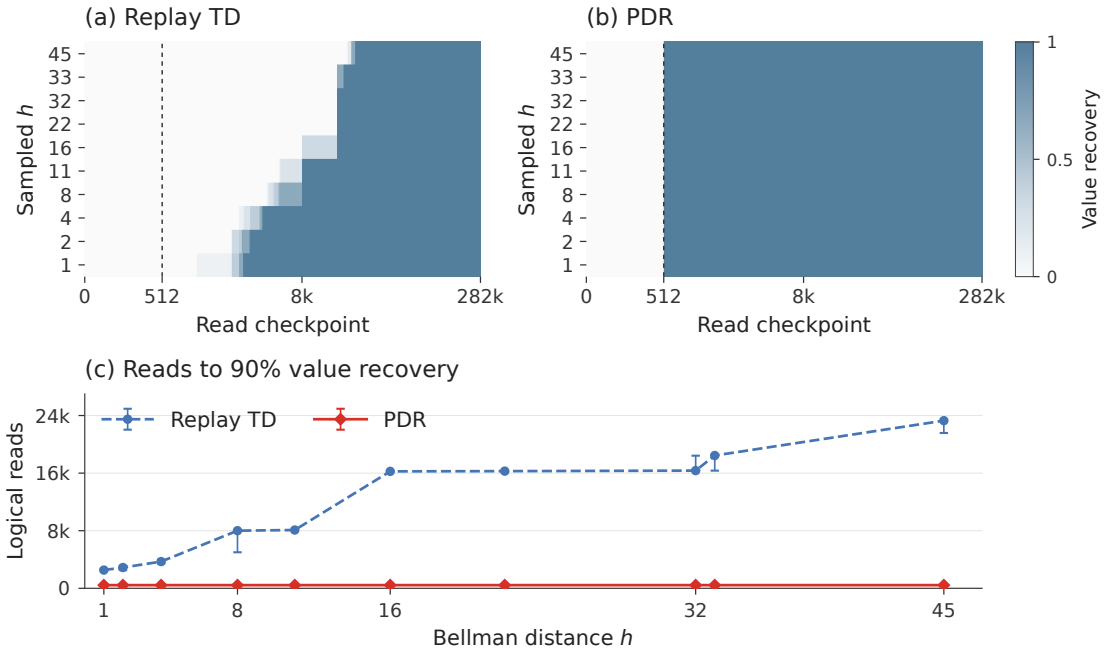


Figure 7.3.: Value recovery at $N = 50$. (a,b) Median recovery on the shared grid, held until the next checkpoint. Distance rows are categorical; reads are linear to 512 (dashed), then logarithmic. (c) Median actual reads to 90% recovery; whiskers show event-bootstrap 95% intervals. PDR takes 442 reads at every sampled distance.

Replay TD repairs nearby probes first (Figure 7.3). The reads it needs grow by 454.11 per step of distance (95% CI $[442.31, 465.72]$), and the slope is positive in all 30 runs. PDR corrects every probe after 442 reads. Appendix D gives further checks.

7.3. Effect of Update Order

If predecessors wait for their successors, updating successors first should remove most of the delay. We reorder the sampled records of each TD run so that successors come first. The records, the update rule, the number of updates and the number of reads stay the same. Only the order changes (Appendix D.6).

Successor-first order cuts the slope from 454.11 to 11.58 reads per step, a 97.45% reduction (Table 7.1; Figure 7.1c). Both orders reach the same values; they differ only

Table 7.1.: Distance slopes of the logical reads to 90% value recovery at $N = 50$: means over 30 independent events, with event-bootstrap 95% confidence intervals.

TD update order	Slope (reads / layer)	95% CI
Replay TD	454.11	[442.31, 465.72]
Successor-first TD	11.58	[11.57, 11.60]

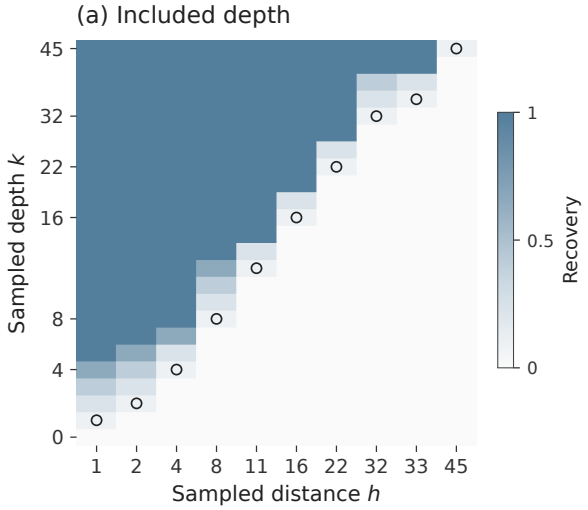
in when successors are ready. The order of updates therefore explains almost all of the distance effect.

7.4. Partial Corrections and Signal Sign

We can control how much of the correction a probe receives by truncating the series of Equation (5.14) at depth k:

$$D^{(k)} = \sum_{j=0}^{k} (\gamma \mathcal{P}^{\pi_{\text{new}}})^j \, c, \tag{7.1}$$

This includes changes up to k steps away, like $k + 1$ TD sweeps.



(b) Value recovery

$k-h$	Rec. (%)	$D^{(k)}$	Q_e	$Q_e - Q_s$	n
−1	0.00	0.000	0.164	−0.336	300
0	9.76	0.059	0.223	−0.277	300
1	23.11	0.139	0.303	−0.197	270
2	41.34	0.248	0.413	−0.087	120
3	66.19	0.398	0.562	+0.062	120
4	100.00	0.601	0.765	+0.265	120

(c) Recovery at completion

PDR depth	Reads/event	Recovery ≥ 90%
$k = 8$	2,818	90/300
$k = 16$	5,194	150/300
$k = 32$	9,946	210/300
$k = 45$	13,807	270/300
Full PDR	442	300/300

30 events × 10 probe distances

Figure 7.4.: Depth truncation at $N = 50$. (a) Median recovery on the recorded categorical (h, k) grid; circles mark first arrival. (b) Stage medians, $Q_e = Q_{\text{old}} + D^{(k)}$, $Q_s = 0.500$; n counts event–probe pairs. (c) Completion reads and probes reaching 90% recovery out of 300. Reads exclude solver arithmetic.

Every probe first receives a correction at $k = h$, its distance to the module (Figure 7.4). This first correction is small. It restores only 9.76% of the change, so Enter (0.223) is

still below Safe and the sign is still wrong. Enter overtakes Safe at $k - h = 3$, and the full correction arrives at $k - h = 4$.

In other words, a correction can reach a probe before it is large enough to flip the decision, and in that interval the signal keeps the wrong sign. At $k = 45$, 13,807 reads bring only 270 of 300 probes to 90% recovery, while full PDR recovers all 300 with 442 reads.

7.5. Effect on Actor Updates

Each checkpoint runs an *actor pulse*: a two-action actor starts at $P(\text{Enter}) = 0.5$ and takes one AWAC update on a batch with each action once (SGD, learning rate 0.1, temperature 0.1). Here $A(\text{Enter}) = [Q(\text{Enter}) - Q(\text{Safe})]/2$ has the sign of the ranking.

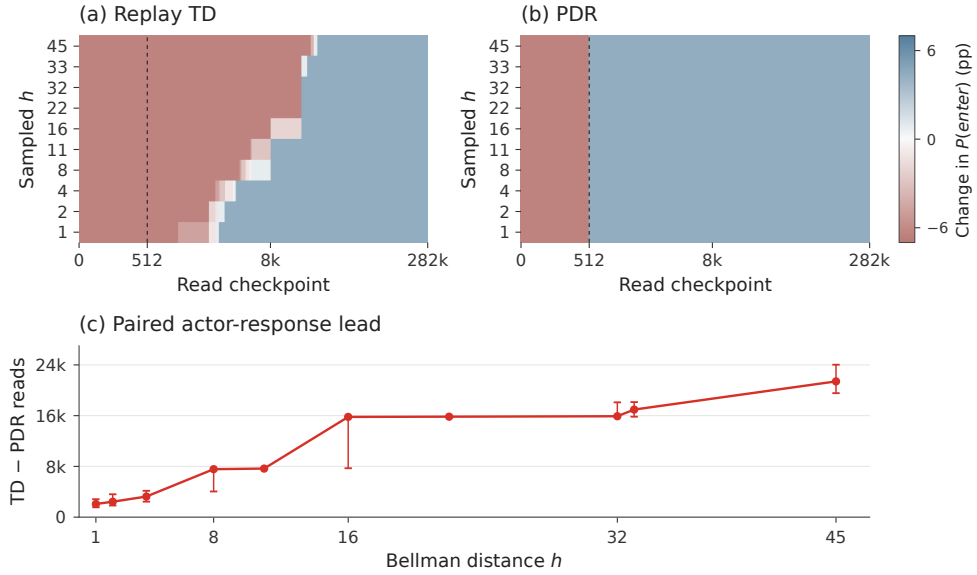


Figure 7.5.: One-update actor response at $N = 50$. (a,b) Change in Enter probability from 50%, in percentage points: red decreases Enter and blue increases it. Distance rows, checkpoint coordinates and aggregation match Figure 7.3. (c) Paired TD-minus-PDR reads to 90% of the exact-value pulse's probability change; points are medians and whiskers are interquartile ranges over 150 paired event–schedule units at each distance.

With TD values, the pulse pushes the actor away from Enter until the correction arrives (Figure 7.5). The delay grows by 454.11 reads per step (95% CI $[442.60, 465.57]$); for PDR, it does not grow.

To confirm the cause, we swap the inputs at a checkpoint where PDR is correct and TD is not (Table 7.2). Removing $\widehat{D}$ from the PDR values flips Enter's sign. Its weight

Table 7.2.: Value-input ablation; means over 30 events. (a) Enter value Q_e, advantage A_e, per-pulse weight ratio w_e/w_s, and one-update probability and shift from 50%. (b) Paired event-bootstrap 95% CIs. (c) Effects across scales; two registered windows at $N = 30$ are unavailable.

(a) Value inputs and one-update responses ($N = 50$)

Condition	PDR $\widehat{D}$	Q_e	A_e	w_e/w_s	P_e^+ (%)	ΔP_e (pp)
Old Q_{old}	—	0.164	−0.168	0.035	43.57	−6.43
Exact Q_{new}	—	0.765	+0.133	14.180	54.36	+4.36
Full PDR	✓	0.765	+0.133	14.180	**54.36**	+4.36
PDR $-\widehat{D}$	—	0.164	−0.168	0.035	43.57	−6.43
Replay TD	—	0.166	−0.167	0.037	43.63	−6.37
TD + PDR values	✓	0.765	+0.133	14.180	**54.36**	+4.36

(b) Paired probability effects ($N = 50$)

Contrast	Effect (pp)	95% CI (pp)	Events with expected sign
Ablated PDR − full PDR	−10.80	[−10.80, −10.80]	30/30
Restored TD − replay TD	+10.73	[+10.71, +10.75]	30/30

(c) Ablation and restoration across all graph sizes

N	Valid / total windows	Ablation (pp)	Restoration (pp)	Events < 0	Events > 0
10	750/750	−10.80	+10.80	30/30	30/30
20	1200/1200	−10.80	+10.66	30/30	30/30
30	1348/1350	−10.80	+10.59	30/30	30/30
40	1350/1350	−10.80	+10.68	30/30	30/30
50	1500/1500	−10.80	+10.73	30/30	30/30

relative to Safe drops from 14.180 to 0.035, and the pulse moves Enter from 50% to 43.57% instead of 54.36%. Both effects hold in all 30 runs at every size.

7.6. Relation to Neural Learners

The same delay appears with a neural critic: in SecureCourier, it gives the shortcut the wrong sign for over 1,000 updates (Figure 8.2). PDR avoids the wait: on a fixed graph, one solve corrects every probe at once, at a cost set by the graph size, not by distance (Appendix G).

7.7. Summary

TD's correction reaches farther actions later, because each update waits for its successor (RQ3); until then, the actor moves away from the better action.

Chapter 8

Analysis of Signal Content

Corollary 1 splits any value error into a common shift, which the actor cannot see, and a relative error, which is the error in the actor's signal. Full PDR corrects both. In this chapter, we ask which correction produces the gains of Chapter 6 (RQ4).

We use five studies. Section 8.1 corrects only the common shift. Section 8.2 measures which part a TD critic repairs first. Section 8.3 changes only the actor's input. Section 8.4 tracks how well each method's update aligns with the correct one and how the policy responds. Section 8.5 compares value accuracy with behaviour.

8.1. Common-Shift Control

Design. We use MiniGridLoCA with an AWAC actor, where the reward switch makes the stale critic's error large. We run two versions, with an exact and a learned reference, each with 20 seeds. Seven conditions continue for 4,096 updates. They share everything except the actor's input (Table 5.1): the critic (TD), the reference, the reference plus the common shift $\widehat{g}$ (Common-only), the reference plus the full change (Full PDR), the reference plus $\widehat{\delta}$ (Relative-only), the reversed change (Reversed), and directly evaluated values (Direct PE).

By Lemma 1, Common-only and Reference give the actor the same signal, A_{ref}. Full PDR and Relative-only give the same signal, $A_{\text{ref}} + \Delta A$. Direct PE also agrees under the conditions of Proposition 1.

Value and gradient error. At the start, the critic equals the learned reference, with a Q MAE of 2.717 (Figure 8.1a). Adding the common shift cuts this error by 98.6%, to 0.038. Yet the actor gradient does not change: it differs from the Reference gradient by only 4.5×10^{-7} (Figure 8.1b). The remaining 0.038 is the relative error. Full PDR, which also removes this small remainder, changes the gradient by 0.545.

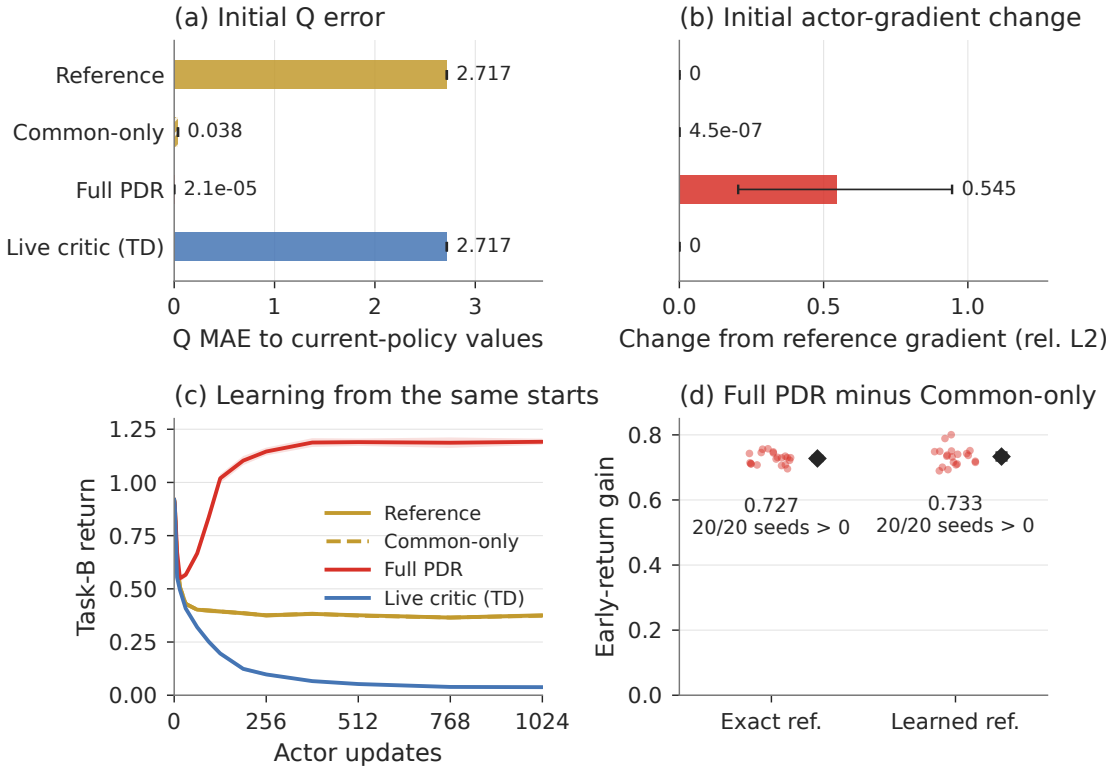


Figure 8.1.: Common-only control in MiniGridLoCA (AWAC). (a,b) Learned reference, at the shared initial actor and scheduled 128-record batch: Q MAE to the current policy's values over all nonterminal pairs, and relative L_2 change of the actor gradient from the Reference gradient. (c) Learned-reference continuations; Reference and Common-only overlap. (d) Paired early-return gains of Full PDR over Common-only in both lanes; points are seeds. Means with 95% seed-bootstrap intervals; 20 seeds per lane.

Learning performance. With the learned reference, early return is 0.383 for Common-only and 0.384 for Reference (difference -0.0004, $[-0.0012, 0.0005]$). Full PDR reaches 1.117. It beats Common-only by $+0.733$ ($[0.721, 0.746]$) with the learned reference and by $+0.727$ ($[0.720, 0.735]$) with the exact one, in all 20 seeds (Figure 8.1c,d). Reversing the relative correction lowers return to 0.363. The critic does worst (0.100): its own updates push the signal the wrong way.

Independence from the solver. Relative-only matches Full PDR: the difference is -0.00002 ($[-0.0007, 0.0006]$). Direct PE gives identical returns and identical final actors in all 20 exact-reference seeds. The gain therefore comes from the relative correction itself, whichever method supplies it. PDR is one way to compute it, and it also exposes the correction as a separate quantity. Appendix E gives all conditions.

8.2. Error Decomposition of Learning Critics

Common-only corrects the invisible part on purpose. Does a TD critic do the same? We split the error of saved critics with Equation (2.10).

SecureCourier. The fixed-actor run of Section 4.3 records the critic's value and signal for the shortcut, with their exact values, in 16 instances and 50 seeds. At the start, the shortcut's Q error of 0.517 splits into a common shift of 0.307 and a relative error of 0.210.

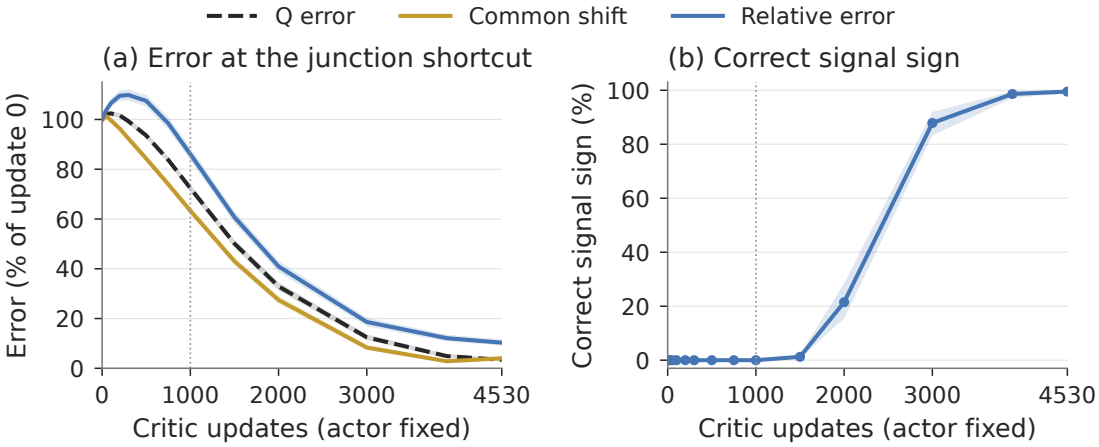


Figure 8.2.: SecureCourier fixed-actor TD repair at the junction shortcut. (a) Q error, common shift and relative error, each as a percentage of the same seed's value at update 0. (b) Percentage of instances where the critic gives the shortcut a positive advantage, as exact evaluation does. Means over 50 seeds (16 instances each) with 95% seed-bootstrap bands; the dotted line marks 1,000 updates.

TD reduces the common shift first (Figure 8.2a). After 500 updates, Q error falls by 6.5% ([5.3, 7.6]), but the relative error *rises* by 7.5% ([5.2, 9.7]). After 1,000 updates, Q error is down 27.6%, while the relative error is down only 13.9%. During this time, the sign is wrong in every instance (Figure 8.2b). It is still wrong in 98.8% of instances at 1,500 updates and in 78.5% at 2,000. Over this period, Q error falls steadily even though the actor's signal does not improve.

MiniGridLoCA. After the reward switch, the baseline critics still hold the old values. Over 240 runs, 99.97% of their squared error is common shift (RMS 2.75, against 0.045 for the relative error). The common part is the old value level, which the actor cannot see. The relative part is small, but it is 2.3 times the size of the true advantage, so it

can misrank actions. With eight critic updates per actor update, Q MAE falls from 2.71 to 0.31 by update 512, yet the critic ranks the best action first in only 61% of states.

In both tasks, then, much of the Q error is invisible to the actor, and TD reduces this part first.

8.3. Actor-Input Interventions

Here we change only where the actor's values come from.

Matched branches. In MiniGridLoCA, we copy the full training state at TD checkpoints 0, 8, 16 and 32 into three branches: Live critic, Full PDR and Exact values. Each branch runs 32 actor updates with the same batches. Only the actor's input differs.

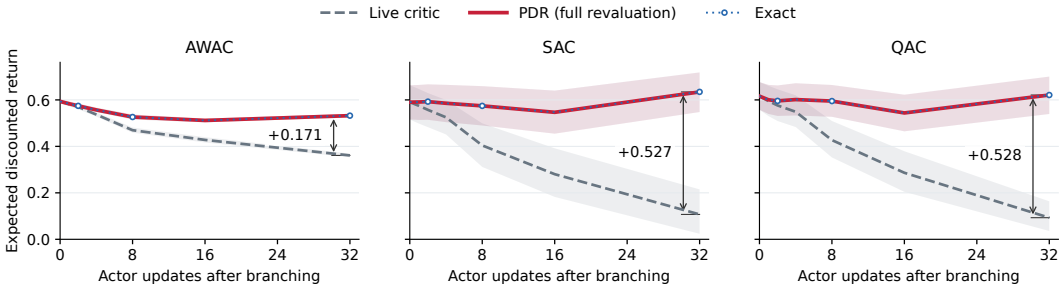


Figure 8.3.: MiniGridLoCA actor-input intervention: Live critic, Full PDR ($\alpha = \beta = 1$) and Exact current-policy values. Branches share the initial actor, critic and optimizer states, objective, scaling and batch sequence, and continue for 32 actor updates. Four starts are averaged within each of 20 paired seeds; bands are pointwise 95% seed-bootstrap intervals.

Full PDR tracks Exact closely and beats the critic under AWAC, SAC and QAC (Figure 8.3). Changing only the actor's input is enough to change the outcome.

8.4. Signal Alignment and Policy Change

We also record the actual actor updates of the methods in Chapter 6 over their first 128 updates.

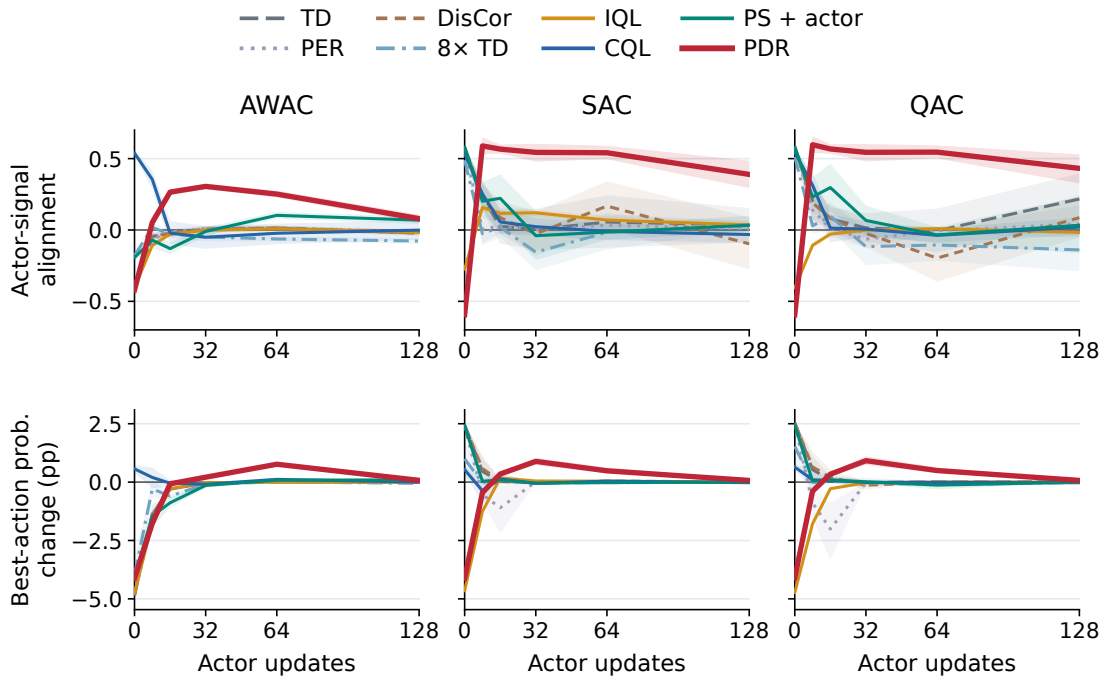


Figure 8.4.: Actor-signal alignment (top) and best-action probability change (bottom) in MiniGridLoCA over the first 128 actor updates. Means over 20 paired seeds with pointwise 95% bootstrap bands.

Actor-signal alignment. At each checkpoint, exact evaluation gives the improvement direction $g(s, a) = \pi(a \mid s)A^{\pi}(s, a)$. From each method's actor loss, we take the negative gradient d on the action logits. Over 226 upstream states,

$$\text{Alignment} = \frac{\langle d, g \rangle}{\|d\| \, \|g\|}. \tag{8.1}$$

A positive value means the update points towards better actions.

Best-action probability change. We also measure how much one update raises the probability of the best actions, averaged over the same states.

PDR's alignment becomes strongly positive early, and its updates then raise the best actions' probability (Figure 8.4). At update 32, alignment is 0.31, 0.54 and 0.54 for AWAC, SAC and QAC, and the probability gains are 0.21, 0.89 and 0.92 points.

8.5. Value Accuracy versus Behaviour

If the actor responds only to the relative signal, value accuracy need not predict behaviour. Two studies show this (Table 8.1).

Table 8.1.: Accuracy and behaviour rank interventions differently. (a) MiniGridLoCA partial-graph revaluation; the baseline is TD, means over 20 paired runs. (b) SecureCourier critic-baseline extension, means over 50 paired seeds. Entries are early-window means over 0–1,024 updates or cycles; the two panels have different protocols and units.

Metric	Baseline	PDR
(a) MiniGridLoCA; PDR at 50% coverage		
Q MAE	3.050	3.029
Advantage MAE	0.0471	0.0563
Actor-weight MAE	0.2351	0.2729
(b) SecureCourier; baseline 8× *TD*		
Shortcut-weight MAE	0.143	0.373
Expected discounted return	0.46675	0.47197

Partial transition model. In MiniGridLoCA, we give PDR only part of the graph and fill the rest with critic values (Appendix E.3.2). At 50% coverage, Q MAE falls slightly, from 3.050 to 3.029. But advantage MAE rises from 0.0471 to 0.0563, and weight MAE from 0.2351 to 0.2729. The values get better while the signal gets worse.

Weight error versus return. In SecureCourier, 8× TD has lower shortcut-weight error than PDR (0.143 against 0.373), yet PDR has higher early return (+0.00521, $[0.00456, 0.00592]$). PDR's reference is imperfect, so its weights on these records are less accurate. But it corrects the whole batch, while the error is measured only on the old shortcut records (Appendix C.5).

8.6. Summary

Only the relative part of a value correction changes the actor (RQ4). Removing 98.6% of a critic's error changes nothing when that error is the common shift. Correcting the remaining relative part gives the full benefit. In both tasks we examine, TD reduces the invisible part first, so Q error falls while the signal stays wrong. Value error is therefore a poor guide to what the actor receives. The signal and its sign should be measured directly.

In PDR, the relative correction is ΔA, added to A_{ref} with equal weight. Chapter 9 asks whether both parts are needed.

Chapter 9

Analysis of Signal Composition

The revalued signal has two parts: the reference preference A_{ref} and the recent change ΔA (Section 5.5). Full revaluation adds them with equal weight, but most settings in Chapter 6 do not. In this chapter, we ask whether the actor needs each part, and whether their ratio matters beyond a common scale (RQ5).

9.1. Component Ablation

Design. We compare reference only $(8, 0)$, both $(8, 8)$ and change only $(0, 8)$, from the same starts and batches. We use learned references with AWAC, SAC and QAC in MiniGridLoCA, AWAC in KeyCorridor, and the PDR-SAC setting with an exact reference in DeepSea. Seeds and windows are as in Table 6.2.

MiniGridLoCA. Without ΔA, early return stays near the start: 0.90 under AWAC and 0.93 under SAC and QAC (Figure 9.1, top). Adding the change raises it to about 1.67 for all three actors (AWAC: +0.767, $[0.765, 0.768]$, 20/20 seeds). Removing the reference costs nothing; change only is even slightly better. After the switch, the reference still prefers the goal that lost value, and only the change term can override this preference.

DeepSea. The pattern reverses (Figure 9.1, bottom). With change only, return stays near zero at every size. Removing A_{ref} costs 0.96–0.97 in early return, in all 65 seeds. Reference only already reaches 0.950–0.959. Adding ΔA helps a little: +0.006 to +0.009, in every seed. Here the reference already ranks the actions correctly. The change alone does not tell the actor which way to go.

Most of PDR-SAC's lead over the AWAC baselines in Chapter 6 therefore comes from the reference. The change term itself adds 0.006–0.009 over reference only; PDR-SAC's separate lead over CQL is 0.005–0.007 at $N = 20$, 30 and 40.

KeyCorridor. On S3R3, change only (60.6%) beats both (47.8%), which beats reference only (17.9%). On S4R3, change only (49.3%) is best and both (31.4%) is worst. On S3R2, both (24.0%) beats each part alone (19.7% and 19.6%; +4.4 points, 20/20 seeds).

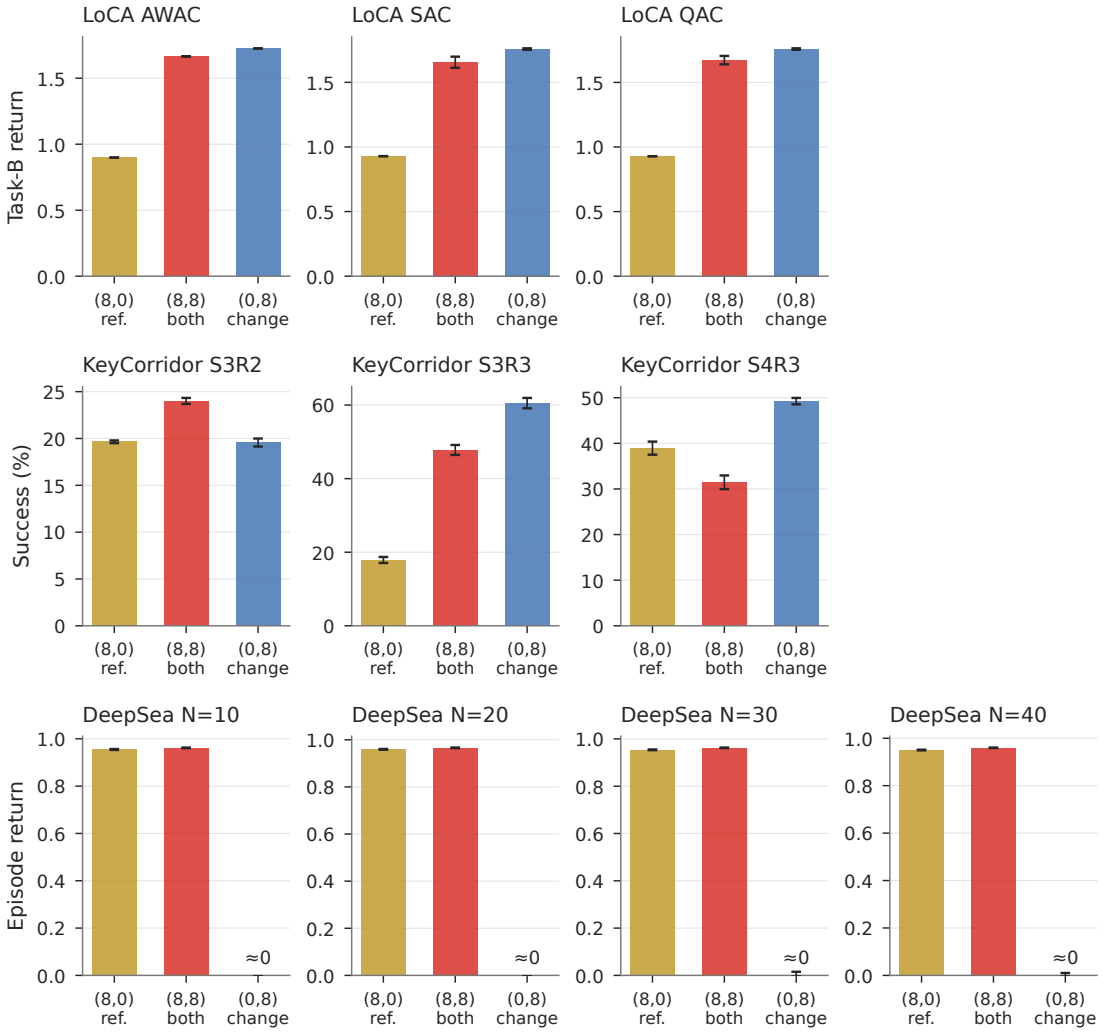


Figure 9.1.: Removing one component of the revalued signal. Early means of reference only $(8, 0)$, both $(8, 8)$ and change only $(0, 8)$, with 95% seed-bootstrap intervals. MiniGridLoCA and KeyCorridor use learned references (20 seeds); DeepSea uses PDR-SAC with its exact soft reference (20/19/13/13 seeds). Change only stays at a return of about zero in DeepSea.

Double dissociation. Each part is essential in one task and nearly useless in another: ΔA in MiniGridLoCA, A_{ref} in DeepSea. Neither part is therefore sufficient on its own. The change matters when the agent must override the reference preference, as after a reward switch. The reference matters when the reference preference is already right and the agent only needs to commit to it.

9.2. Coefficient Ratio and Scale

Coefficient sweeps. Table 9.1 varies the ratio in KeyCorridor and MiniGridLoCA. Weighting ΔA more beats the swapped setting in every case. On S3R3, $(1, 8)$ reaches 62.0% early success against 21.3% for $(8, 1)$. In MiniGridLoCA with AWAC, the same swap lowers early return from 1.724 to 0.892. In DeepSea, every pair with $\alpha \geq 1$ reaches 0.953–0.969.

Table 9.1.: Reference and change weighting with learned references. Mean performance over actor updates 0–128 for KeyCorridor (success, %) and 0–1,024 for MiniGridLoCA (Task-B discounted return), trapezoidal AUC divided by the window length; 20 seeds per setting. Bold marks the highest mean in each row.

		Emphasize ΔA ($\beta > \alpha$)			Emphasize A_{ref} ($\alpha > \beta$)			Equal weighting	
	A_{ref} weight α	1	2	4	8	8	8	1	8
	ΔA weight β	8	8	8	1	2	4	1	8
KeyCorridor (AWAC)	S3R2	21.4	21.5	22.0	19.9	20.3	21.5	19.7	**24.0**
	S3R3	**62.0**	61.5	54.1	21.3	25.3	33.9	50.8	47.8
	S4R3	47.9	45.5	39.1	37.8	35.7	32.9	**48.3**	31.4
MiniGridLoCA	AWAC	**1.724**	1.722	1.713	0.892	0.885	0.867	1.117	1.665
	SAC	1.762	1.767	**1.772**	0.929	0.929	0.929	1.682	1.658
	QAC	1.765	1.768	**1.772**	0.929	0.929	0.929	1.663	1.674

Ratio versus scale. A common scale (k, k) only changes the AWAC temperature (Equation (5.37)). An unequal ratio is therefore useful only if it beats the best common scale, which it does in several settings. In MiniGridLoCA (AWAC), $(0, 8)$ beats $(8, 8)$ by 0.060 ($[0.059, 0.061]$, 20/20 seeds), with the scale 8 chosen on three separate calibration seeds. In KeyCorridor, $(16, 32)$ beats the best equal setting on S3R3 by 6.3 points ($[5.3, 7.2]$), and $(2, 32)$ beats $(4, 4)$ on S5R3 by 7.8 points ($[5.2, 10.6]$), on the tuning seeds.

9.3. Summary

Both parts of the signal matter, and which one matters more depends on the task (RQ5): the change in MiniGridLoCA, the reference in DeepSea. Their ratio also matters. Other combination rules do not remove this dependence (Appendix F).

Chapter 10

Discussion

In this chapter, we interpret the results in terms of how actor–critic learning works, compare PDR with the alternatives, and state the limits of the evidence.

10.1. The Lag Is a Signal Problem

In the one-step TD actor–critic methods we study, credit is assigned through time by bootstrapping: each value estimate is trained towards the estimates that follow it. This makes an agent's own improvement a source of change. When the agent learns a later skill, earlier choices that lead there can gain value, although nothing in the environment has changed. Capability-to-choice lag arises where this self-induced change meets the division of labour in actor–critic learning. The skill is learned by the actor at the later state, but its value must travel back through the critic before the actor at the earlier state can use it.

The usual measures cannot see this. Return averages over all choices, and value error averages over all actions at a state; neither isolates the comparison that drives the earlier choice. In SecureCourier, the lag becomes visible only when skill, value, signal and choice are measured separately. During adaptation, most of a critic's error can sit in the part that centred actor objectives hide, so a critic can look better while the actor's signal does not.

Correcting only the selected shortcut records slows the decline in shortcut choice (17.9% against 13.5%), while correcting the whole batch with PDR raises it to 33.4%. Stale weights on the other records in each batch therefore also push against the shortcut: the actor is shaped by the signal on every record it trains on, not only on the action that changed. The public benchmarks show the same effect with state-conditioned actors.

10.2. PDR as Correction and Instrument

Planning is usually used to find something better, such as better actions for the policy to imitate or better values for the critic (Section 3.7). PDR targets a problem these

methods do not directly address: once an agent has improved, its earlier choices are still judged by values from before. PDR re-evaluates those choices, computes only the change the improvement has caused, and hands it to the actor while the critic keeps learning. Full PDR with exact inputs provably equals policy evaluation, and direct evaluation then gives the same actor updates (Chapter 8); its gains there come from delivering current values earlier, not from a different target. The benchmark settings of Chapter 6 also weight the two components.

Its reference-plus-change form gives a differential view of policy iteration. The performance-difference identity expresses a new policy's return through the old policy's advantages (Kakade and Langford, 2002); PDR expresses the new action values through the old values and a change, and solves for the change exactly. This form turns the interventions of Chapters 7–9 into one-line changes to the actor's input, with everything else fixed. Section 3.7 compares it with direct evaluation and prioritized sweeping. When a closed model is available, PDR is therefore a direct way both to correct the actor's signal and to ask which part of it the actor needs; with only a growing replay buffer, these methods are natural complements.

The component deletions sharpen how the benchmark results should be read. In DeepSea, reference-only PDR-SAC already reaches 0.950–0.959 in early return (Chapter 9): the decisive input is an accurate reference preference, and adding the change raises early return by a further 0.006–0.009 in every seed. PDR-SAC also leads CQL, which shares its actor and critic, by 0.005–0.007 at $N = 20, 30, 40$. In MiniGridLoCA, the change term is decisive instead. What the actor needs is an accurate signal, and which part of it carries the information depends on the task.

10.3. What the Actor Needs

Timing follows from how a correction is propagated. With the one-step tabular backups of our controlled task, a backup can pass on only what its successor already knows, so each backup moves a correction back by at most one step, and the order of updates decides when it arrives. Multi-step returns, eligibility traces and function approximation can carry a correction further, so the size of the delay depends on the update rule. What we show is that, for one-step TD, its dependence on distance comes almost entirely from order, and that a correction can arrive before it is large enough to flip the sign. A convergence guarantee alone does not say when the actor first receives the correct sign.

Content follows from what the actor consumes. Under centred objectives, value functions that differ by a statewise shift are equivalent for the actor: it sees only their advantages. Value regression does not respect this equivalence, so a critic can spend most of its learning on the part the actor discards. In the tasks we study, TD does exactly this early in adaptation.

Composition concerns how the two parts are combined. As an intuition, the reference preference carries what the old values supported, and the recent change carries what the improvement has added; the reference is not a statistical prior, and the change need not come from new observations. Neither part was enough on its own: MiniGridLoCA needs the change to override the old ranking, and DeepSea needs the reference to keep it. The two tasks also differ in actor objective and reference type, which may be why fixed ranking gates rarely trigger (Appendix F). A controlled test would vary only the reward change in MiniGridLoCA. Setting T_1's new reward below T_2's reverses the reference ranking, setting it above keeps it, and a third condition keeps the ranking while widening the gap. We would predict a higher best ratio β/α when the ranking is reversed; the third condition would show whether emphasizing the change also helps without a reversal.

10.4. When a Reweighted Signal Beats the Exact One

Most of the best benchmark settings do not use full revaluation. With $(\alpha, \beta) = (1, 1)$, an exact reference and an exact, closed model, PDR supplies the current policy's exact relative values (Proposition 1); the selected settings instead weight the recent change more. The clearest case uses an exact reference: in MiniGridLoCA with AWAC, the change alone at gain 8 reaches an early return of 1.726, against 1.666 for equal weights at the same gain (Appendix F). With learned references, where the equal-weight signal is itself an estimate, reweighting also leads: $(0, 8)$ beats $(8, 8)$, whose scale was calibrated on separate seeds, by 0.060 in MiniGridLoCA, and $(16, 32)$ and $(2, 32)$ beat the best equal settings on KeyCorridor S3R3 and S5R3 by 6.3 and 7.8 points (Section 9.2). A deliberately inexact signal can thus train the actor faster than the exact one, and this sharpens what an accurate signal is for.

Three explanations are open, and each makes a distinct prediction. First, the effect of reweighting may depend on the actor update rule. In AWAC, the signal passes through an exponential and a cap, so the ratio of the two parts can shape the weights in ways that a common scale, which only changes the temperature (Equation (5.37)), cannot. In SAC, the value term is linear in the signal, but its balance against the entropy term changes with the weights. This predicts that the benefit of reweighting differs between actor update rules. Second, with a learned reference, down-weighting A_{ref} may limit the reference critic's errors, which predicts a larger benefit with learned than with exact references. Third, the recent change isolates what is new, the effect of the policy's improvement and, in MiniGridLoCA, of the reward switch, so emphasizing it may speed the actor's response to new information; this predicts a benefit that grows with the size of the change.

Biased signals that train faster are familiar in actor–critic learning: GAE accepts bias in the advantage for lower variance (Schulman et al., 2016), and normalization and clipping change its scale. Weighted PDR goes a step further, because its bias acts on a named part of the signal and its effect can be measured. This gives a precise division of roles.

An accurate relative signal is what a critic should be evaluated against and what a correction should deliver; how strongly the actor follows each part is a design choice on top of it, which the test proposed in Section 10.3 can link to the reference ranking.

10.5. Evaluating Critics by Their Actor Signal

A critic is trained to predict values, but the actor needs comparisons. The same mismatch between what a component is trained for and what control needs appears in model-based reinforcement learning, where model accuracy can be weakly related to control performance (Lambert et al., 2020), and in value learning, where Bellman error is a poor proxy for value error (Fujimoto et al., 2022). Policy-gradient theory says what the actor needs: a critic with compatible features, fitted by least squares, gives an exact policy gradient even though its values differ from the true ones (Sutton et al., 1999a; Konda and Tsitsiklis, 1999). Our results measure how far a TD critic can be from that part during adaptation. After MiniGridLoCA's reward switch, 99.97% of its squared error is a common shift, and in SecureCourier Q error falls by 6.5% over the first 500 updates while the relative error rises by 7.5%.

Where exact values exist, a critic can therefore also be reported by its relative error (Corollary 1), sign agreement and update alignment, and replay schemes by when the actor's signs become correct. These measures complement value error rather than replace it (Section 2.7).

10.6. Limitations

PDR is built for settings where the transitions the current policy can reach are known, and there its gains are large. Our experiments supply these transitions as a fixed graph of stored records, and PDR has the highest early mean in 10 of 11 benchmark settings: in MiniGridLoCA with AWAC, early return is 1.724 against 0.100 for TD, and in KeyCorridor S5R3 success is 85.2% against 71.2%. The graph need not be complete. With part of it missing (Appendix E), PDR still helps at every coverage level in KeyCorridor and DeepSea $N = 10$, and in MiniGridLoCA once coverage is high. Building the graph from a growing replay buffer is the next step toward online learning.

The benchmarks have discrete actions and graphs that can be enumerated, up to 181,248 states in KeyCorridor S5R3. On them, PDR's solve is cheap: with preparation, optimized PDR takes 1.04–1.87 times TD's training time (Appendix G). Every public-benchmark run continues from a saved checkpoint with fixed replay, which isolates the effect of the actor's input; the DeepSea results refer to starts that pass the eligibility rule of Section 6.3.

Chapter 11

Concluding Remarks

In this thesis, we asked why an actor–critic agent can learn a new skill before its earlier choices use it, and how to close this gap. The question touches a basic feature of reinforcement learning: an agent's own improvement changes the value of its earlier choices, and in actor–critic learning that change must reach the actor through the critic. We argued that the gap lies in the signal the actor receives, that is, how each action compares with the policy's usual choice, and we proposed PDR, a reference correction that delivers the current policy's signal directly.

11.1. Conclusion

The lag is a signal problem (RQ1). Learning a later skill makes earlier choices better, yet the critic, which learns by bootstrapping, keeps reporting the old comparison for a while. In SecureCourier, shortcut choice falls from 18.9% to 13.5% although the skill is already learned, because the critic gives the shortcut the wrong sign; correcting that signal alone improves route choice and return (Chapter 4). The lag is therefore not a failure to learn the skill, but a delay in telling the actor what the skill is worth.

PDR corrects the signal (RQ2). PDR does not search for better actions. It computes how the agent's own improvement has changed the value of its current choices, relative to a saved reference, and hands this change to the actor at once. With exact inputs, it equals policy evaluation. Against seven critic-side methods on three public benchmarks, it has the highest early mean in 10 of 11 settings (Chapters 5 and 6).

The actor needs a timely, relative, balanced signal (RQ3–RQ5). On *timing*, in our controlled task each one-step backup moves a correction back by at most one step, so update order decides when it arrives: correcting successors first removes 97.45% of the distance effect, and a correction can arrive before it is large enough to flip the sign. Convergence of the values alone does not tell when the actor first receives the correct sign. On *content*, the actor sees only how actions compare: removing 98.6% of a critic's error changes nothing when that error is shared by all actions, yet in the cases we measured, this is the part critic learning reduces first. On *composition*, the signal

combines the old preference with the recent change; both can be necessary, and our results suggest that which one matters depends on whether the old preference should be kept. Reweighting can even beat the exact signal (1.726 against 1.666 with an exact reference in MiniGridLoCA): accuracy defines what a correction delivers, and weighting how strongly the actor follows each part (Chapters 7–9).

Critics should be judged by their signal. A critic is trained to predict values, but the actor needs comparisons. Value error can therefore fall while the actor's signal stays wrong; relative error, sign agreement and update alignment measure what the actor actually receives (Chapter 10).

11.2. Future Work

In generalized policy iteration, evaluation and improvement alternate (Sutton and Barto, 2018), and convergence analyses of actor–critic methods assume that the critic tracks the policy on a faster timescale (Konda and Tsitsiklis, 1999). Capability-to-choice lag is one consequence of this assumption failing in practice: the actor keeps learning from comparisons that describe a policy it no longer follows.

Credit assignment across levels of skill. In hierarchical agents built on options (Sutton et al., 1999b), improving an option is a policy change inside it, and its callers should then choose it more often. The higher-level critic, however, learns of the improvement only through bootstrapping, so the lag studied here can recur at every level. The reference-plus-change form suggests a direct route. PDR inside the option gives the change in the option's value at its initiation states; this change is a source for the higher level, with discount γ^τ for an option that lasts τ steps, and one solve there delivers it to every caller. The question is whether the higher-level policy then picks up an improved option sooner, for example in KeyCorridor with options for fetching the key, opening the door and reaching the ball.

What the critic should hand the actor. Our results suggest three changes to how critics are built. First, a critic could be trained on what the actor uses: in MiniGridLoCA, 99.97% of the squared error after the reward switch is a common shift the actor cannot see, so a loss on centred values would spend its capacity on the relative error. Second, its updates could be scheduled by their effect on the actor's signal rather than by TD error: successor-first order already removes 97.45% of the distance effect, and a priority on records whose advantage sign may be wrong would target the lag itself. Third, critics could be compared by when the actor first receives the correct sign, alongside their value error.

Revaluation from experience. Can PDR's revaluation run from a learned model and a growing replay buffer, and how does it compare with prioritized sweeping at matched computation?

Keeping or overriding what was learned. Can an agent judge, from its own signals, when to keep what it has learned and when to overturn it? This points to a possible link with the stability–plasticity trade-off of continual learning, which our experiments do not test.

Learning to act well is often treated as a matter of knowing how good things are. In actor–critic learning, the critic matters through the comparisons it hands the actor, and these can stay stale while the critic catches up. Fast adaptation therefore depends on how soon the actor receives an accurate comparison between its choices, which overall value error does not measure.

Appendix A

PDR Implementation Details

A.1. Graph Construction and Batch Lookup

In this appendix, we give the implementation details behind Section 5.7. Each state key must determine transitions, rewards, legal actions and termination, and must map to one policy input. For each key, the graph stores one observation and an ordered list of action inputs. Figure A.1 shows how records become a graph.

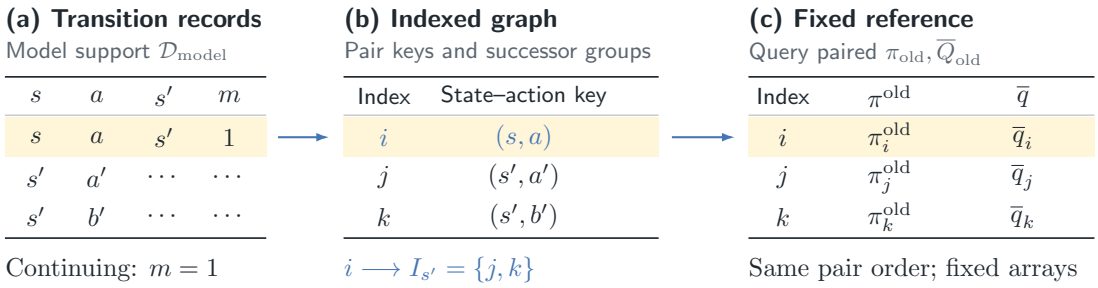


Figure A.1.: Graph construction and reference alignment (schematic).

Index graph states by σ_u, $u = 1, \ldots, S$, and pairs by $x_i = (s_i, a_i)$, $i = 1, \ldots, N$, as in Equation (5.23). The parent index κ_i satisfies $s_i = \sigma_{\kappa_i}$; the ordered group $I_u = \{i : \kappa_i = u\}$ lists the legal actions at state u. All policy and value arrays use the same pair order:

$$\overline{q}_i = \overline{Q}_{\text{old}}(s_i, a_i), \qquad \pi_i^{\text{old}} = \pi_{\text{old}}(a_i \mid s_i), \qquad \pi_i^{\text{new}} = \pi_{\text{new}}(a_i \mid s_i). \tag{A.1}$$

We estimate transition probabilities by counting outcomes. For pair i, $C_i(v, m)$ counts records with successor v and mask m, where v is a continuing state or a terminal outcome:

$$C_i = \sum_{v,m} C_i(v, m), \qquad p_i(v, m) = \frac{C_i(v, m)}{C_i}, \qquad \mathcal{O}_i = \{(v, m, p_i(v, m)) : C_i(v, m) > 0\}. \tag{A.2}$$

Terminal records count in the denominator but add no continuing mass.

The reference policy and critic come from the same checkpoint, and hashes check that they match. The reference is fixed; replacing it requires a new matched policy–value pair and realigned arrays.

For batch row n, let a_{nk} be its k-th candidate action and L_{nk} the legal-action mask. Dictionary lookup defines the batch-to-graph mapping

$$J_{nk} = \text{index}(\text{id}(s_n), a_{nk}), \qquad q_{nk} = Q_{\text{actor},J_{nk}} \quad \text{for } L_{nk} = 1. \tag{A.3}$$

The actor parameters stay fixed from the query to the loss; the optimizer updates them afterwards. The networks have no dropout or batch normalization. If a batch requests a pair or legal-action list that is not in the graph, the program stops before the actor update. For an ended episode, the batch supplies a placeholder action, which the terminal mask removes; it is not a graph node.

A.2. Indexing Example and Cached Inputs

Figure A.2 expands Equation (A.3), including padding.

Indices are int32, masks are Boolean, and full-table arrays are float64. Batch values are float32 on the actor device, with zero padding. The optional token adapter uses int64 tensors of shape $B \times L_o$ for observations and $B \times K_B \times L_a$ for actions. Tokens can be cached, but encodings must be recomputed after the encoder changes.

Table A.1.: Reuse conditions. The current trainer does not rebuild the graph or change its reference automatically.

Object	Reuse condition
Keys, $\kappa_i, I_u, \mathcal{O}_i$, tokenized inputs	Graph, legal-action order and input text are unchanged.
$\pi^{\text{old}}, \bar{q}$	The reference pair and graph ordering are unchanged.
Current encodings, $\pi^{\text{new}}, \widehat{b}, \widehat{c}, \widehat{D}$, weights	Valid for the actor snapshot queried; recomputed before the next active actor update.
Full-table corrected values	Available for any covered key within that snapshot.
Streaming batch values	Only the requested rows were saved; a new batch requires another pass.

Table A.1 lists when each cached object can be reused. Changing the graph or action order requires realigning the reference arrays. Changing the reference requires reevaluating it. New training records leave the graph unchanged, but their keys must be covered. The separate critic-repair routine samples only pairs that survive closure pruning.

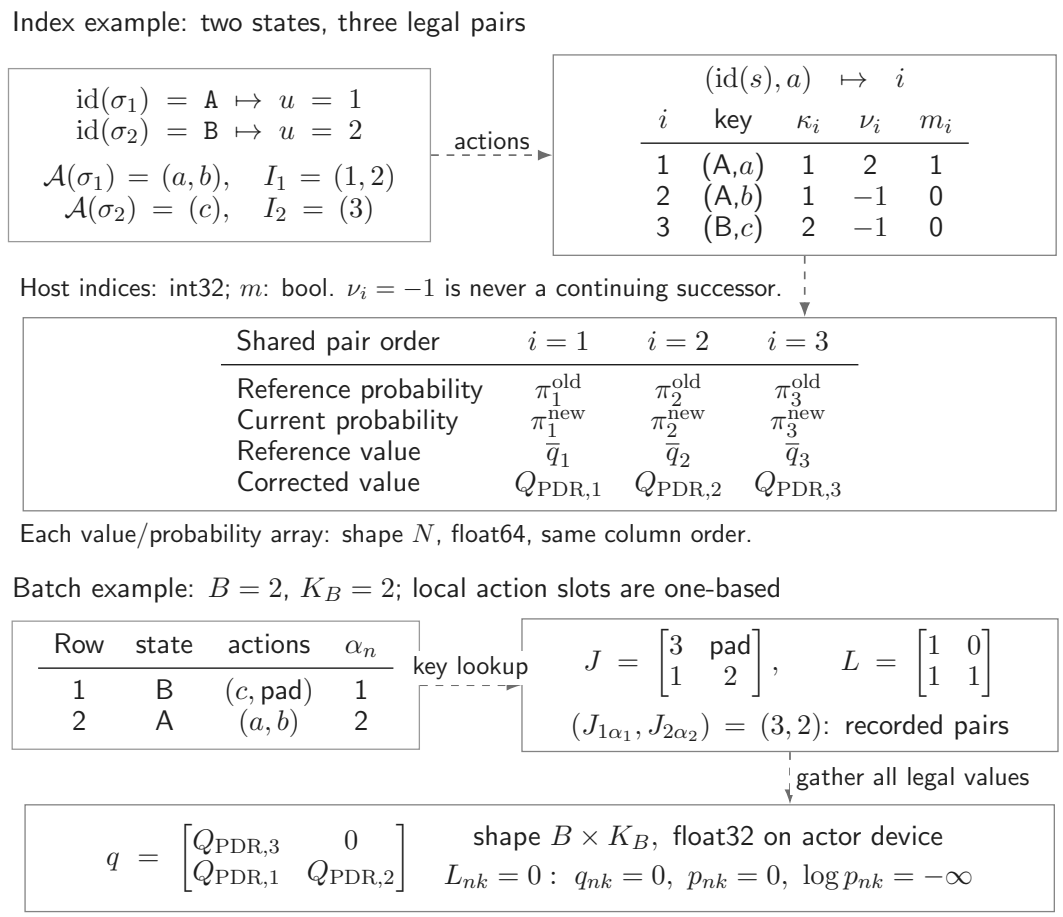


Figure A.2.: A complete state/pair/batch mapping, shown one-based. The implementation uses zero-based indices.

A.3. State Reduction, Streaming and Costs

Let $R_{iv} = \widehat{P}_m(\sigma_v \mid s_i, a_i)$ and $U_{ui} = \pi_i^{\mathrm{new}}$ for $\kappa_i = u$, zero otherwise. Then $\widehat{M}_{\mathrm{new}} = RU$, and taking state means gives

$$(I_S - \gamma UR)g = U\widehat{c}, \qquad \widehat{D} = \widehat{c} + \gamma Rg. \tag{A.4}$$

For $\gamma < 1$, this is an exact reduction of the action system. The dense kernel solves it with `numpy.linalg.solve`.

For reward changes and soft values, let $\Delta r_i = r_{\mathrm{new},i} - r_{\mathrm{old},i}$ and

$$\mathcal{H}(\pi; u) = -\sum_{i \in I_u} \pi_i \log \pi_i.$$

The source is

$$\begin{aligned}\widehat{b}_u^{\text{ext}} &= \widehat{b}_u + \alpha_{\text{ent}}[\mathcal{H}(\pi^{\text{new}}; u) - \mathcal{H}(\pi^{\text{old}}; u)],\\ \widehat{c}_i^{\text{ext}} &= \Delta r_i + \gamma \sum_{(v,1,p)\in\mathcal{O}_i} p\widehat{b}_v^{\text{ext}}.\end{aligned} \tag{A.5}$$

SAC uses its fixed entropy coefficient; AWAC and QAC set $\alpha_{\text{ent}} = 0$. A terminating transition keeps only Δr_i. Probabilities are clipped at 10^{-15} before logarithms. With unchanged rewards and $\alpha_{\text{ent}} = 0$, this reduces to Equation (5.39).

The streaming kernel uses the fixed-reward source (Figure A.3). For a deterministic finite-horizon model, it keeps only the current workspace and the next layer's state means. Let S_0 count physical states, K the action-slot width and H the horizon. Policies, transitions and legal actions must not depend on time; reference values may.

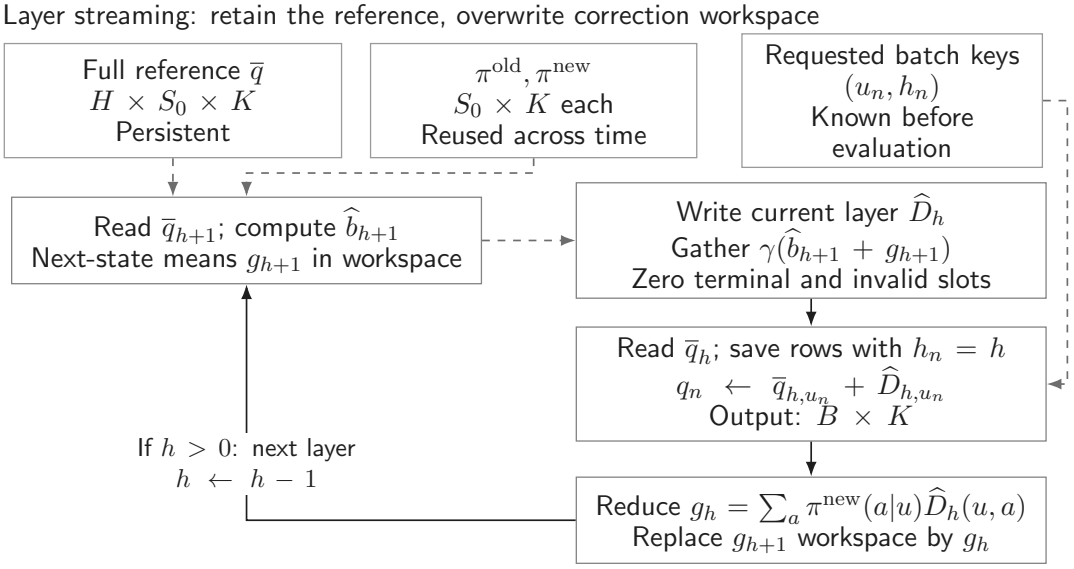


Figure A.3.: Streaming for the fixed-reward, policy-only source; the complete reference array remains stored.

This float32 kernel is separate from the float64 DAG used in training.

Work and storage. Let E count distinct outcomes, Z count pair-to-pair entries before merging, and C_π be the policy-query cost. Table A.2 separates graph work from network work. The AWAC trainer scans the arrays at every layer, so its cost is higher than the preindexed kernel's.

The loader builds I_u once in $O(SN)$ time and caches it. Sparse factorization cost depends on fill-in.

Table A.2.: Core evaluation costs. Full-table methods also store the reference; layer streaming stores it in $O(HS_0K)$.

Operation	Work	Storage
Policy query	C_π	probabilities, workspace
Sparse graph	$O(N+E+Z)$ + factorization	$O(N+E+Z)$ + factors
Preindexed DAG	$O(N+S+H)$	$O(N+S+H)$
Layer-scanning DAG (AWAC)	$O(H(N+S))$	$O(N+S)$
Dense state solve	$O(N+E)+O(S^3)$	$O(N+E+S^2)$
Layer streaming	$O(HS_0K)$	$O(S_0K+BK)$
Batch lookup and weights	$O(BK_B)$	$O(BK_B)$

A.4. Factorized Query and Validation Records

We first show that the factorized query is exact, then list the checks.

A.4.1. Policy-Query Equivalence

The SecureCourier token actor encodes the observation and each action separately, concatenates them, and applies one linear layer. Its score is

$$\mathrm{score}(o,a) = f(o) + g(x_a). \tag{A.6}$$

The term $f(o)$ is shared by all actions and cancels in the softmax. The optimized query therefore scores only the unique action rows and maps them back to graph order. Action encodings are recomputed for each actor snapshot. This shortcut does not apply to networks in which observation and action interact.

A.4.2. Numerical and Gradient Checks

The AWAC implementation checks key uniqueness, action order, array alignment, termination and coverage. Each solve checks finite inputs, valid policies and finite output. The solver residual is

$$\rho_\infty = \left\| \widehat{D} - \widehat{c} - \gamma \widehat{M}_{\mathrm{new}} \widehat{D} \right\|_\infty . \tag{A.7}$$

A small residual checks the linear system, not the accuracy of $\overline{Q}_{\mathrm{old}}$ or the model. The sparse and DAG solvers stop on a failed check; the dense kernel only logs its residual.

Graph queries compute probabilities in float32, convert them to float64 and renormalize each row (tolerance 2×10^{-12}). The actor batch uses float32 with tolerance 10^{-6} and zero padding mass. The DAG requires $\rho_\infty \le 10^{-10}$.

Native and factorized policy queries agree within 5×10^{-7}. Backward and sparse solvers agree within 2×10^{-10} on the same inputs. Zero-source, terminal and repeat checks also

pass. Runtime assertions confirm that action values, advantages and weights carry no gradient.

SAC and QAC use the objectives of Section 5.7 and the source of Equation (A.5). Appendix G.4 gives the loss conventions of the earlier benchmark runs.

A.5. Optimizing Repeated Revaluation

With a fixed graph and reference, successor indices, masks, reference sums and buffers are prepared once. Each refresh then queries the actor, recomputes the source and the change, and gathers the batch values. Reusable buffers avoid repeated allocation, and fixed tensor shapes allow a captured execution graph. Sparse systems avoid dense work, and acyclic graphs allow one backward pass. None of this changes the equation or the actor interface: Appendix G.3 reports the savings and checks that the actor receives the same values and gradients. The SecureCourier token actor also allows the factorized query above (Appendix G.8).

A.6. Timing Boundaries

We time preparation, solving, batch lookup and network updates separately. Preparation, which is outside the training timers, includes building the model, compiling the graph, tokenizing inputs and querying the reference. Table A.3 gives the scope of each timer within a cycle. Lookup time is part of actor-update time, which is part of the cycle time. The solver timer includes the source and checks, which the sparse solver's own timer excludes, so the timers overlap and are not interchangeable.

Table A.3.: Recorded timer boundaries in the main training adapter.

Field	Included work
`policy_seconds`	Current full-graph policy query and normalization.
`solve_seconds`	Input checks, source construction, backward evaluation and output checks.
`lookup_seconds`	All-action batch lookup and conversion to actor-device tensors.
`actor_update_seconds`	Batch policy query, lookup, weights, backward pass, clipping and optimizer step, with adapter diagnostics.
`total_cycle_seconds`	Collection, insertion, sampling, critic and actor work, target update and in-loop diagnostics before logging; scheduled scoring/checkpoint work follows this timer.

Appendix B

Experimental Details and Complete Results

In this appendix, we give the protocols of Chapter 6 and report every method. We then give the protocols of the earlier full-revaluation studies, whose results appear in Appendices C, E and G. Results from different protocols are never pooled.

B.1. Actor, Critic and Solver Configurations

All four families start from saved checkpoints. Methods in a comparison share the initial state and replay (Section 6.1).

Table B.1.: Current PDR training settings. Rates are actor/critic Adam learning rates. The table describes PDR, not every baseline's native recipe. SAC entropy is 0.001; AWAC weights are capped at 20.

Setting	Batch	Rate	γ	Target τ	Actor
SecureCourier	64	3×10^{-4}	0.97	0.005	AWAC, temperature 0.3
KeyCorridor	64	3×10^{-4}	0.97	0.005	AWAC, temperature 0.3
LoCA	128	10^{-3}	0.99	0.02	AWAC (0.25), SAC, QAC
DeepSea	64	3×10^{-4}	1	0.005	Discrete SAC

SecureCourier batches mix six offline and 58 online records; its 40-step graph covers 16 variants. KeyCorridor uses full-state features and a fixed action mask. LoCA uses state-ID inputs (embedding 128, hidden layers 256 and 512, LeakyReLU) and updated reward labels. DeepSea mixes 16 historical and 48 support records, and its CQL penalty is 0.01 at $N = 10$ and 0.1 otherwise. For LoCA SAC, the actor loss is $\mathbb{E}_\pi[0.001 \log \pi - (4Q_{\text{ref}} + 8\widehat{D})]$.

Software and hardware. We use Python 3.10 and PyTorch 2.5.1+cu121 on NVIDIA RTX 3090 GPUs. KeyCorridor uses CUDA graphs; DeepSea and PS use C++ kernels. Hardware and library differences can change floating-point trajectories. Timing protocols are in Appendix G.

Table B.2.: PDR configurations. Graph building and reference fitting are not counted in any training budget.

Setting	Actor	Reference	Live critic	Solver	(α, β)
SecureCourier	AWAC	learned	TD	layered DAG	$(1, 1)$
KeyCorridor S3R2	AWAC	learned	TD	time-expanded	$(0, 6)$
KeyCorridor S3R3	AWAC	learned	TD	time-expanded	$(16, 32)$
KeyCorridor S4R3	AWAC	learned	TD	time-expanded	$(1.25, 3)$
KeyCorridor S5R3	AWAC	learned	TD	time-expanded	$(2, 32)$
LoCA	AWAC	learned	TD	sparse	$(1, 8)$
LoCA	SAC, QAC	learned	TD; twin for SAC	sparse	$(4, 8)$
DeepSea	SAC	exact soft	twin TD + CQL	layered DAG	$(1, 32)$

Table B.3.: Settings searched per method; "+" marks a later, separate search. KeyCorridor PDR: 274 settings on S3R2 and S4R3, 90 on S3R3 and S5R3, large grids screened on five seeds first; DeepSea PDR: 12 pairs, two actor objectives. Selection used the early mean, except the full-horizon mean (KeyCorridor CQL; PDR on S3R2, S4R3) and the final return (LoCA IQL, CQL).

Method	Searched	KeyCorridor	MiniGridLoCA	DeepSea
TD, PER, DisCor, 8× TD	value scale	2	2	2
PS	scale, budget	6	6	6
IQL	expectile, temperature	8	$8 + 56$	8
CQL	penalty, entropy, rates	$9 + 78$	$9 + 60$	9
PDR	(α, β)	90–274	7	24

B.2. Complete Public-Benchmark Results

Tables B.4 and B.5 report all eight methods. KeyCorridor entries are success percentages; the others are returns. We do not average different metrics into one score. Figure B.1 extends Figure 6.4 to 8,192 actor updates; PDR-SAC and CQL end at 0.99 at every size.

Table B.4.: All selected methods: early normalized-AUC means. Settings, cohorts and windows match Section 6.3.

Family	Setting	TD	PER	DisCor	8× TD	IQL	CQL	PS	PDR
KeyCorridor	S3R2	18.9	18.9	18.9	19.3	23.8	11.4	23.6	19.0
KeyCorridor	S3R3	47.1	47.1	47.1	47.9	50.5	14.6	58.9	65.5
KeyCorridor	S4R3	46.4	46.4	46.3	46.8	45.9	37.6	49.3	51.1
KeyCorridor	S5R3	71.2	71.3	71.4	71.9	73.0	23.6	79.1	85.2
LoCA	AWAC	0.100	0.105	0.100	0.925	0.290	0.029	1.440	1.724
LoCA	SAC	0.024	0.010	0.025	0.219	0.073	0.202	0.151	1.772
LoCA	QAC	0.023	0.013	0.023	0.085	0.289	0.032	0.119	1.772
DeepSea	$N = 10$	0.530	0.535	0.529	0.827	0.454	0.867	0.851	0.963
DeepSea	$N = 20$	0.622	0.621	0.622	0.686	0.697	0.962	0.692	0.969
DeepSea	$N = 30$	0.429	0.428	0.429	0.435	0.520	0.961	0.475	0.966
DeepSea	$N = 40$	0.366	0.365	0.366	0.307	0.475	0.960	0.369	0.965

Table B.5.: All selected methods: final means at the full update budget. Settings, cohorts and windows match Section 6.3.

Family	Setting	TD	PER	DisCor	8× TD	IQL	CQL	PS	PDR
KeyCorridor	S3R2	39.8	38.5	39.9	47.7	40.1	24.9	61.3	75.0
KeyCorridor	S3R3	96.0	96.0	96.1	97.2	96.0	83.8	92.9	97.0
KeyCorridor	S4R3	69.2	69.6	69.3	69.6	63.4	24.7	75.7	78.4
KeyCorridor	S5R3	84.6	84.5	84.5	82.8	81.4	48.8	84.8	91.5
LoCA	AWAC	0.541	0.475	0.540	1.702	0.391	1.600	1.589	1.787
LoCA	SAC	0.036	0.006	0.014	1.299	0.931	1.482	0.497	1.816
LoCA	QAC	0.019	0.001	0.001	0.439	0.391	1.596	0.434	1.816
DeepSea	$N = 10$	0.963	0.963	0.963	0.962	0.985	0.990	0.960	0.990
DeepSea	$N = 20$	0.878	0.877	0.878	0.875	0.936	0.990	0.867	0.990
DeepSea	$N = 30$	0.850	0.849	0.848	0.828	0.824	0.990	0.798	0.990
DeepSea	$N = 40$	0.797	0.794	0.804	0.783	0.765	0.990	0.733	0.990

B.3. Relationships between Experimental Studies

The SecureCourier comparison in Chapter 6 reuses the 50 seeds of the earlier study, so it is not an independent replication. The coefficient sweeps of Chapter 9 are run separately from the selected configurations; we omit the earlier DeepSea AWAC rows, which do not use the PDR-SAC recipe. The LoCA signal study includes one regenerated SAC/8× TD trajectory, with no spliced checkpoints. The input swaps of Section 8.3 average four starts per seed (720 runs). The cost studies time the full-revaluation implementations, not the weighted configurations.

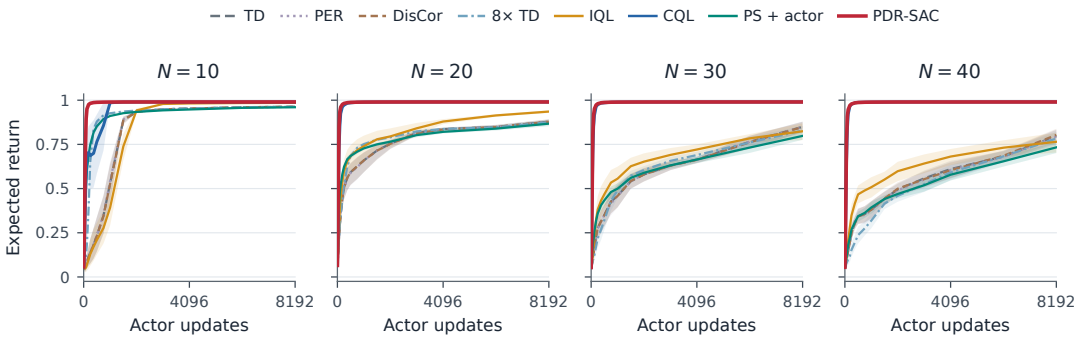


Figure B.1.: All current DeepSea continuations through 8,192 actor updates. This is the same selected trajectory for each seed and method as the early view, not a separately selected final-performance cohort.

B.4. Protocols of the Full-Revaluation Studies

SecureCourier continuation protocol. The 50 runs start from the checkpoints of Chapter 4. Paired conditions share the initial networks, optimizer states and replay. Each cycle collects one assisted episode and takes one critic, one actor and one target update. The critic learns by TD for all 2,048 cycles. Standard TD uses the critic's weights throughout. PDR and Exact (selected) replace the weights until cycle 1,024 and then return to the critic's weights (Table B.6). PDR's reference is the policy and critic saved before gate practice, and its graph is enumerated over the 16 variants. Except in the Exact (selected) intervention, exact values are used only for evaluation in SecureCourier.

SecureCourier critic baselines. A second comparison adds PER, DisCor and eight critic updates per actor update (8× TD) on the same 50 checkpoints. All five conditions are rerun with the original collection and AWAC objective. The critic interventions and PDR run until cycle 1,024, followed by ordinary training to cycle 2,048. Appendix C.5 gives the settings.

Public-benchmark inputs of the earlier studies. These definitions apply only to the earlier studies. Figure B.2 shows the tasks.

- **BabyAI KeyCorridor**: find a key, unlock a door and pick up a ball (Chevalier-Boisvert et al., 2019; Farama Foundation, 2026). We use S3R2, S3R3 and S4R3 (`-v0`), with limits of 270, 270 and 480 steps.
- **MiniGridLoCA**: a local reward change (van Seijen et al., 2020; Rahimi-Kalahroudi et al., 2023). T1's reward falls from four to one; T2's stays at two. A one-way region surrounds T1. Evaluation starts uniformly over states, with a 100-step limit.

- **DeepSea** (`bsuite`): N decisions to reach a treasure, with a small cost for moving right (Osband et al., 2020). We use $N = 10, 20$, with AWAC actors initialized from Bootstrapped DQN.

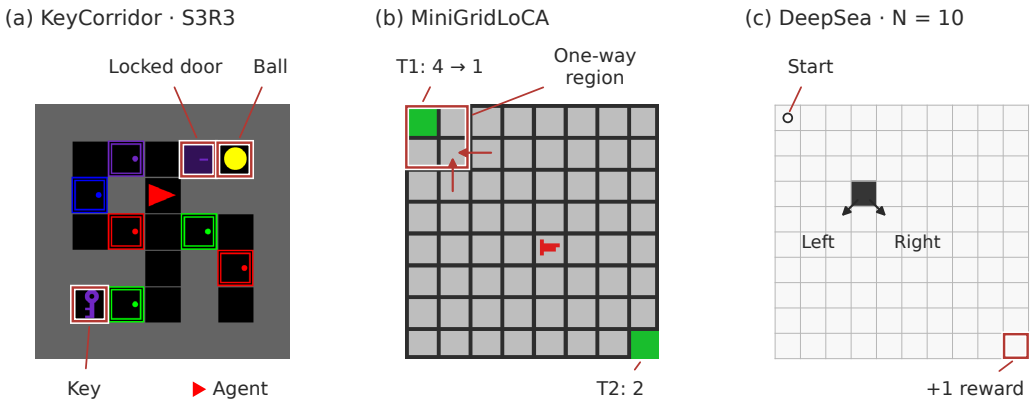


Figure B.2.: Public benchmark layouts. (a) KeyCorridor S3R3: collect the key, unlock the marked door and pick up the ball. (b) MiniGridLoCA: T1's reward changes from 4 to 1; T2 remains at 2. Arrows mark entry into a region the agent cannot leave. (c) DeepSea, $N = 10$: the filled cell is an example agent position. Arrows show left/right moves, each advancing one row. Right moves cost $0.01/N$; a final right move from the marked cell earns $+1$.

Compared methods. *Standard TD* uses the critic's values. *PDR* keeps a fixed reference, solves for the change before each actor update, and adds it to the reference. *Static reference* (Reference in Table 5.1) uses the reference values alone, so comparing it with PDR isolates the effect of the change. Within each comparison, the actor objective, initialization, data and schedule are the same. KeyCorridor and DeepSea use AWAC (Nair et al., 2020); LoCA also uses discrete SAC (Haarnoja et al., 2018) and Expected-SARSA QAC.

The AWAC gradient convention differs between studies. SecureCourier and LoCA detach the whole weight. KeyCorridor and DeepSea detach the values but keep the gradient through the policy average. Each study is therefore internally consistent, but the studies do not share one implementation (Appendix G.4).

KeyCorridor and LoCA compare exact and neural references; DeepSea uses an exact one. All use fixed data with complete transition support.[1] In LoCA, stored transitions are relabelled with Task-B rewards, which change only T1 arrivals. All critics train on these labels. PDR's source adds $r_B - r_A$ to the policy difference and, for SAC, the entropy difference.

[1] KeyCorridor's neural references fit a model-derived Bellman teacher; LoCA's use full-support Bellman backups. Exact references evaluate the old policy directly. Appendix E separates reference accuracy from transition support.

Additional critic baselines. On KeyCorridor S3R3, DeepSea $N = 10, 20$ and MiniGrid-LoCA with AWAC, we also compare PDR with PER (Schaul et al., 2016), DisCor (Kumar et al., 2020a) and 8× TD. These change critic sampling, loss weighting and update count. All five methods share the starting actor, critic and actor batches. PDR uses an exact reference; its critic still takes one TD step per actor update. Table B.6 summarizes the interventions.

Table B.6.: What changes between learning conditions. Entries describe the active intervention: cycles 1–1,024 in SecureCourier and the full actor-update budget on the public benchmarks. Exact (selected) is used only in SecureCourier; Static reference only on the public benchmarks. All conditions retain a live TD critic.

Method	Actor's value source	Records receiving replacement feedback	Critic updates per actor update
Standard TD	Live critic	None	1 ordinary TD
Static reference	Frozen old-policy values	All sampled records, without revaluation	1 ordinary TD
Exact (selected)	Current-policy exact values	Selected old shortcut records; other weights use the live critic	1 ordinary TD
PDR	Frozen reference plus differential	All sampled records; revalued before each actor update	1 ordinary TD
PER	Live critic	None	1, prioritized sampling
DisCor	Live critic	None	1, weighted TD loss
8× TD	Live critic	None	8 ordinary TD

Measurements and learning budgets. SecureCourier is evaluated every 16 cycles up to 2,048. Behaviour is averaged over the 16 variants, and signal metrics over fixed diagnostic records. Route probabilities and returns are exact under the current policy. Weight MAE compares each condition's weights on the old shortcut records with exact weights under its own policy. The gradient diagnostic uses a fixed 64-record batch per seed.

On the public benchmarks, learning axes count actor updates. Exact evaluation gives KeyCorridor success, LoCA Task-B return and T2 arrival, and DeepSea return. Table B.7 lists the windows.

The *early mean* is the trapezoidal area over the window divided by its length (1,024 cycles in SecureCourier; Table B.7 elsewhere). *Final* is the last evaluation.

Final SecureCourier success uses one greedy run per variant at cycle 2,048, for three tasks: full completion, completion on the safe route, and the gate sequence. The planned non-

Table B.7.: Learning measures and budgets. SecureCourier counts learner cycles; the public benchmarks count actor updates. Early means use normalized trapezoidal AUC; Final is the last evaluation.

Setting	Main learning measures	Early window	Full budget
SecureCourier	Weight MAE, shortcut probability, discounted return	0–1,024	2,048
KeyCorridor	Task-success probability	0–128	512
MiniGridLoCA	Discounted Task-B return; T2 arrival	0–1,024	4,096
DeepSea	Expected reward minus movement costs	0–2,048	8,192

inferiority test requires each difference (PDR minus TD) to have a one-sided 95% lower bound above -1 point.

Aggregation and uncertainty. SecureCourier intervals use 20,000 paired seed bootstrap resamples (seed 20260907); bands are pointwise 95% intervals. The earlier benchmark studies use all 20 seeds (ten per DeepSea size) and 95% paired intervals. Their bands show one standard error, except in the critic-baseline comparison, which shows 95% intervals.

B.5. Critic-Baseline Configurations

These settings belong to the earlier exact-reference studies (Appendices C and G), not to Chapter 6, whose search is in Table B.3.

B.5.1. KeyCorridor Baselines

Shared inputs. All five methods use `BabyAI-KeyCorridorS3R3-v0` (world seed 1000, 270-step limit) and share the actor, optimizer, old policy, exact old values and the 13,568-state graph. No new data are collected; the 20 seeds vary critic initialization and sampling.

Actor and critic. The actor has two 128-unit ReLU layers; the residual critic has two 256-unit layers and a zero-initialized output. Both use Adam with learning rate 3×10^{-4}. The discount is 0.97, AWAC temperature 0.3, weight cap 20, gradient-norm cap 5 and target coefficient 0.005. The actor loss detaches the values but keeps the gradient through the policy average.

Each actor batch mixes 58 demonstration records with six uniformly sampled records. PER keeps these groups. 8× TD uses the actor batch and then seven fresh batches,

updating the target after each step. PDR solves before each actor update and revalues every record. Table B.8 lists the transferred settings.

Table B.8.: Transferred critic-baseline settings, each selected from three candidates on three SecureCourier development seeds.

Method	Configuration
PER	Priority $\lvert\delta_{\mathrm{TD}}\rvert + 10^{-6}$, exponent 0.4, initial priority 1. Importance exponent increases from 0.4 to 1 over 512 updates. Weights correct towards the original within-group sampling law, normalized by the full-mixture maximum.
DisCor	Error network: one extra 256-unit ReLU layer, Softplus outputs, learning rate 10^{-4}. Targets use policy-expected successor error with terminal masking. Critic weights have mean one; error regression is unweighted. Error-target and temperature updates use coefficient 0.005; temperature starts at 10.
8× TD	Eight critic updates per actor update, giving 4,096 critic updates and 512 actor updates in total.

Evaluation. All 100 runs completed. The 14 checkpoints from update 0 to 512 are evaluated exactly. For each seed, the first four actor batches form a fixed panel of 256 records for weight MAE. Early means cover updates 0–128. Intervals use 10,000 paired seed bootstrap draws, not adjusted for multiplicity.

Software and hardware. Runs use Python 3.10.13, PyTorch 2.5.1 (CUDA 12.1), MiniGrid 3.0.0 and Gymnasium 1.3.0 on a server with AMD EPYC 7343 CPUs, 252 GiB of memory and eight RTX 3090 GPUs; all five methods of a seed share a GPU.

B.5.2. DeepSea and MiniGridLoCA Baselines

This study adds 120 PER, DisCor and 8× TD runs to 80 TD and PDR runs (ten seeds per DeepSea size, 20 in LoCA); all share the actor, critic, optimizer states and batches. Batches, learning rates and discounts follow Table B.1; AWAC temperatures are 0.3 in DeepSea and 0.25 in LoCA, DeepSea caps the gradient norm at 5, LoCA detaches the weights fully, and both cap weights at 20. PER and DisCor follow Table B.8, and PER keeps DeepSea's two replay groups. 8× TD takes 65,536 critic steps in DeepSea and 32,768 in LoCA. Evaluation is exact; intervals use 10,000 paired bootstrap draws.

Appendix C

Additional Results on SecureCourier

In this appendix, we give further SecureCourier results for Chapters 4 and 6: the full weight and repair curves, a replay-support study, and the full-batch revaluation comparison with its diagnostics and critic baselines. Protocols are in Section B.4.

C.1. Training Weights and Critic Repair

Figures C.1 and C.2 show every evaluation point behind Tables 4.1 and 4.2. Curves average 50 seeds of 16 instances each, with 95% bootstrap bands.

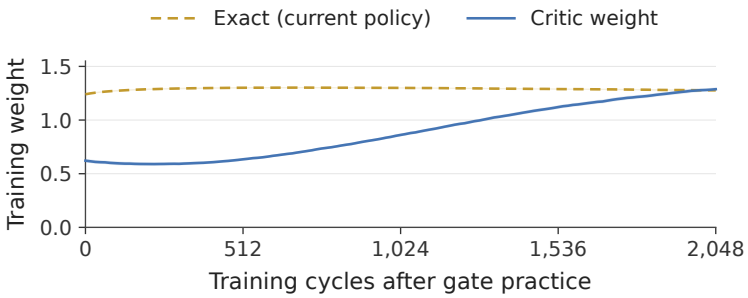


Figure C.1.: Shortcut weights during actor–critic continuation after gate practice. Exact weights use the current policy at each evaluation. Lines show 50-seed means; shading shows pointwise 95% bootstrap confidence intervals.

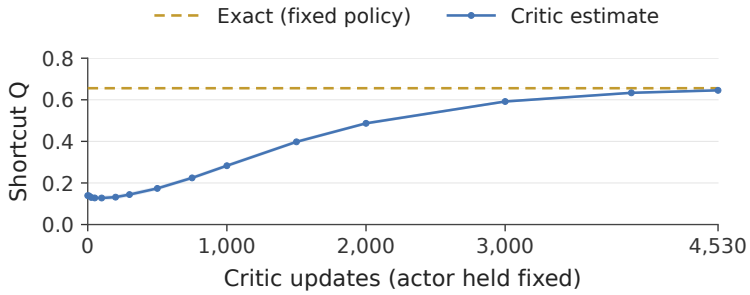


Figure C.2.: Shortcut Q at the junction during fixed-actor TD repair. Points mark the 15 recorded evaluations. Lines show 50-seed means; shading shows pointwise 95% bootstrap confidence intervals.

C.2. Replay Support and Critic Repair

Which replay records does the critic need to learn the shortcut's new value? We extend the fixed-actor experiment of Section 4.3 with three replay sets: all records, all records except junction shortcut actions, and all records except those in the shortcut corridors and gate states. The actor is fixed, and each run takes 4,530 TD updates with uniform batches of 64. The shortcut has a positive exact advantage in all 16 instances. We call an estimate *stale* while the critic still gives it a nonpositive advantage.

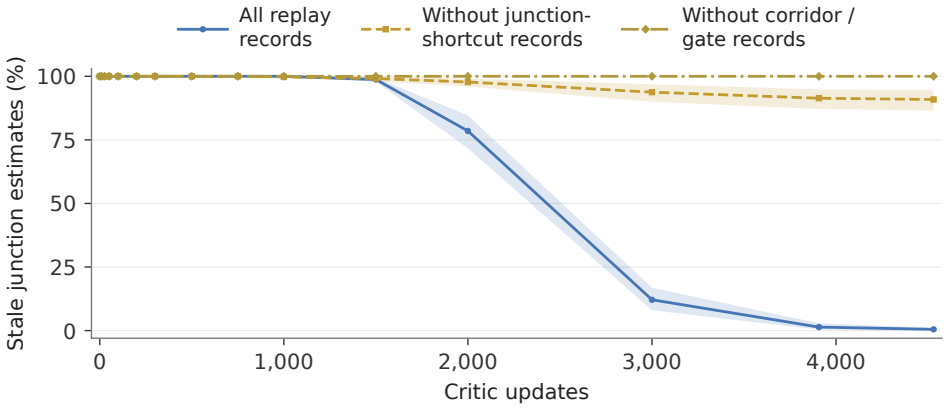


Figure C.3.: Stale junction estimates during fixed-actor TD training. Lines join means at all 15 recorded evaluations across 50 seeds, each measuring the stale fraction over 16 instances. Shading shows pointwise 95% bootstrap confidence intervals. The horizontal axis counts critic updates, ending at 4,530.

With all records, the stale fraction falls from 100% to 0.5% (Figure C.3).[1] Without junction shortcut records, 90.9% stay stale. Without corridor and gate records, all stay stale. TD therefore repairs the junction only when both parts of the route remain in replay.

C.3. Full-Batch Revaluation Results

Here we give the full SecureCourier comparison of Chapter 6. *Standard TD* uses the critic's weights. *Exact (selected)*, the Corrected-weights condition of Chapter 4, uses exact weights only for the old shortcut records. PDR corrects the whole batch from a learned reference. All use the same 50 checkpoints.

C.3.1. Shortcut Training Signal

PDR makes the initial shortcut advantage positive, as exact evaluation does (Figure C.4). The mean advantage is −0.145 with Standard TD, +0.064 with exact values and +0.115 with PDR. PDR therefore fixes the sign but overestimates the size. It also lowers early weight MAE from 0.621 to 0.373 (Figure C.9).

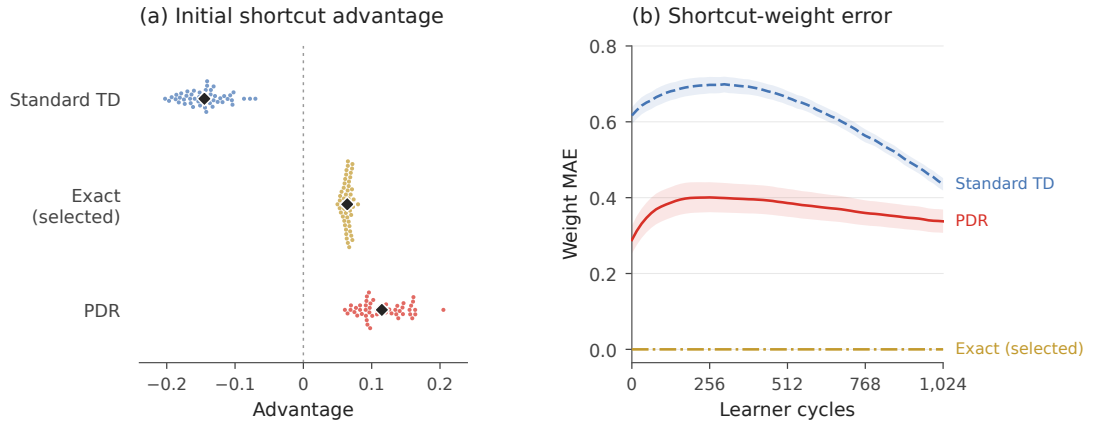


Figure C.4.: (a) Initial advantage on the selected old shortcut records. Small points show 50 paired seed means per method at the shared policy. Diamonds mark means: Standard TD −0.145, Exact (selected) +0.064, PDR +0.115. (b) Weight MAE against exact current-policy weights; Exact (selected) has zero error. Curves show means with pointwise 95% bootstrap intervals.

[1] In 37 of the 50 runs, the critic also ranks the shortcut first in every instance at some evaluation. Neither removal condition reaches this in any run.

PDR's initial gradient is also closer to the exact one (cosine 0.993 against 0.959), but Standard TD later overtakes it (Figure C.10).

C.3.2. Shortcut Choice

Shortcut use rises with the better weights (Figure C.5). From 18.9% at the junction, PDR reaches 33.4% at cycle 1,024, while Standard TD falls to 13.5% and Exact (selected) ends at 17.9%. From the depot, shortcut use rises from 25.9% to 40.1% with PDR, against 16.2% for Standard TD and 21.4% for Exact (selected).

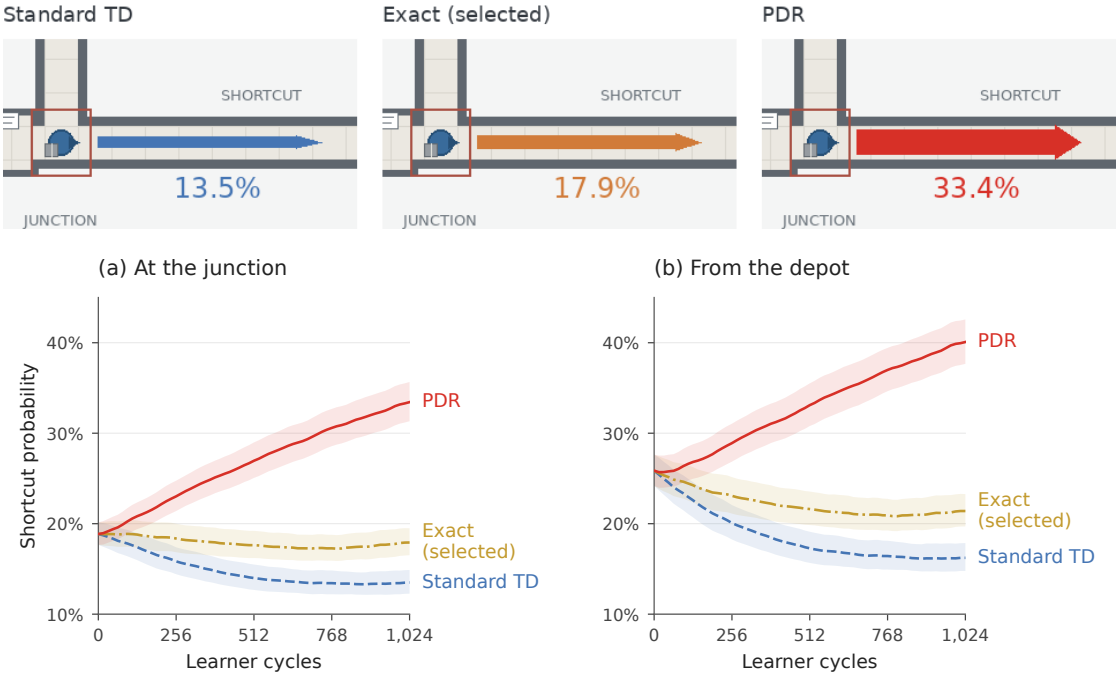


Figure C.5.: Using the shortcut. Top: the same junction view is overlaid with shortcut probabilities after 1,024 learner cycles: Standard TD 13.5%, Exact (selected) 17.9%, PDR 33.4%. Values are 50-seed means; arrow widths use a shared linear scale. Bottom: shortcut use over learner cycles 0–1,024. (a) Conditional probability at the canonical junction state at environment step eight. (b) Probability that a stochastic-policy journey from the depot uses the shortcut. Lines show 50-seed means; shaded bands show pointwise 95% bootstrap intervals.

C.3.3. Early Return

PDR also has a higher early return (Figure C.6): 0.47197 against 0.46233 for Standard TD, a gain of 0.00964 (95% CI $[0.00888, 0.01049]$). It also beats Exact (selected), which reaches 0.46503, by 0.00694 ($[0.00629, 0.00766]$). Exact (selected) corrects only the old shortcut records; PDR corrects the rest of the batch too. PDR beats both in all 50 seeds (Figure C.11).

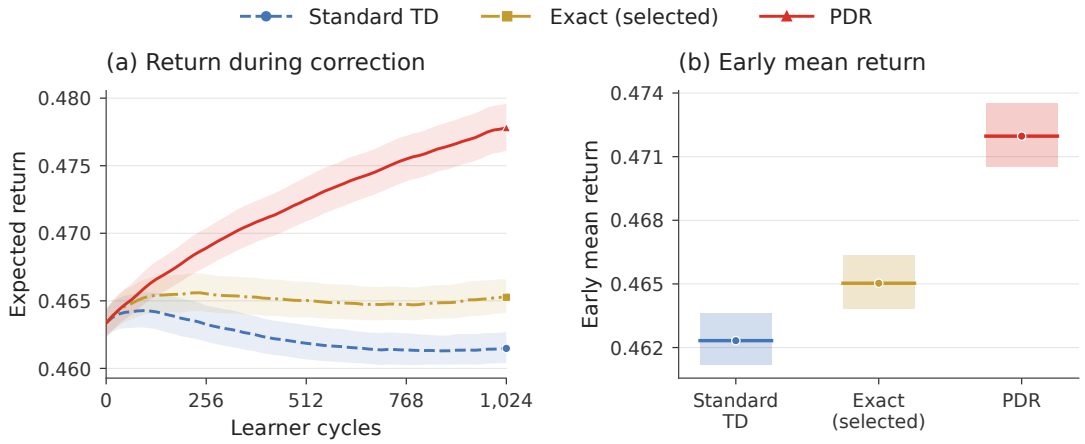


Figure C.6.: Expected discounted delivery return from the depot. (a) Exact stochastic-policy return over the correction window; curves show 50-seed means and pointwise 95% bootstrap intervals. (b) Early mean return, defined in Section B.4, averaged across the same 50 seeds. Markers show means: Standard TD 0.46233, Exact (selected) 0.46503, PDR 0.47197; shading shows 95% bootstrap confidence intervals.

C.3.4. Persistence after Correction

Most of the gain survives after correction stops (Figure C.7). PDR's junction probability dips to 29.2% at cycle 1,456 and then recovers. At cycle 2,048, the junction probabilities are 33.6% for PDR, 18.3% for Standard TD and 22.2% for Exact (selected); returns are 0.47794, 0.46546 and 0.46875.

Higher return does not mean higher final greedy success. At cycle 2,048, full-task success is 99.500% for PDR and 99.875% for Standard TD (difference -0.375 points, 95% CI $[-1.25, 0.25]$). The non-inferiority test fails, but the decrease is not significant either. Both keep 100% gate-sequence and safe-route success.

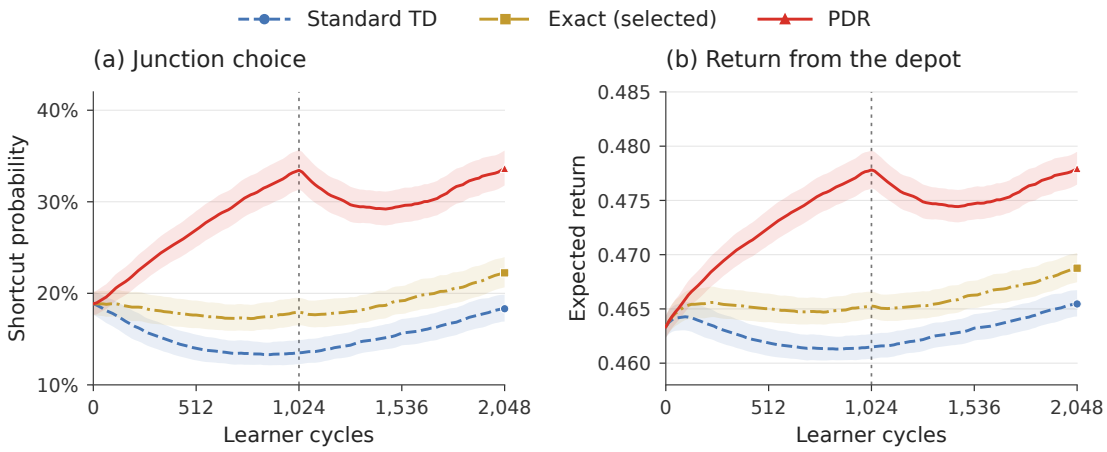


Figure C.7.: Continued training through cycle 2,048. (a) Conditional shortcut probability at the same junction state as in Figure C.5(a). (b) Expected discounted return from the depot. The dotted lines at cycle 1,024 mark the end of correction: PDR and Exact (selected) restore live-critic weights from cycle 1,025; Standard TD uses them throughout. Curves show 50-seed means with pointwise 95% bootstrap intervals.

C.3.5. Comparison with Alternative Critic Updates

We also compare PDR with PER, DisCor and 8× TD on the same checkpoints (Figure C.8).

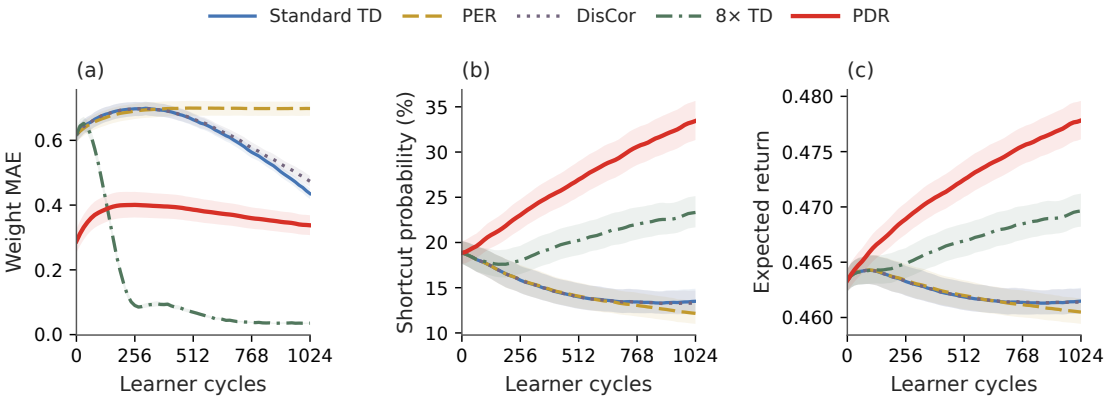


Figure C.8.: SecureCourier comparisons. (a) Weight MAE on fixed old shortcut records against each current policy's exact weights. (b) Junction shortcut probability. (c) Expected discounted depot return. Means over 50 paired seeds at every checkpoint; bands: pointwise 95% bootstrap intervals.

Table C.1.: Early means over cycles 0–1,024 (normalized AUC). Return gains are PDR minus each comparator, with paired 95% bootstrap intervals.

				PDR return gain ($\times 10^{-3}$)	
Method	Weight MAE	Shortcut (%)	Return	Gain	95% CI
Standard TD	0.621	14.80	0.46233	9.64	[8.88, 10.49]
PER	0.690	14.59	0.46222	9.74	[8.98, 10.59]
DisCor	0.629	14.78	0.46233	9.63	[8.87, 10.48]
8× TD	0.143	20.24	0.46675	5.21	[4.56, 5.92]
PDR	0.373	26.66	0.47197	—	

PER and DisCor stay close to Standard TD (Table C.1). 8× TD raises early return by 0.00443, and PDR adds a further 0.00521 ($[0.00456, 0.00592]$). PDR's shortcut probability is 6.42 points higher than 8× TD's ($[5.63, 7.25]$). 8× TD has the lower weight error (0.143 against 0.373), since it refines the critic while PDR relies on a fixed, imperfect reference. But the error is measured only on the old shortcut records, while the actor uses the whole batch. Appendix C.5 gives the full continuation (Figure C.12) and the paired effects (Table C.2).

C.4. Weight and Policy-Response Diagnostics

These diagnostics use the same 50 seeds. Early weight MAE falls from 0.621 to 0.373, a 39.9% reduction (paired difference -0.248, 95% CI $[-0.284, -0.211]$). The error is lower in 49 of 50 seeds (Figure C.9).

PDR's initial gradient is closer to the exact direction, but Standard TD overtakes it later in the window (Figure C.10). Exact (selected) stays closest.

PDR's gains over Standard TD exceed those of Exact (selected) in all 50 seeds (Figure C.11). The mean probability gains are 11.86 and 3.09 points; the mean return gains are 0.00964 and 0.00271.

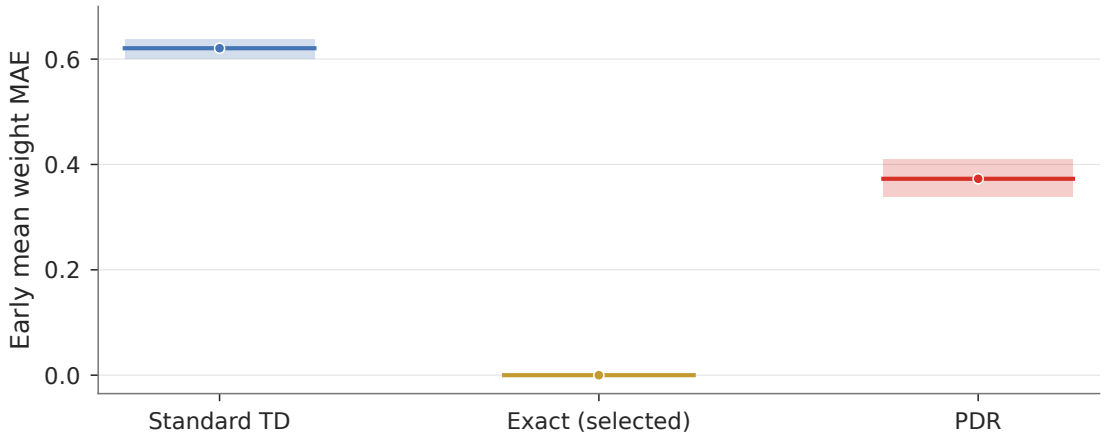


Figure C.9.: Early mean weight MAE on the selected old shortcut records. Markers show means across all 50 seeds; shading shows 95% bootstrap confidence intervals. Each seed's value is the normalized trapezoidal integral over cycles 0–1,024. Exact (selected) supplies the exact weights on these records throughout that window.

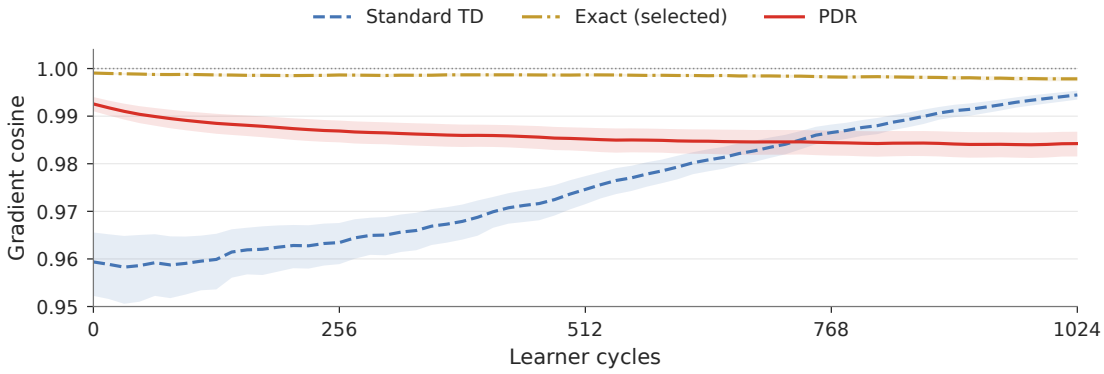


Figure C.10.: Cosine similarity to the exact-weight actor gradient. Each condition uses its current policy and the same fixed 64-record diagnostic batch for a given seed. The comparator gradient uses exact current-policy weights on every diagnostic record, whereas Exact (selected) corrects only the selected records. Lines show 50-seed means and bands show pointwise 95% bootstrap intervals.

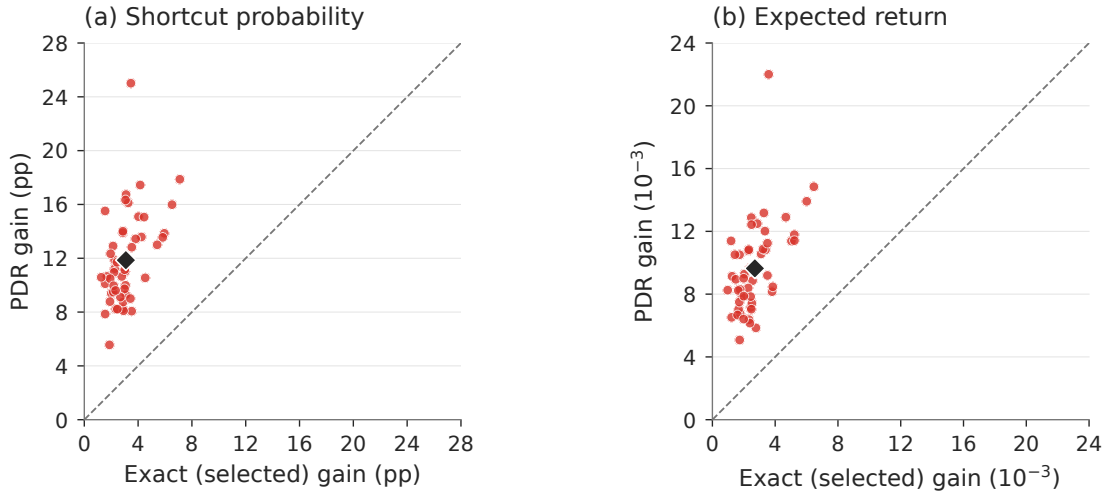


Figure C.11.: Paired early mean gains over each seed's Standard TD result. (a) Junction shortcut probability, in percentage points. (b) Expected discounted return, in units of 10^{-3}. Each point is a seed, and diamonds mark means. The horizontal axis is the Exact (selected) gain; the vertical axis is the PDR gain. Both axes share a scale within each panel, and the dashed diagonal marks equal gains.

C.5. Critic-Training Baselines

Comparison design. All five methods share each seed's initial networks, optimizer states and replay. Each cycle collects one assisted episode and takes one actor step. Actor batches mix six offline and 58 online records; online replay holds the latest 20,000 records. PDR corrects every record from the pre-practice reference. Both networks use Adam with learning rate 3×10^{-4}, discount 0.97, AWAC temperature 0.3, weight cap 20, gradient-norm cap 5 and target coefficient 0.005.

Critic interventions. PER (Schaul et al., 2016) changes only critic sampling. Priorities are $|\delta_{\mathrm{TD}}| + 10^{-6}$ with exponent 0.4, and the importance exponent rises from 0.4 to 1. DisCor (Kumar et al., 2020a) weights the critic loss with an error network (one 64-unit ReLU layer, Softplus output, learning rate 10^{-4}, initial temperature 10). 8× TD takes eight critic updates per actor step: 9,216 in total, against 2,048 otherwise. All methods detach the AWAC weights.

Selection and evaluation. Settings were chosen by early return on three separate seeds, from PER exponents 0.4, 0.6, 0.8; DisCor learning rates 10^{-4}, 3×10^{-4}, 5×10^{-4}; and critic-update ratios 2, 4, 8. They were then fixed for the 50 evaluation seeds. Interven-

tions stop after cycle 1,024. Evaluation follows Section B.4; intervals are not adjusted for multiplicity.

Execution. Runs use PyTorch 2.5.1 and CUDA 12.1 on the server of Appendix B.5.1. Within a seed, methods run on the same GPU in rotating order.

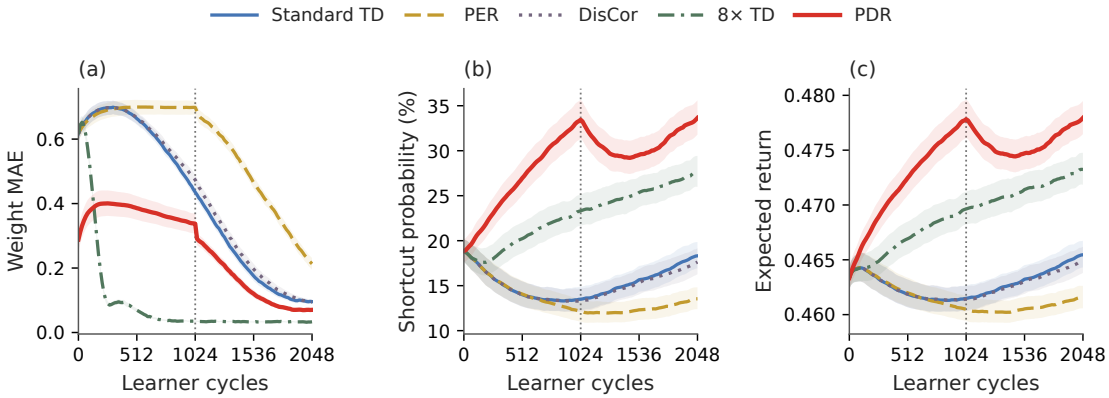


Figure C.12.: Full SecureCourier continuation, with metrics and bands as in Figure C.8. At cycle 1,024 (dotted), all methods resume ordinary TD and live-critic actor weights.

Table C.2.: Early paired effects with 95% bootstrap intervals (PDR minus comparator). Negative weight-MAE effects mean lower error. Shortcut effects are percentage points; log-odds are $\log p(\text{shortcut})$ minus $\log p(\text{safe})$.

	Weight MAE		Shortcut (pp)		Log-odds	
Comparator	Effect	95% CI	Effect	95% CI	Effect	95% CI
Standard TD	−0.248	[−0.284, −0.211]	11.86	[10.94, 12.85]	0.827	[0.775, 0.880]
PER	−0.317	[−0.355, −0.278]	12.07	[11.16, 13.06]	0.842	[0.791, 0.894]
DisCor	−0.256	[−0.292, −0.219]	11.88	[10.97, 12.86]	0.830	[0.779, 0.882]
8× TD	0.230	[0.197, 0.265]	6.42	[5.63, 7.25]	0.396	[0.351, 0.441]

At cycle 2,048, PDR keeps the highest shortcut probability and return (Table C.3). Greedy success is 99.5–100% for all methods.

Table C.3.: Cycle 2,048: stochastic shortcut probability and return; greedy Task, forced Gate and forced Safe success (Section B.4). Scores average 16 variants per seed, then 50 seeds.

Method	Shortcut (%)	Return	Task (%)	Gate (%)	Safe (%)
Standard TD	18.32	0.46546	99.875	100.000	100.000
PER	13.54	0.46155	99.875	100.000	100.000
DisCor	17.63	0.46493	100.000	100.000	100.000
8× TD	27.64	0.47324	99.875	100.000	100.000
PDR	33.65	0.47794	99.500	100.000	100.000

C.6. Gate and Delivery States

Figure C.13 shows the gate and delivery states of the scripted shortcut used in Chapter 4.

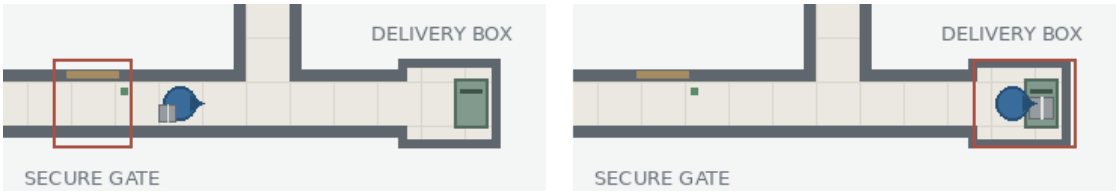


Figure C.13.: Gate use and delivery in the 2D view of Figure 4.1, after opening the gate at episode action 17 (left) and placing the parcel in the delivery box at action 22 (right). These recorded states follow the environment's scripted shortcut plan: scan the badge, enter the code, open the gate, then complete the delivery. Red boxes mark the gate and delivery box.

Appendix D

Additional Results on Signal Timing

In this appendix, we give further results for Chapter 7: robustness checks, actor controls, a longer-distance study, measurement details and the full backup-order profiles.

D.1. Synchronous Propagation and Distance Convention

In the controlled study, h is the shortest Bellman distance from a probe pair to a nonzero entry of the source c. After t synchronous sweeps, the change includes terms up to depth $t-1$, so a probe with $h \geq t$ has no correction yet. Figure D.1 shows this boundary. Over all five sizes, 131,850 such rows were checked; the largest value was 2.78×10^{-17}.

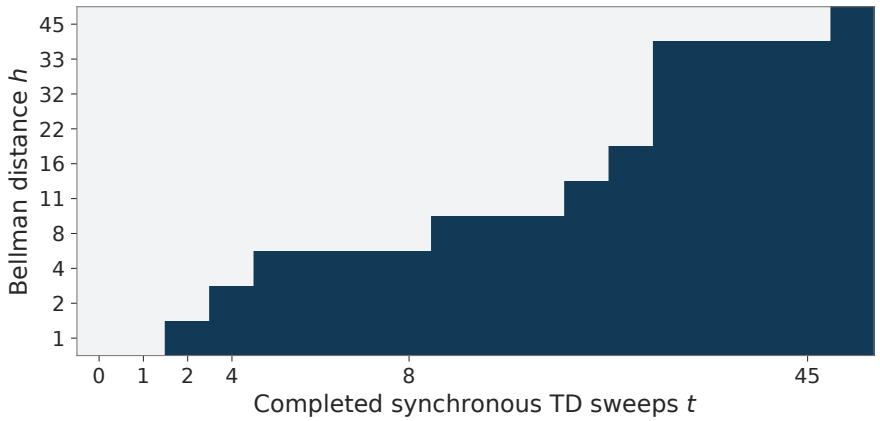


Figure D.1.: Synchronous TD support at $N = 50$. Dark cells indicate a nonzero correction above 10^{-14}, and pale cells indicate its absence. Columns enumerate completed sweep counts observed in the checkpoint grid, through 50 sweeps; rows enumerate the ten registered probe distances. The figure aggregates the identical synchronous corrections across events.

D.2. Sensitivity to Graph Size and Response Threshold

The TD distance slope is positive at every size (Figure D.2): 151.99, 120.01, 373.54, 465.74 and 454.11 reads per layer for $N = 10, 20, 30, 40, 50$. The slope need not grow with N, since each size has a different replay and distance grid. At $N = 10$, 29 of 30 event slopes are positive; at every larger size, all 30 are. PDR's slope is zero throughout.

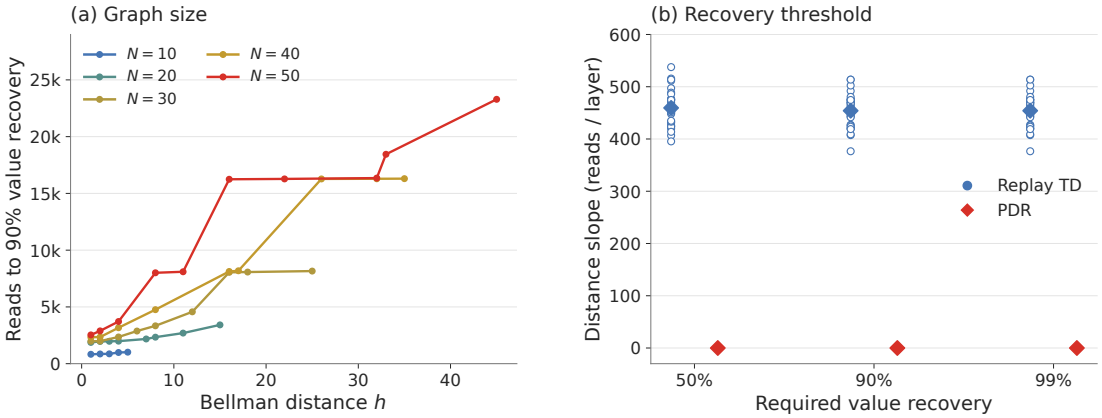


Figure D.2.: Size and threshold checks. (a) Median replay-TD value response latency at each registered distance, with schedules aggregated within the 30 events at each size. (b) Distance slopes for the 50%, 90% and 99% value-recovery criteria at $N = 50$. Circles show event slopes; diamonds mark their means. No response in these comparisons is censored.

The main analysis uses the planned 90% threshold. The 50% and 99% thresholds give the same separation. Timings are the first qualifying checkpoints, so the curves show the steps of the checkpoint grid.

D.3. Actor-Response Controls

Actor recovery divides the one-update change in $P(\text{enter})$ by the change under exact values (plus 10^{-12}). It is not clipped, so an update against enter has negative recovery. PDR gives recovery one; blocking the change gives about -1.4738. Their difference is 2.47380 in every event, so its interval is degenerate. Restoring the change improves recovery by 2.45975 (95% CI $[2.45507, 2.46383]$; Figure D.3a).

The PDR and restored gradients point the same way as the exact gradient (Figure D.3b). The blocked gradient is reversed, and the mean TD cosine is -0.99867. Table 7.2 gives the size of the probability change.

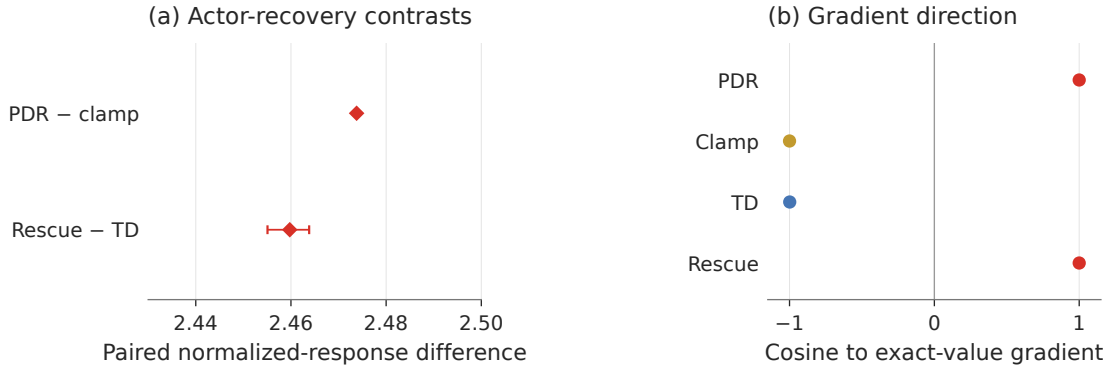


Figure D.3.: Additional actor-mediation measurements at $N = 50$. (a) Registered block and rescue contrasts with event-bootstrap 95% intervals. (b) Mean cosine similarity to the exact-value actor gradient: $+1$ is aligned and -1 is reversed. Every pulse shares the actor parameters, optimizer, two-action batch and random-number state.

Two null controls change the policy without changing value. Over all 150 events and five schedules, both give zero difference in probability, response and gradient (tolerance 10^{-12}). A policy change without a Bellman source therefore creates no correction.

D.4. Longer-Distance Actor Continuation

An earlier study extends the task to $h = 495$ with ten learned checkpoints. Here the actor keeps updating over many sweeps, and each probe's two actions appear once per sweep (Figure D.4, $N = 500$). It has its own seeds and schedule, so we do not pool it with the main study.

At the 0.8 probability threshold, the response lead grows with distance in all ten seeds: by 1.003173 actor sweeps per step (95% CI [1.002686, 1.003639]). No responses are censored. This is the multi-update counterpart of the single-update result in Chapter 7.

D.5. Measurement Details

Task and policies. The task has a five-state skill module and $N - 5$ upstream layers, for $N \in \{10, 20, 30, 40, 50\}$. In the module, the correct-action probability rises from 0.70 to 0.95. Wrong actions end the episode with zero reward; correct actions cost 0.002; the treasure pays one. Each probe offers *enter*, a zero-reward chain to the module, and *safe*, which ends with reward 0.500. With $\gamma = 1$, enter's value rises from 0.164 to 0.765

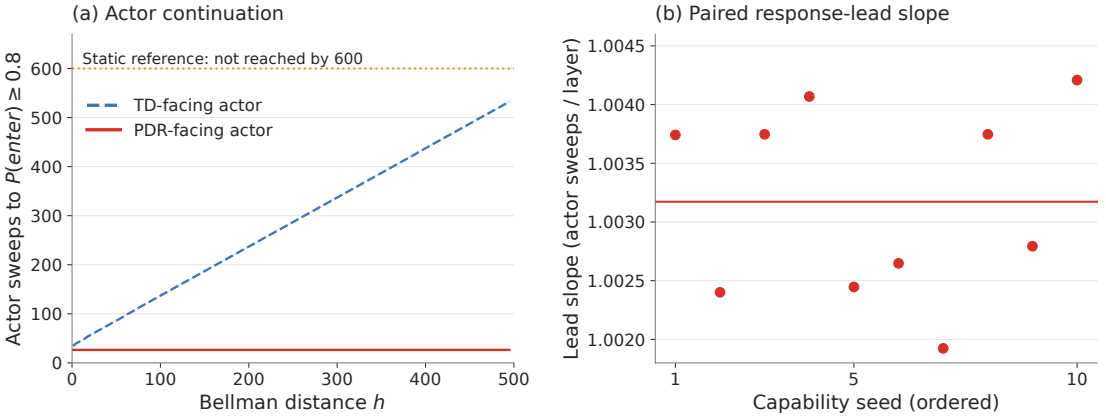


Figure D.4.: Retrospective long-distance continuation. (a) Sweeps to $P(enter) \geq 0.8$ for every probe at $N = 500$. Lines show medians over ten capability seeds; bands span their minimum and maximum. Static reference never reaches the threshold within 600 sweeps, marked as a censoring boundary. (b) The ten seed-level response-lead slopes from the original cross-distance analysis; the horizontal line marks their mean.

at every probe. Revaluation starts from the exact Q_{old}, with both policies fixed. The graph is acyclic and closed, so the series is finite.

Response measurements. For exact change D^* and measured change $\widehat{D}$, value recovery is $1 - |\widehat{D} - D^*|/(|D^*| + 10^{-12})$. A value response needs 90% recovery with the correct sign. Each actor pulse resets a two-action softmax actor to $P(\text{enter}) = 0.5$ and takes one detached AWAC update with both actions once (SGD, learning rate 0.1, temperature 0.1, weight cap 20). An actor response needs 90% of the exact-value change in the same direction. Pulses do not affect the fixed policy.

Logical reads and aggregation. Logical reads count accesses to replay rows, edges, policy probabilities and successor values. Source computation is counted; index preparation is not. Latency is read at the first qualifying checkpoint on a grid of 157 checkpoints up to 282,112 reads. Curves take medians over schedules, then over events; slopes are fitted per event and averaged. Intervals use 10,000 event-bootstrap resamples. Truncation depths are 0, powers of two, and $h-1, h, h+1$ for each h, within $[0, N-5]$. The distance grids are:

N	Bellman distances h
10	$1, 2, 3, 4, 5$
20	$1, 2, 3, 4, 7, 8, 11, 15$
30	$1, 2, 4, 6, 8, 12, 16, 18, 25$
40	$1, 2, 4, 8, 16, 17, 26, 32, 35$
50	$1, 2, 4, 8, 11, 16, 22, 32, 33, 45$

Each nonzero dependency costs one row read plus one edge, one probability and one successor value. On the $N = 50$ graph, PDR's source touches 145 rows and 99 dependencies, so it costs $145 + 3(99) = 442$ reads. Its maximum value error over all sizes is 1.11×10^{-16}.

The block/rescue swap uses the first checkpoint where PDR has responded and TD has not. This window exists for 6,148 of 6,150 units; the two missing units are not replaced. All 1,500 units at $N = 50$ are available.

D.6. Backup Order and Distance Dependence

Figure D.5 shows all ten distances for Table 7.1; Table D.1 gives the paired interval.

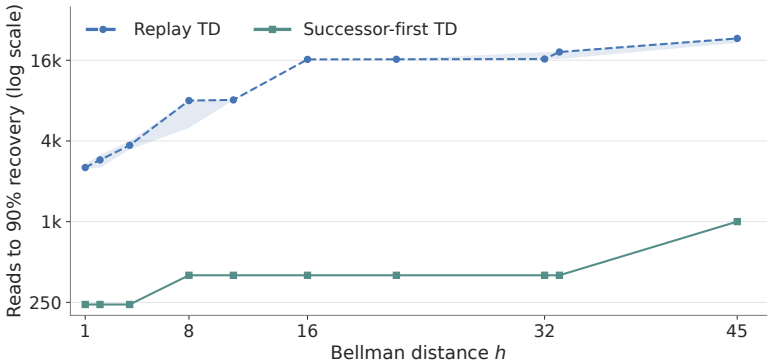


Figure D.5.: Actual reads to 90% value recovery under replay and successor-first TD at $N = 50$. Points mark the registered distances on a numeric axis; bands show event-bootstrap 95% intervals. The read axis is logarithmic.

Table D.1.: Distance slopes relative to replay TD at $N = 50$. The successor-first ratio is 2.55% (paired event-bootstrap 95% CI: $[2.49, 2.62]$%); PDR's slope is numerically zero. Figure D.5 shows the corresponding TD latencies.

Method	Relative slope (%)	95% CI (%)
Replay TD	100	—
Successor-first TD	2.55	[2.49, 2.62]
PDR	0	—

Appendix E

Additional Results on Signal Content

In this appendix, we give further results for Chapter 8: all Common-only conditions, the critic-error tables, and two earlier studies of PDR's inputs, namely the accuracy of the reference and missing graph transitions.

E.1. Complete Common-Shift Control Results

Table E.1 reports all seven actor inputs of Section 8.1 for both references. Each uses 20 seeds (different sets) and 4,096 updates; early values cover updates 0–1,024. The planned equivalence checks (95% interval within ±0.01) pass for Relative-only against Full PDR, Common-only against Reference, and Direct PE against Full PDR, for both return and T_2 arrival.

Table E.1.: Common-only ablation in MiniGridLoCA (AWAC): early and final Task-B return and T_2-arrival probability, means over 20 seeds.

	Exact reference			Learned reference		
Actor input	Early	Final	Early T_2	Early	Final	Early T_2
Live critic (TD)	0.108	0.535	0.025	0.100	0.542	0.020
Reference	0.384	0.403	0.017	0.384	0.384	0.018
Common-only	0.384	0.405	0.017	0.383	0.386	0.018
Reversed	0.364	0.387	0.004	0.363	0.374	0.004
Relative-only	1.112	1.224	0.714	1.117	1.236	0.715
Full PDR	1.112	1.220	0.713	1.117	1.236	0.715
Direct PE	1.112	1.220	0.713	1.117	1.234	0.715

At the start (learned reference), Q MAE is 2.717 for Reference and the critic, 0.038 for Common-only, 2.751 for Relative-only and 2×10^{-5} for Full PDR. Advantage MAE to Full PDR is 0.036 for Reference and Common-only and 1.5×10^{-7} for Relative-only. So Relative-only has the largest Q error but the same signal as Full PDR.

One-update pulses at checkpoints 0, 128 and 1,024 keep the actor, its optimizer state and the batch fixed. At update 128, return changes by +0.0046 for Full PDR, Relative-only

and Direct PE, and by −0.0028 for Reference and Common-only. At update 0, just after the reward switch, every input lowers return.

E.2. Critic-Error Decomposition

Table E.2 gives the SecureCourier values behind Figure 8.2, with reductions relative to update 0.

Table E.2.: SecureCourier fixed-actor TD repair at the junction shortcut: mean absolute Q error, common shift $|\bar{e}|$ and relative error $|\tilde{e}|$, and the percentage of instances with the correct signal sign; 50 seeds, 16 instances each.

Critic updates	Q error	$\lvert\bar{e}\rvert$	$\lvert\tilde{e}\rvert$	Correct sign (%)
0	0.517	0.307	0.210	0.0
500	0.482	0.258	0.224	0.0
1,000	0.373	0.194	0.179	0.0
1,500	0.258	0.132	0.126	1.3
2,000	0.169	0.084	0.085	21.5
3,000	0.064	0.026	0.038	87.9
4,530	0.017	0.012	0.021	99.5

For MiniGridLoCA, we record each critic's actor input at updates 0–512, divide out its value gain, and compare it with exact values ($\gamma = 0.99$; soft values for SAC). At update 0, 99.97% of the squared error is common shift in all 240 TD, PER, DisCor and 8× TD runs. Over 512 updates, TD, PER and DisCor do not reduce Q MAE (AWAC: 2.71 to 3.11). Meanwhile, the share of states where the AWAC critics rank the best action first falls from 0.96 to about 0.38.

E.3. Sensitivity to Reference and Transition Support

These studies use the earlier full-revaluation configurations. Two inputs matter. The reference values enter the local change (Equation (5.22)). The transitions decide which successors enter the solve (Equation (5.26)). We vary each in turn.

E.3.1. Learned Reference Accuracy

We vary the reference error in LoCA–AWAC and keep everything else fixed. From the saved critic $\widehat{Q}_{\text{old}}$ and the exact values $Q^{\text{exact}}_{\text{old}}$, we build

$$Q_\lambda = Q^{\text{exact}}_{\text{old}} + \lambda\big(\widehat{Q}_{\text{old}} - Q^{\text{exact}}_{\text{old}}\big), \qquad \lambda \in \{0, 0.25, 0.5, 0.75, 1\}.$$

This scales the critic's errors without changing their pattern: $\lambda = 0$ is exact and $\lambda = 1$ is the learned reference. PDR revalues every record; Static uses Q_λ directly.

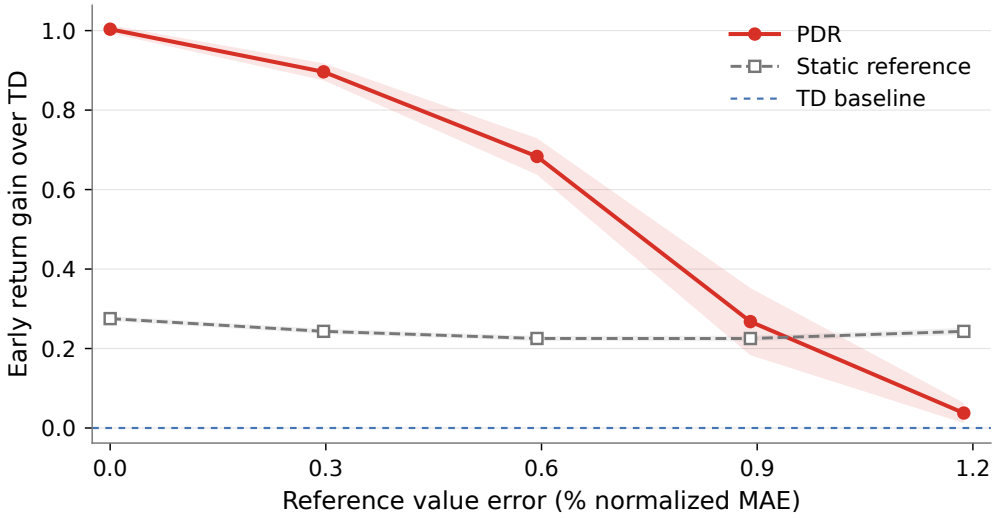


Figure E.1.: LoCA–AWAC: early return gains over TD between the exact and original learned references ($0 \leq \lambda \leq 1$). Early return is normalized AUC over actor updates 0–1,024, using exact 100-step policy evaluation. Points are paired means; bands are simultaneous 95% bootstrap confidence bands (20 paired seeds; 10,000 draws). Reference error is MAE divided by mean absolute exact Task-A value, averaged across runs. Lines join the measured points.

PDR beats TD at all five levels, with bands above zero (Figure E.1). As the reference error falls from 1.19% to zero, the gain grows from 0.0376 to 1.0035.

The gain falls with the reference's Bellman residual. Let $e_{\text{ref}} = \overline{Q}_{\text{old}} - T^{\pi_{\text{old}}}\overline{Q}_{\text{old}}$, where $T^{\pi_{\text{old}}}$ is the old policy's Bellman operator under the old reward. Subtracting the Bellman equation of $Q^{\pi_{\text{new}}}$ from Equation (5.26) gives

$$Q_{\text{PDR}} - Q^{\pi_{\text{new}}} = \left(I - \gamma P^{\pi_{\text{new}}}\right)^{-1} e_{\text{ref}},$$

which is Proposition 1 when $e_{\text{ref}} = 0$. The solve adds up the residual along the new policy's paths, with total weight up to $1/(1-\gamma) = 100$ here, and scaling the reference error by λ scales this error by λ. The saved critic used in this sweep has a maximum Bellman residual of 0.705 on average. The MiniGridLoCA learned references of Chapters 6 and 8 are fitted to Bellman backups on the full graph and are used only if their maximum residual is below 10^{-4}; with them, full PDR gains about 1.0 over TD (Section 8.1). A learned reference for PDR therefore needs to be consistent with the Bellman equation, not only close in value.

E.3.2. Missing Graph Transitions

We now remove transitions from PDR's graph, keeping the replay and the exact reference fixed. We use the AWAC runs on KeyCorridor S3R3, MiniGridLoCA and DeepSea $N = 10, 20$.

Critic Values at Missing Transitions

The closed-graph solve needs every successor. Here we fill missing transitions with the current critic. Let $\mathcal{I}$ index observed rows and $\mathcal{B}$ missing rows. We set $\widehat{D}_{\mathcal{B}} = (Q_{\mathrm{TD}} - Q_{\mathrm{old}})_{\mathcal{B}}$ and solve

$$\big(I - \gamma \widehat{M}_{\mathcal{I}\mathcal{I}}\big)\widehat{\boldsymbol{D}}_{\mathcal{I}} = \widehat{\boldsymbol{c}}_{\mathcal{I}} + \gamma \widehat{M}_{\mathcal{I}\mathcal{B}}\widehat{\boldsymbol{D}}_{\mathcal{B}}. \tag{E.1}$$

The blocks of $\widehat{M}$ use the observed transitions and the current policy. In LoCA, the source also includes the reward change.

After each TD step, PDR solves and gives $Q_{\mathrm{old}} + \widehat{D}$ to every actor record. Missing pairs receive the critic's value. Zero coverage is plain TD; full coverage is full PDR.

For each run, we shuffle the transition rows and keep nested subsets at 0, 25, 50, 75, 90, 95, 99 and 100% coverage. Each mask is fixed during training; KeyCorridor uses the same mask at every time layer. Everything else is fixed across coverage levels. The budgets are 512 updates for KeyCorridor, 4,096 for LoCA and 8,192 for DeepSea, with 20 runs each for the first two and ten per DeepSea size (480 runs in total). Gains and losses at the six intermediate levels use simultaneous 95% intervals from 10,000 bootstrap resamples.

Value Accuracy

We measure Q MAE over all legal actions in the first actor batch, against exact values under each run's current policy. Advantage and weight MAE use the recorded actions. With an exact reference, Q MAE also measures the error in D.

Q error falls with coverage in all four settings (Figure E.2). At 99%, it is 88.3% below TD in KeyCorridor, 86.2% in DeepSea $N = 10$ and 95.6% in DeepSea $N = 20$. LoCA improves more slowly: from 3.050 to 2.289 (24.9%). Full graphs bring all errors to numerical precision ($\leq 1.6 \times 10^{-8}$).

Q error and weight error can move apart. In LoCA at 50% coverage, Q MAE falls from 3.050 to 3.029, but advantage MAE rises from 0.0471 to 0.0563 and weight MAE from 0.2351 to 0.2729. At 99%, they are 0.1100 and 0.4913. A lower Q error therefore does not mean a better actor signal.

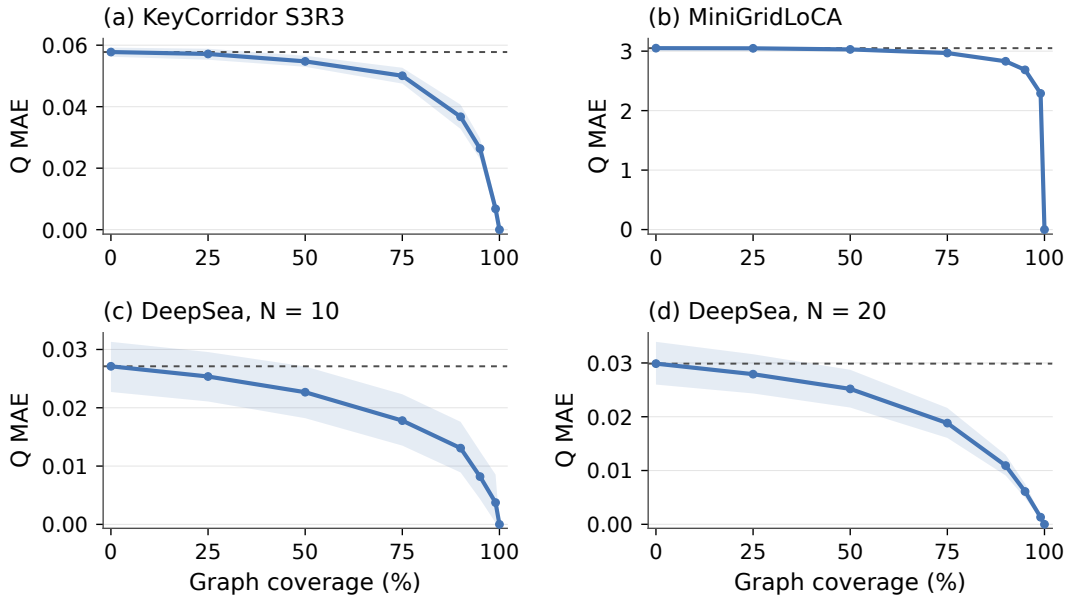


Figure E.2.: Q error as graph coverage increases. Points show the area under the MAE curve divided by the early-window length, averaged across runs. Windows are updates 0–128 for KeyCorridor, 0–1,024 for LoCA and 0–2,048 for DeepSea. Shading shows pointwise 95% bootstrap intervals; dashed lines show standard TD. Straight lines join measured means. Lower MAE means greater accuracy.

Early and Final Performance

We measure KeyCorridor success, LoCA Task-B return and DeepSea return, over the early window and at the end of training.

KeyCorridor and DeepSea $N = 10$ beat TD at every nonzero coverage. In KeyCorridor, the early gain grows from 1.04 points at 25% to 8.18 points on the full graph. DeepSea $N = 20$ gains clearly at 90% and above. In LoCA, early return falls below TD at 25–75% and exceeds it at 95% and 99%; the gain is 0.0408 at 99% and 1.0035 on the full graph (Figure E.3).

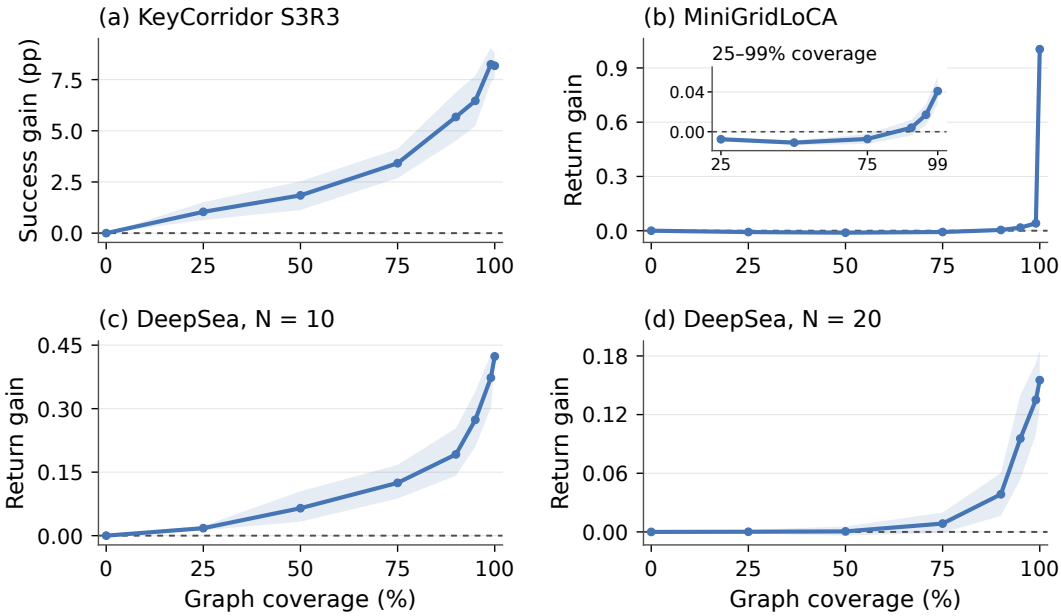


Figure E.3.: Paired early-performance gains over standard TD, in percentage points (pp) for KeyCorridor and return units elsewhere. Shading shows pointwise 95% bootstrap intervals. The LoCA inset enlarges the incomplete-graph conditions. Lines join measured means.

Final results differ (Figure E.4). KeyCorridor keeps gains of 0.86–3.15 points. LoCA at 75% ends 0.0638 above TD despite its early loss. DeepSea $N = 10$ shows no clear final difference, and DeepSea $N = 20$ ends below TD at 50–90%. So incomplete graphs can help early learning, but the effect depends on coverage and on the time window.

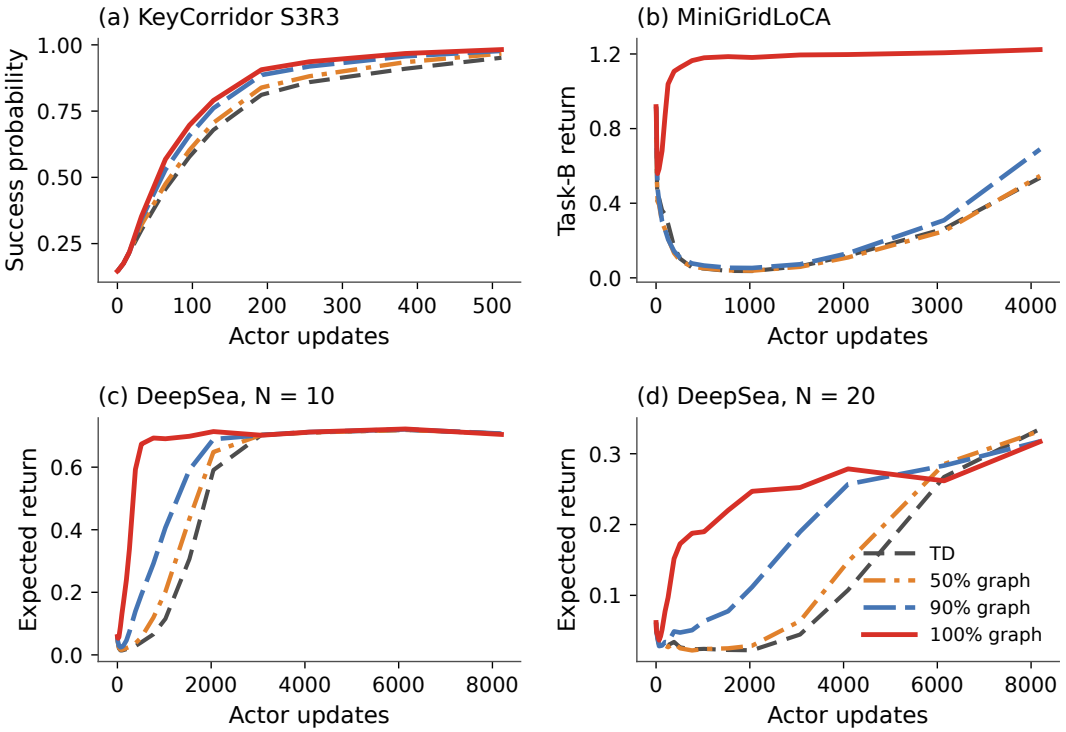


Figure E.4.: Learning curves through the full training budget for standard TD and PDR with 50%, 90% and 100% graph coverage. Colours and line patterns distinguish the four conditions; complete-graph PDR is the red solid line. The horizontal axis counts actor updates. Lines join mean performance at the recorded checkpoints without smoothing. The metrics are defined in Appendix E.3.2.

Appendix F

Additional Results on Signal Composition

In this appendix, we test other ways to combine the two parts of the signal, for Chapter 9.

These studies use the earlier configurations: AWAC in KeyCorridor and DeepSea, all three actors in MiniGridLoCA, and the earlier DeepSea AWAC recipe with an exact reference (not PDR-SAC). Every arm starts from the same checkpoints and batches. The seeds are the development seeds (20 per setting; 10/10/9 in DeepSea), so the results are exploratory. Entries are early means; KeyCorridor reports success probability and the others return.

F.1. Ranking-Based Gates

Gate-any switches a state from full revaluation to ΔA when the reference and the current values rank any two actions in opposite order. Gate-top switches when they disagree on the best action.

The gates rarely switch (Table F.1); in DeepSea at $N = 10, 20$ they never do, so they match full revaluation. In this DeepSea AWAC recipe, full revaluation at gain 1 also learns little at $N = 20, 30$. A common scale of 8 recovers much of the return, while change only fails at both scales.

F.2. Weight Mixtures

For AWAC, we also combine the capped weights of Full 8 and Δ 8. Mix-raw averages them; Mix-norm first divides each by its batch mean. Table F.2 gives the early means.

Table F.1.: Full revaluation, change only and statewise gates at gain 1, with full revaluation and change only at gain 8.

Setting	Full 1	Δ 1	Gate-any 1	Gate-top 1	Full 8	Δ 8
KeyCorridor S3R2, exact	0.199	0.172	0.168	0.185	0.241	0.198
KeyCorridor S3R2, learned	0.197	0.170	0.166	0.182	0.240	0.196
KeyCorridor S3R3, exact	0.499	0.511	0.483	0.500	0.465	0.606
KeyCorridor S3R3, learned	0.508	0.521	0.493	0.509	0.478	0.606
KeyCorridor S4R3, exact	0.478	0.458	0.482	0.478	0.305	0.491
KeyCorridor S4R3, learned	0.483	0.465	0.486	0.483	0.314	0.493
KeyCorridor S5R3, exact	0.732	0.756	0.742	0.732	0.736	0.813
LoCA AWAC, exact	1.112	1.344	1.349	1.349	1.666	1.726
LoCA AWAC, learned	1.117	1.350	1.355	1.356	1.665	1.725
LoCA SAC, exact	1.708	1.757	1.726	1.734	1.694	1.755
LoCA SAC, learned	1.682	1.759	1.724	1.731	1.658	1.758
LoCA QAC, exact	1.680	1.757	1.725	1.732	1.676	1.757
LoCA QAC, learned	1.663	1.760	1.721	1.724	1.674	1.759
DeepSea $N = 10$, AWAC	0.386	−0.004	0.386	0.386	0.856	−0.004
DeepSea $N = 20$, AWAC	0.009	−0.004	0.009	0.009	0.676	−0.004
DeepSea $N = 30$, AWAC	−0.003	−0.004	−0.003	−0.003	0.438	−0.004

Table F.2.: Mixtures of AWAC weights from full revaluation and change only, both at gain 8.

Setting	Full 8	Δ 8	Mix-raw	Mix-norm
KeyCorridor S3R2, exact	0.241	0.198	0.250	0.246
KeyCorridor S3R2, learned	0.240	0.196	0.248	0.243
KeyCorridor S3R3, exact	0.465	0.606	0.528	0.548
KeyCorridor S3R3, learned	0.478	0.606	0.540	0.554
KeyCorridor S4R3, exact	0.305	0.491	0.364	0.441
KeyCorridor S4R3, learned	0.314	0.493	0.358	0.442
KeyCorridor S5R3, exact	0.736	0.813	0.784	0.771
LoCA AWAC, exact	1.666	1.726	1.703	1.696
LoCA AWAC, learned	1.665	1.725	1.702	1.695
DeepSea $N = 10$, AWAC	0.856	−0.004	0.671	0.491
DeepSea $N = 20$, AWAC	0.676	−0.004	0.022	0.001
DeepSea $N = 30$, AWAC	0.438	−0.004	−0.004	−0.004

Appendix G

Computational Cost Analysis

In this appendix, we report computational costs. All studies use the earlier full-revaluation configurations (Section B.4), not the weighted configurations of Chapter 6 (see Section 6.3). We first compare optimized PDR with the critic baselines, and then compare native and optimized PDR.

Retiming protocol. We retime TD, PER, DisCor, 8× TD and Optimized PDR on the four exact-reference settings of Appendix B.5.1, with all 60 seeds. For each seed, the five methods run one after another on one RTX 3090, in rotating order. Up to eight workers use separate GPUs, separate CPU cores and one thread each, on a server with two AMD EPYC 7343 CPUs (64 threads) and 251.8 GiB of memory running Ubuntu 24.04.3. One clock covers training; a second adds loading and preparation. Evaluation is timed separately (Appendix G.2).

G.1. Cost Comparison with Critic Baselines

We compare Optimized PDR with the critic baselines in time and in learning against time.

G.1.1. Relative Training Cost

Each method keeps its starting models, objective, replay schedule and update budget. Table G.1 gives training time and preparation plus training.

TD is fastest everywhere. Including preparation, Optimized PDR takes 1.87× TD's time in KeyCorridor, 1.16× in LoCA, and 1.06× and 1.04× in DeepSea $N = 10$ and $N = 20$. It is faster than PER and DisCor in LoCA and DeepSea, and faster than 8× TD everywhere, using 23.2–40.3% of its time. These are means of within-seed ratios; Table G.2 (Appendix G.2) gives intervals.

Table G.1.: Cost of the five methods on matched RTX 3090 continuations. Entries are mean seconds over 20 seeds for KeyCorridor and LoCA, and ten per DeepSea size. Full budgets are 512, 4,096 and 8,192 actor updates, respectively. Training includes each method's sampling, critic, actor and target work, plus PDR refresh or DisCor auxiliary updates where used. 8× TD takes eight critic steps per actor update. Preparation adds input/model loading and method setup, including capture or compilation for PDR. Existing replay and reference inputs precede this clock; evaluation, validation, warm-up and output I/O are timed separately.

Setting	TD	PER	DisCor	8× TD	Opt. PDR
(a) Training time (s)					
KeyCorridor S3R3	3.66	4.43	5.41	17.33	6.71
LoCA AWAC	24.85	32.05	38.97	116.71	28.71
DeepSea $N = 10$	50.99	58.77	78.66	230.07	53.45
DeepSea $N = 20$	51.14	60.75	78.52	230.43	52.98
(b) Preparation + training (s)					
KeyCorridor S3R3	3.83	4.66	5.58	17.57	7.08
LoCA AWAC	25.03	32.23	39.16	116.89	28.95
DeepSea $N = 10$	51.16	58.95	78.82	230.24	54.01
DeepSea $N = 20$	51.32	60.93	78.69	230.60	53.53

G.1.2. Learning against Elapsed Time

Figure G.1 plots LoCA and DeepSea learning against elapsed time, up to the time TD finishes its budget. PDR's early gains remain. At the time TD needs for its early window, LoCA return is 1.184 for PDR and 0.038 for TD (6.39 s). In DeepSea, it is 0.710 against 0.587 at $N = 10$ (12.96 s) and 0.243 against 0.023 at $N = 20$ (12.93 s). PDR also beats PER, DisCor and 8× TD at these times. Later, TD catches up at $N = 10$ and overtakes PDR at $N = 20$. In KeyCorridor S3R3, PDR is behind at TD's early-window time: success is 51.1% against 65.4% for TD (Table G.3).

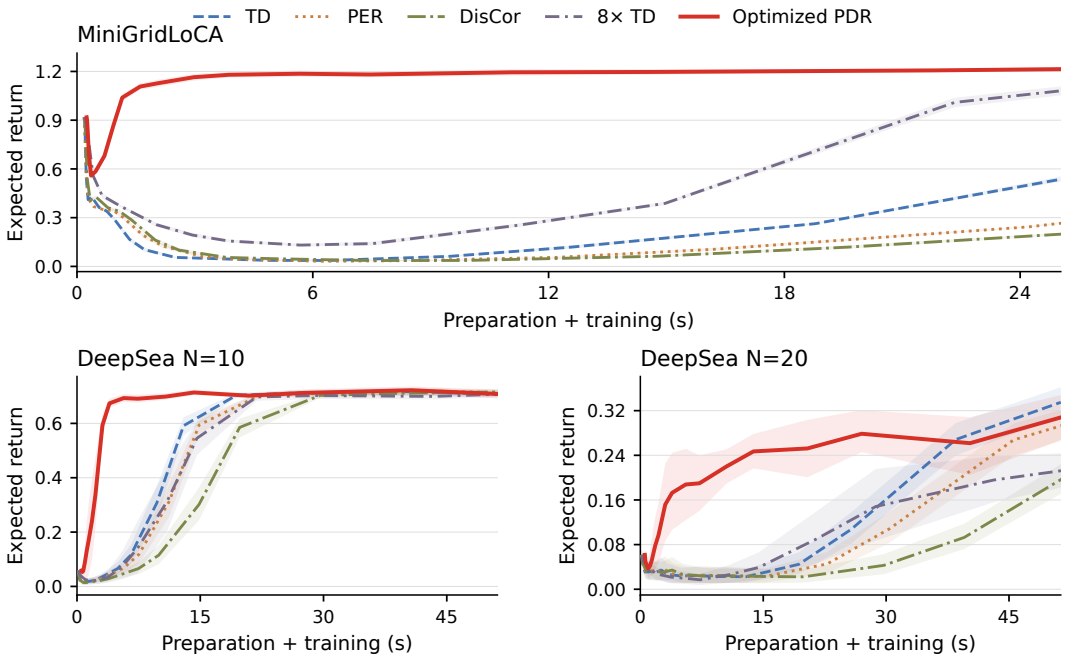


Figure G.1.: Learning performance against preparation-plus-training time. Lines join mean scores at mean measured times on the original evaluation grid; shading shows 95% seed-bootstrap intervals for the scores (10,000 resamples). The three panels show MiniGridLoCA and DeepSea $N = 10, 20$, with all original seeds included. Each panel displays the time through TD's full-budget endpoint. Longer runs continue beyond the displayed window; no curve is smoothed.

G.2. Baseline Retiming Protocol

This study repeats all five methods on KeyCorridor S3R3 (20 seeds, 512 updates), LoCA AWAC (20 seeds, 4,096 updates) and DeepSea $N = 10, 20$ (ten seeds each, 8,192 updates). All 300 runs keep their original models, objectives, replay and baseline settings. PDR uses the optimized implementation of Section A.5.

The five methods of a seed run one after another on one RTX 3090, in balanced order. GPU logs are sampled every two seconds, and we keep only runs with no competing GPU jobs. Four warm-up updates are followed by a full restore of all learner state.

GPU synchronization separates three intervals: batch and critic work, values given to the actor, and actor and target updates. Their sum is the training time. It includes all method-specific work: PER sampling, DisCor's error network, 8× TD's extra steps and PDR's query and solve. Preparation covers loading inputs, building the LoCA graph, and PDR compilation. Evaluation, checks, warm-up and saving are timed separately.

At four checkpoints, native and optimized PDR give values to the same actor and batch. All 240 value and gradient checks agree exactly.

Table G.2.: Optimized PDR preparation-plus-training time divided by each baseline's time. Values are means of within-seed ratios [95% paired-seed bootstrap CI, 10,000 draws, seed 20260919]. A ratio below one indicates less time for the same actor-update budget. Settings and seeds match Table G.1.

Setting	PDR/TD	PDR/PER	PDR/DisCor	PDR/8× TD
KeyCorridor S3R3	1.87 [1.77, 1.98]	1.53 [1.46, 1.60]	1.28 [1.22, 1.34]	0.40 [0.39, 0.42]
LoCA AWAC	1.16 [1.14, 1.17]	0.90 [0.88, 0.92]	0.74 [0.73, 0.75]	0.25 [0.24, 0.25]
DeepSea $N = 10$	1.06 [1.03, 1.08]	0.92 [0.90, 0.94]	0.69 [0.67, 0.70]	0.23 [0.23, 0.24]
DeepSea $N = 20$	1.04 [1.03, 1.05]	0.88 [0.86, 0.89]	0.68 [0.67, 0.69]	0.23 [0.23, 0.23]

The equal-time comparison uses TD's mean time to reach the end of its early window (update 128, 1,024 and 2,048 for KeyCorridor, LoCA and DeepSea). Each method's score at that time is interpolated between its checkpoints. Table G.3 reports the scores; paired intervals use the per-seed values.

Table G.3.: Mean task scores at a shared preparation-plus-training time. KeyCorridor reports success percentages; LoCA and DeepSea report their original expected-return metrics. Times are the mean TD early-window endpoints, with all original seeds retained. The lower panel gives paired PDR-minus-baseline differences and 95% bootstrap intervals (10,000 draws); KeyCorridor differences are in percentage points, and the others are in return units.

Setting	Time (s)	TD	PER	DisCor	8× TD	Opt. PDR
KeyCorridor S3R3	1.11	65.393	56.514	54.486	29.480	51.073
LoCA AWAC	6.39	0.038	0.033	0.041	0.135	1.184
DeepSea $N = 10$	12.96	0.587	0.447	0.230	0.437	0.710
DeepSea $N = 20$	12.93	0.023	0.023	0.024	0.033	0.243

Setting	PDR − TD	PDR − PER	PDR − DisCor	PDR − 8× TD
KeyCorridor S3R3	−14.3	−5.4	−3.4	+21.6
	[−25.7, −3.4]	[−15.6, 4.0]	[−13.0, 5.7]	[12.6, 30.1]
LoCA AWAC	+1.146	+1.151	+1.143	+1.049
	[1.138, 1.156]	[1.141, 1.160]	[1.133, 1.153]	[1.033, 1.064]
DeepSea $N = 10$	+0.122	+0.263	+0.480	+0.273
	[0.099, 0.147]	[0.213, 0.313]	[0.429, 0.526]	[0.180, 0.362]
DeepSea $N = 20$	+0.220	+0.220	+0.219	+0.210
	[0.200, 0.240]	[0.198, 0.243]	[0.195, 0.245]	[0.188, 0.233]

G.3. Native versus Optimized PDR

Native and optimized PDR start from the same models, reference, replay and objective. The 16 settings cover both reference types in KeyCorridor and LoCA, all three LoCA actors, and DeepSea $N = 10, 20, 30$, with 289 seed pairs (Table G.4). The two arms of a pair run one after another on one RTX 3090, in alternating order. *Training time* sums the timed updates; *preparation plus training* adds loading and compilation. Speedups are medians of paired ratios, with 95% intervals from 10,000 bootstrap samples. At four checkpoints per run, we also compare the two paths on the same actor and batch (2,312 checks), and we compare scores at every checkpoint (Appendices G.4 and G.5).

G.3.1. Benchmark Training Time

Optimization (Section A.5) reduces training time in all 289 pairs (Table G.4). The three main KeyCorridor tasks are 5.91–7.01× faster. For example, exact-reference S4R3 falls from 48.72 to 6.97 s over 512 updates, a speedup of 7.01× [6.83, 7.10].

LoCA is 1.61–1.85× faster, DeepSea 1.04–1.10×, and KeyCorridor S5R3 2.62×.

Table G.4.: Paired native–optimized PDR on RTX 3090, with matched starting models, reference and replay schedule. Wall times are seed means of synchronized training updates, excluding evaluation, validation, warm-up and serialization. Speedups are paired medians [95% bootstrap CI]. The final column adds input/model loading and optimized preparation. QAC: Expected-SARSA QAC.

Setting	Ref.	Training wall time (s)		Paired speedup	Prep. + train
		Native	Optimized	median [95% CI]	optimized (s)
KeyCorridor: AWAC, 512 updates, 20 pairs per row					
S3R2	Exact	27.31	**4.26**	6.36× [6.20, 6.56]	4.88
S3R3	Exact	28.00	**4.73**	5.97× [5.76, 6.06]	5.39
S4R3	Exact	48.72	**6.97**	7.01× [6.83, 7.10]	7.72
S5R3	Exact	121.44	**45.95**	2.62× [2.59, 2.68]	48.01
S3R2	Neural	27.53	**4.32**	6.40× [6.28, 6.48]	4.96
S3R3	Neural	27.68	**4.69**	5.91× [5.81, 5.96]	5.34
S4R3	Neural	47.97	**6.96**	6.88× [6.73, 7.10]	7.70
MiniGridLoCA: 4,096 updates, 20 pairs per row					
AWAC	Exact	27.60	**15.18**	1.82× [1.78, 1.84]	15.61
SAC	Exact	32.56	**20.24**	1.62× [1.60, 1.63]	20.69
QAC	Exact	27.49	**14.99**	1.84× [1.82, 1.87]	15.40
AWAC	Neural	27.57	**15.16**	1.82× [1.80, 1.85]	15.60
SAC	Neural	32.85	**20.36**	1.61× [1.59, 1.63]	20.82
QAC	Neural	27.12	**14.63**	1.85× [1.84, 1.86]	15.10
DeepSea: AWAC, 8,192 updates; 10, 10, 9 pairs					
$N = 10$	Exact	33.75	**32.41**	1.04× [1.03, 1.05]	33.07
$N = 20$	Exact	36.75	**34.00**	1.07× [1.06, 1.11]	34.68
$N = 30$	Exact	38.54	**35.14**	1.10× [1.06, 1.12]	35.83

G.3.2. Component Costs

Computing and delivering the correction takes 93.1–98.1% of native time in KeyCorridor, 49.3–59.6% in LoCA and 11.0–20.4% in DeepSea (Figure G.2). The savings are therefore largest where this share is largest.

KeyCorridor. The native loop allocates tensors and launches GPU work from Python. The optimized path prepares indices and buffers once and replays a captured CUDA graph over the horizon. Query, solve and delivery become 8.89–10.57× faster on the three main tasks and 2.70× on S5R3.

LoCA. The optimized path reuses successor indices and reference sums, vectorizes the source, and replaces the dense solve with a sparse direct solve of the same 256-state system. Refresh and delivery fall from about 16.2 to 3.9 s per run.

DeepSea. A compiled C++ backward pass replaces Python loops. Refresh becomes 1.43–1.83× faster, but it is only 11.0–20.4% of training time.

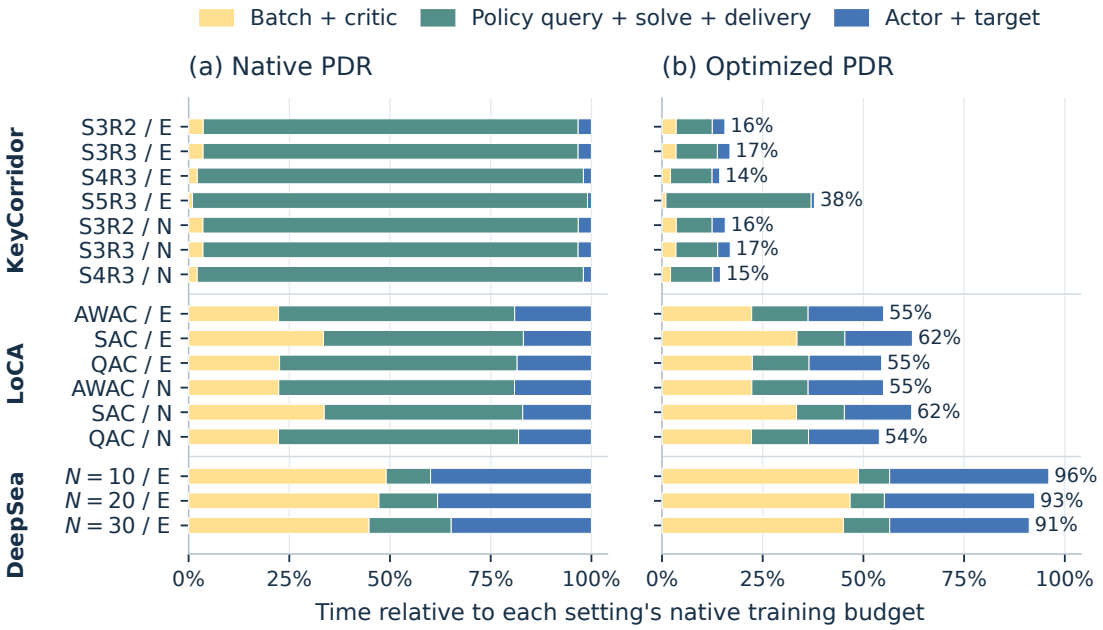


Figure G.2.: Paired benchmark component times (all seeds in Table G.4). Both arms normalize mean seed totals by that setting's mean native training time. E/N: exact/neural reference. Teal includes query, solve and actor-batch delivery; labels give the optimized total. Preparation and evaluation are excluded.

G.3.3. Actor-Signal Preservation

Both implementations give the actor almost identical values and gradients (Table G.5). Over 2,312 checks, the largest value difference is 1.14×10^{-13} and the largest relative gradient difference is 2.07×10^{-9}.

Table G.5.: Maximum implementation differences across four matched-actor checks per trajectory. Batches: 64 records for KeyCorridor/DeepSea, 128 for LoCA. Gradient error: relative L_2. Curve gaps cover all score checkpoints and seeds; LoCA includes all three backbones.

Benchmark	Ref.	Checks	Batch Q error	Gradient rel. L_2	Score-curve gap
KeyCorridor	Exact	640	0	0	0
KeyCorridor	Neural	480	0	0	0
MiniGridLoCA	Exact	480	1.14×10^{-13}	2.07×10^{-9}	2.86×10^{-8}
MiniGridLoCA	Neural	480	0	0	0.0323
DeepSea $N = 10$	Exact	80	0	0	0
DeepSea $N = 20$	Exact	80	0	0	0
DeepSea $N = 30$	Exact	72	0	0	0

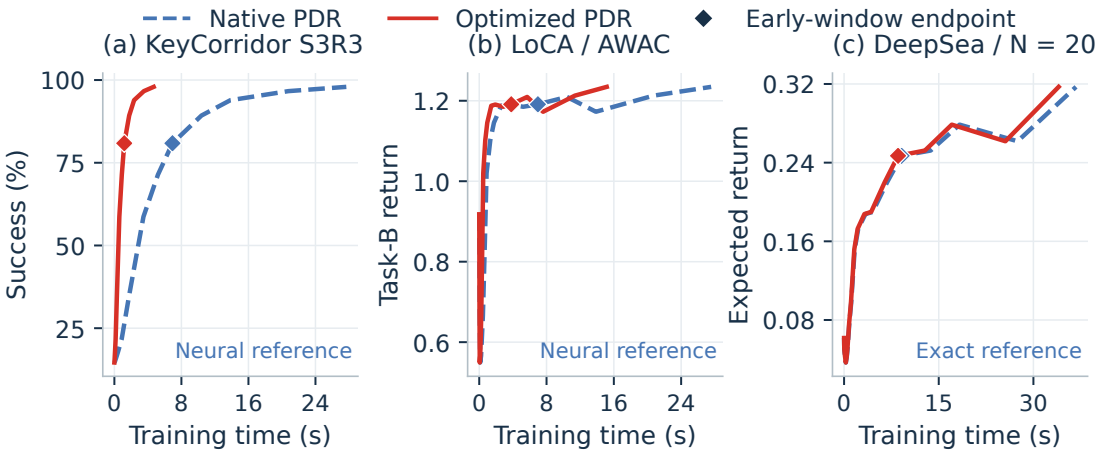


Figure G.3.: Mean score versus mean cumulative training time at recorded checkpoints. Neural KeyCorridor S3R3/LoCA AWAC: 20 pairs each; exact DeepSea $N = 20$: ten. Diamonds mark updates 128/1,024/2,048. Timer and budgets match Table G.4.

KeyCorridor and DeepSea score curves match exactly and finish sooner (Figure G.3). LoCA also matches, except for one neural-QAC run after the early window, which dips below native and then ends above it (Appendix G.5).

G.3.4. Persistence of the Savings

The savings remain after preparation is included (Figure G.4). KeyCorridor's 0.17–0.30 s preparation is reused over 512 updates, and the speedups stay at 5.32–6.39× on the three main tasks and 2.56× on S5R3. LoCA keeps 1.58–1.83× and DeepSea 1.03–1.07×. The size of the saving depends on how much time revaluation took in the first place.

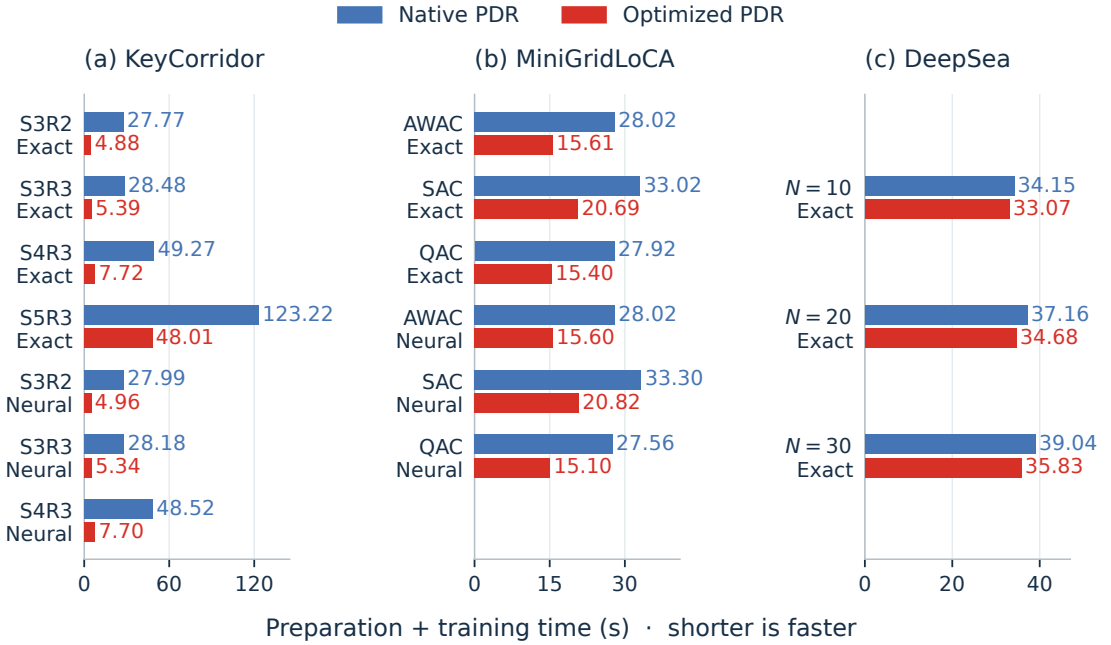


Figure G.4.: Mean preparation-plus-training time for native and optimized PDR. Bars and labels report seconds over the full budgets in Table G.4: 20 pairs per KeyCorridor/LoCA setting; 10, 10 and 9 pairs for DeepSea $N = 10, 20, 30$. Preparation adds input/model loading and optimized capture, indices or compilation; graph acquisition and reference fitting precede the stored inputs. All axes are linear, with separate ranges.

G.4. Paired Benchmark Continuations

This study uses all 289 seed pairs. KeyCorridor has 20 pairs for each of four exact-reference tasks (S3R2, S3R3, S4R3, S5R3) and three neural-reference tasks (S3R2, S3R3, S4R3). LoCA has 20 pairs for each reference and actor. DeepSea uses the AWAC runs, with ten pairs at $N = 10, 20$ and all nine available pairs at $N = 30$. Both arms restore the same checkpoints and receive the same inputs and batches. The budgets are 512, 4,096 and 8,192 updates.

Execution and timers. Eight workers use separate RTX 3090 cards, eight CPU cores each and one thread. The two arms of a pair run one after another on the same GPU, in alternating order. Each arm takes four warm-up updates and then restores its state. For update u, the recorded time is

$$t_u = t_{\text{batch+critic},u} + t_{\text{query+solve+delivery},u} + t_{\text{actor+target},u},$$

and training time is $\sum_u t_u$. Evaluation, signal checks, warm-up and saving are timed separately. LoCA keeps its residual check in both arms. Preparation plus training adds loading, and charges compilation and index preparation once, to the optimized arm only. Building the graph and the reference comes before these inputs.

Section G.3.2 describes the implementation changes used in this comparison.

Actor objectives and signal checks. Each pair keeps its original actor objective: LoCA AWAC detaches the whole weight, while KeyCorridor and DeepSea keep the gradient through the policy average.

At updates $0, U/4, U/2, U$, both paths read the same actor and batch (64 records for KeyCorridor and DeepSea, 128 for LoCA). We compare gradients before the update, using the relative error $\|g_{\text{native}} - g_{\text{opt}}\|_2/(\|g_{\text{native}}\|_2 + 10^{-12})$. All 2,312 checks pass. LoCA and DeepSea also check the full corrected table: the largest LoCA difference is 4.71×10^{-14}, and DeepSea agrees exactly.

G.5. Benchmark Learning Curves

Table G.6 reports every setting and seed. Early means cover updates 0–128, 0–1,024 and 0–2,048 for KeyCorridor, LoCA and DeepSea. Intervals use 10,000 paired seed resamples.

All KeyCorridor and DeepSea scores match exactly. In LoCA, the largest difference is 2.86×10^{-8}, except for neural-QAC seed 1910484143. There, optimized minus native return is -0.01577 at update 2,048 and $+0.03235$ at update 4,096 (final returns 1.78586 and 1.75352; Figure G.5). Its early means differ by only 3.74×10^{-10}. Over all 20 neural-QAC pairs, the mean final difference is 0.00162 (95% CI $[0, 0.00485]$). Both runs pass the matched-actor checks, so the records do not explain the later difference.

G.6. Training Timers and Benchmark Costs

Table G.7 collects the costs recorded by the original learners. These timers differ from the common update timer of Appendix G.4.

Table G.6.: Task scores in the paired native–optimized PDR continuations. Early and final scores are seed means, shown to four decimal places. KeyCorridor uses success percentages; the other families use returns. Seed counts and update budgets match Table G.4.

Setting	Ref.	Early mean		Final score	
		Native	Optimized	Native	Optimized
KeyCorridor: success (%)					
S3R2	Exact	21.3408	21.3408	61.7355	61.7355
S3R3	Exact	51.6933	51.6933	98.2141	98.2141
S4R3	Exact	49.6563	49.6563	79.6796	79.6796
S5R3	Exact	73.9988	73.9988	85.9011	85.9011
S3R2	Neural	21.0105	21.0105	61.1437	61.1437
S3R3	Neural	52.7137	52.7137	97.9374	97.9374
S4R3	Neural	49.5797	49.5797	77.3424	77.3424
MiniGridLoCA: discounted Task-B return					
AWAC	Exact	1.1122	1.1122	1.2235	1.2235
SAC	Exact	1.7080	1.7080	1.8036	1.8036
QAC	Exact	1.6796	1.6796	1.7925	1.7925
AWAC	Neural	1.1165	1.1165	1.2345	1.2345
SAC	Neural	1.6825	1.6825	1.8045	1.8045
QAC	Neural	1.6628	1.6628	1.6875	1.6892
DeepSea: expected return					
$N = 10$	Exact	0.6123	0.6123	0.7046	0.7046
$N = 20$	Exact	0.1812	0.1812	0.3171	0.3171
$N = 30$	Exact	0.0029	0.0029	0.0631	0.0631

Part (c) gives SecureCourier costs, averaged over each seed's first 1,024 cycles and then over the 50 seeds. Its timer covers collection, network updates and diagnostics; logging, scoring and checkpointing are outside it (Appendix A.6). Part (a) gives KeyCorridor costs, with scoring and diagnostics subtracted and preparation excluded. Part (b) uses LoCA's run timer, which includes scoring and diagnostics. Here exact-reference PDR also checks its values against direct evaluation at every update, while the other arms check only at diagnostic checkpoints.

G.7. Recorded Time Curves and Refresh Accounting

Figure G.6 shows the seed endpoints behind Table G.7(a).

Figure G.7 plots learning against recorded time. TD is above PDR over much of the shared interval, while PDR reaches its higher final success later.

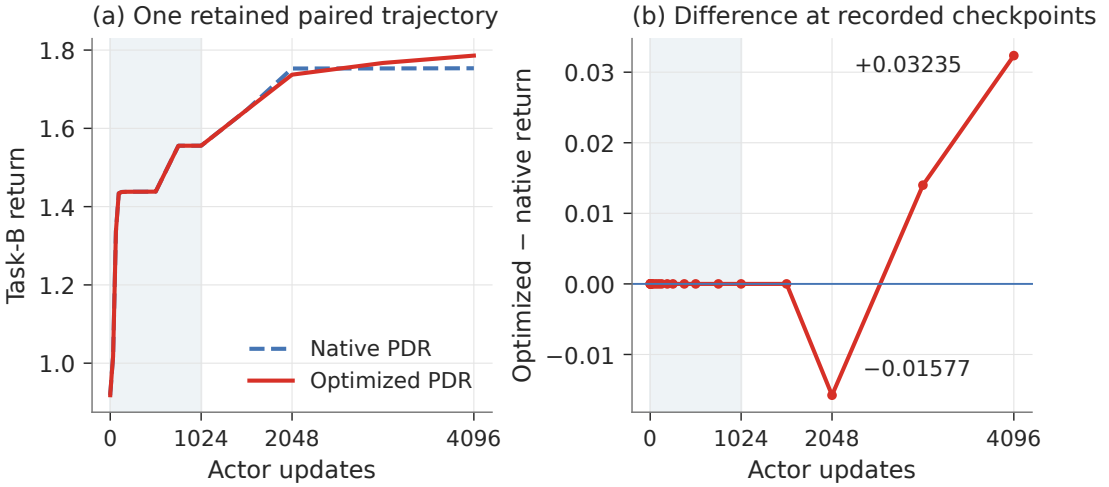


Figure G.5.: The retained neural-reference LoCA QAC pair, seed 1910484143. (a) Task-B return at the original checkpoints. (b) Optimized minus native return. Shading marks the early window through update 1,024. All intermediate rises and falls are retained.

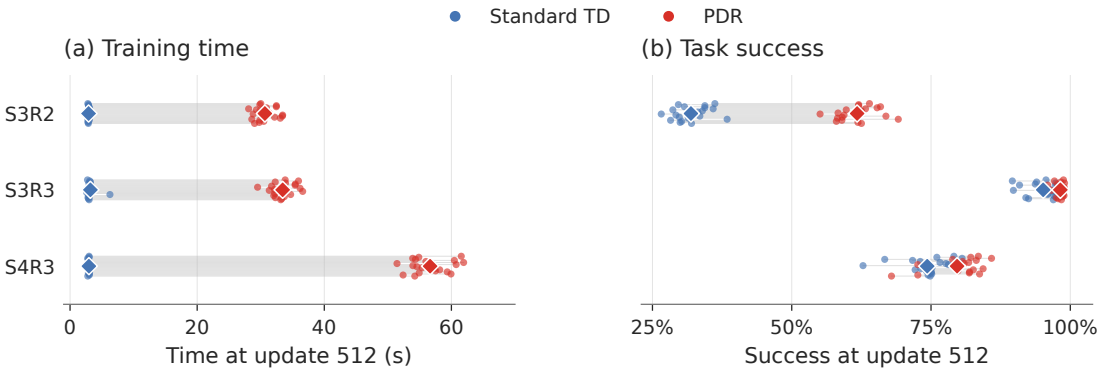


Figure G.6.: Standard TD and PDR on exact-reference KeyCorridor after 512 updates. (a) Recorded continuation time minus scoring and signal diagnostics. (b) Task-success probability. Small points show all 20 paired seeds per task; horizontal segments connect pairs. Diamonds mark means; numerical means are reported in Table G.7(a).

Table G.7.: Recorded costs in the learning runs. Times and success average seed means; ratios divide mean times. KeyCorridor excludes scoring and signal diagnostics; LoCA includes them. SecureCourier averages cycles within seeds; lookup is included in actor time.

(a) KeyCorridor: exact reference, 512 updates, 20 paired seeds/task

Task	Time (s)		Time ratio	Success (%)	
	Standard TD	PDR	PDR/TD	Standard TD	PDR
S3R2	2.913	30.654	10.52×	31.91	61.74
S3R3	3.202	33.449	10.44×	95.13	98.21
S4R3	2.963	56.649	19.12×	74.32	79.68

(b) LoCA: 4,096 updates, 20 seeds/cell; run time (s)

Reference	Backbone	Standard TD	Static reference	PDR
Exact	AWAC	37.86	35.89	64.04
Exact	SAC	52.05	47.82	75.30
Exact	QAC	37.29	36.35	63.07
Neural	AWAC	31.53	30.58	52.02
Neural	SAC	45.98	46.67	65.95
Neural	QAC	30.51	30.40	50.97

(c) SecureCourier learner: 1,024 active cycles/seed, 50 seeds

Measured component	Mean time (ms/cycle)
Current-policy query	1.116
Differential solve	1.495
Query + solve	2.611
Batch lookup (inside actor update)	6.953
Actor update (including lookup)	15.051

G.8. Independent SecureCourier Implementation Study

This study times only the policy query and the solve, on matched SecureCourier checkpoints, where the actor allows extra input sharing.

We use Python 3.10.13, NumPy 2.2.6 and PyTorch 2.5.1 on CPU with one thread. After warm-up, each path is timed three times on 30 seed–checkpoint pairs. Both use the same actor, learned reference and 14,576-pair graph. Query and solve are timed separately and summed; preparation, batch delivery and actor updates are excluded.

The one-, two- and five-action groups contain 9,616, 1,200 and 512 states, so $1{,}200 \times 2 + 512 \times 5 = 4{,}960$ multi-action rows map to 74 unique token rows (Table G.8). The native

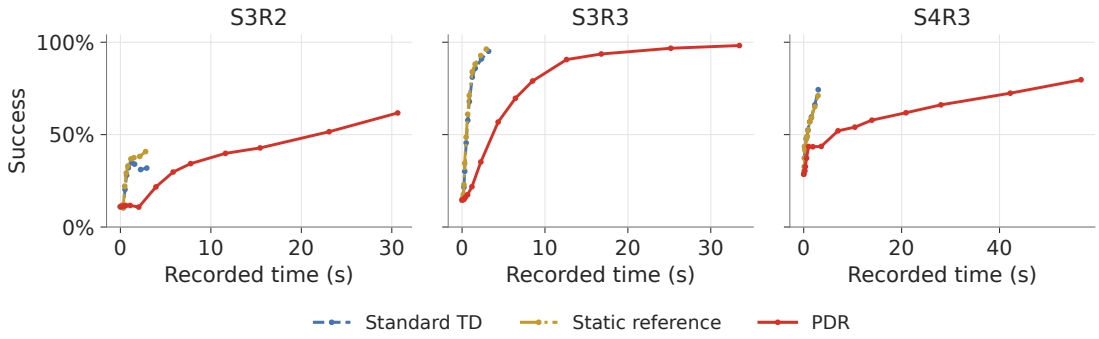


Figure G.7.: Exact-reference KeyCorridor: mean time and success at each recorded checkpoint, over all 20 seeds. Lines end at update 512 per arm.

solver builds 22,096 operator entries; the backward sweep computes 13,728 corrections over 40 layers.

The 3.040 s compile time includes parsing the graph, indexing, tokenizing, grouping actions, loading the reference and serializing arrays. Training the reference and generating the graph are not included. Equation (G.1) charges compilation once.

On a stored 64-record batch (update 2,048, seed 283212493), the two paths give identical values, advantages, weights and gradients for the same policy. With the factorized policy, the largest gradient difference is 1.49×10^{-8} (cosine 0.999999999999998).

The native query encodes all 11,328 observations and 14,576 actions. The optimized query skips the 9,616 single-action states and encodes only the 74 unique action inputs; the observation term cancels in the softmax (Equation (A.6)). The optimized solve runs backwards over the fixed 40-layer DAG.

Median query time falls from 1,710.55 to 0.798 ms and solve time from 39.28 to 1.177 ms, for totals of 1,749.87 and 1.975 ms. The median paired speedup is 884.72× (Figure G.8) (10th–90th percentiles 805.93–916.19×). Most of the saving comes from the query. Table G.9 compares the values and gradients of the two implementations.

Repeated-refresh accounting. With the compile time and the median refresh time,

$$\overline{T}(K) = \frac{T_{\text{compile}}}{K} + T_{\text{refresh}}. \tag{G.1}$$

With $T_{\text{compile}} = 3.040\,\text{s}$, the average is 25.73 ms for $K = 128$ refreshes and 4.94 ms for $K = 1{,}024$ (5.063 s in total; Figure G.9a).

Table G.8.: Paired SecureCourier PDR implementations on CPU, one thread: ten seeds, three checkpoints each, three repeats (90 timings). (a) Code/input counts. (b) Wall-clock query plus solve; the cumulative row sums the 90 recorded intervals. Preparation, delivery and actor optimization are excluded. Percentiles describe timing variation. (c) One recorded compilation; repeated-use costs follow Equation (G.1) using median refresh time.

Quantity	Native PDR	Optimized PDR
(a) Computation per refresh		
Observation rows encoded	11,328	0
Action rows encoded	14,576	74
Operator entries assembled	22,096	0
Differential evaluation	Sparse solve	40-layer reverse sweep
(b) Measured wall-clock time		
Policy query, median (ms)	1,710.55	0.798
Differential solve, median (ms)	39.28	1.177
Refresh, median (ms)	**1,749.87**	**1.975**
Refresh, mean (ms)	1,760.29	2.013
Refresh, 10th–90th percentiles (ms)	1,721.14–1,804.98	1.941–2.202
Sum of 90 timed refreshes (s)	158.426	0.181
Median paired speedup		**884.72×**
10th–90th percentiles		805.93–916.19×
(c) Optimized preparation and reuse		
Recorded compilation (s)		3.040
Accounted mean, $K = 128$ (ms/refresh)		25.73
Accounted mean, $K = 1{,}024$ (ms/refresh)		**4.94**
Accounted total, $K = 1{,}024$ (s)		5.063

Table G.9.: Native–optimized PDR agreement on SecureCourier. Graph-value errors are maxima over 90 refresh records (30 seed–checkpoint pairs): same-policy holds probabilities fixed; complete-path includes each implementation's query. Delivered values and gradients compare complete paths on one stored 64-record AWAC batch at one checkpoint, with matched actor parameters and detached weights.

Comparison	Difference or agreement
Differential, same-policy maximum absolute difference	4.44×10^{-16}
Corrected Q, complete-path maximum absolute difference	7.18×10^{-8}
Delivered Q, checked-batch maximum absolute difference	2.98×10^{-8}
AWAC gradient, maximum absolute difference	1.49×10^{-8}
AWAC gradient cosine	0.999999999999998

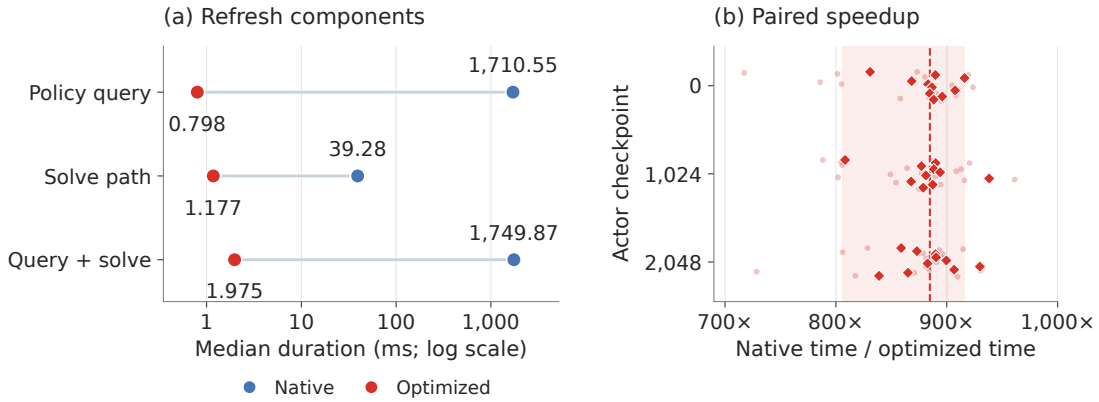


Figure G.8.: Paired SecureCourier timings for current-policy query and differential solve. Native and optimized PDR use matched actor checkpoints, a fixed learned reference and the same 14,576-pair graph. (a) Medians across 90 timings (10 seeds × 3 checkpoints × 3 repeats), on a log axis. Each repetition's total is query plus solve; preparation, batch delivery and actor optimization are excluded. (b) Points: paired time ratios; diamonds: three-repeat medians for each seed–checkpoint pair. Line and band: overall median and 10th–90th percentiles.

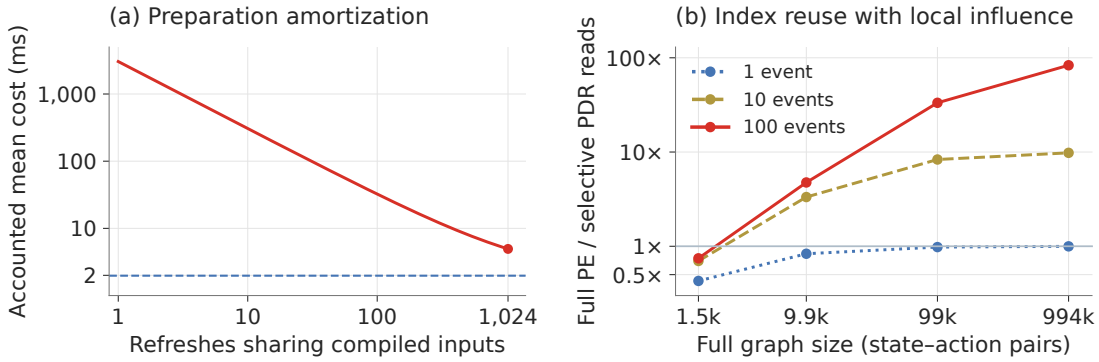


Figure G.9.: (a) SecureCourier accounting curve (Equation (G.1)); dashed: median refresh alone. At 1,024 refreshes, the accounted mean cost is 4.94 ms. (b) Independent sparse-change graphs: full-PE/selective-PDR transition reads versus full graph size, with 994 affected state–action pairs fixed. Index preparation is amortized over 1, 10 or 100 events; changed support is supplied. Counts coincide across ten seeds. Ratios above one favour selective PDR.

G.9. Locality, Source Discovery and Index Reuse

Here the changed region stays fixed in each of ten seeds while unchanged components are added to the graph. The five-entry source and its 994-pair influence region stay the same. Full revaluation reads every row once. Selective PDR, given the changed entries and a predecessor index, uses 993 discovery reads, five source reads and 994 solve reads: 1,992 per event. A matched local evaluator uses 1,987.

Building the index needs a full scan. If K events reuse it, PDR costs

$$1{,}992 + \frac{|\mathcal{X}|}{K}$$

reads per event. At $|\mathcal{X}| = 994{,}001$, full evaluation needs 499.00× as many reads without the index cost, and 0.998× as many if one event pays for the whole index (Figure G.9b).

Finding the source automatically from two policy snapshots costs more. That scan grows with the whole graph, even though propagation stays local (Table G.10, which counts all primitive reads).

Table G.10.: PDR source discovery from dense policy snapshots. Means over ten seeds; counts coincide across seeds. The predecessor-index build is accounted separately.

Graph pairs	Source reads	Propagation reads	Total reads
1,495	3,005	3,979	6,984
9,941	19,897	3,979	23,876
99,401	198,817	3,979	202,796
994,001	1,988,017	3,979	1,991,996

The local evaluator gets the same values with slightly fewer reads. In the controls, a dense change touches 1,998 of 2,000 pairs and needs 5,992 reads; a two-entry source with a 1,998-pair region needs 3,996. Both cost more than the 2,000-row full pass. A zero policy change needs no reads. Savings therefore need both a small source and a small influence region.

G.10. Additional Graph Computations and Scaling

The backend study runs on one AMD EPYC 7343 thread with Python 3.10.13, NumPy 2.2.6, SciPy 1.15.3 and OpenBLAS 0.3.29, in float64 with $\gamma = 0.97$. It uses seven saved SecureCourier systems of 14,576 pairs (five checkpoints of one seed and two from other seeds), with two warm-ups and seven timed runs each.

With the policy given, the median source takes 251.60 ms, the operator 170.21 ms and the sparse solve 8.07 ms. Reusing compiled arrays cuts the full numerical path from 430.36 to 2.902 ms with factorized Bellman evaluation (Figure G.10).

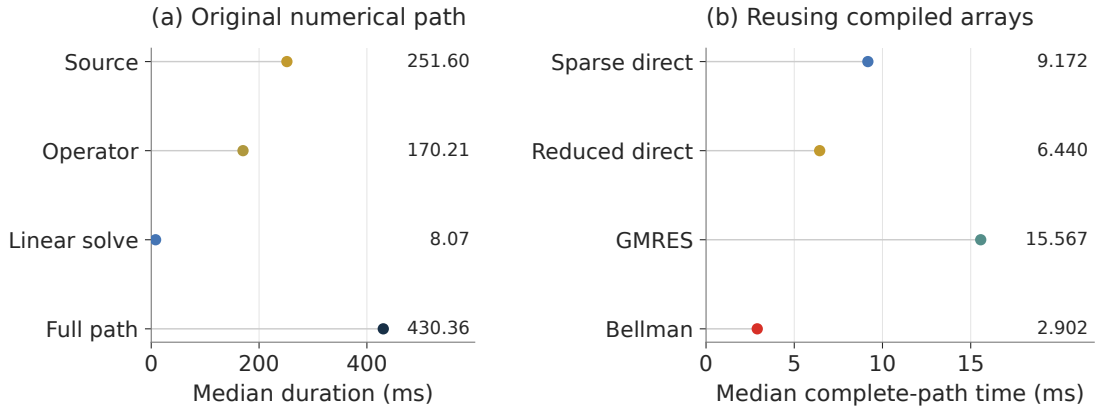


Figure G.10.: Additional numerical-path timings with policy probabilities supplied: seven frozen systems, seven repetitions each. (a) Original component and total medians. (b) Complete-path medians using compiled arrays; GMRES and Bellman use factored operators. Axes differ. Components and totals are summarized separately.

These timers cover source and operator construction, assembly, solve and projection; policy probabilities are given. Appendix G.8 times the neural query separately.

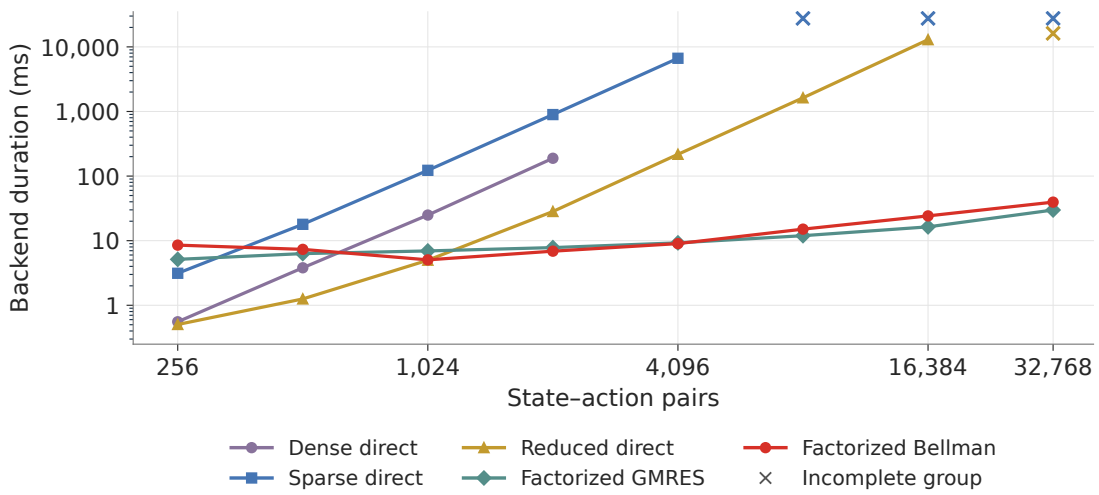


Figure G.11.: Independent stochastic scaling systems with policy probabilities supplied. Lines join median backend durations over seven completed repetitions at each size. Crosses at the top identify incomplete measurement groups and have no per-solve duration. Table G.11 defines the timed work.

The scaling study uses $N = 256, 512, \ldots, 32{,}768$ pairs, four actions per state, three random successors per row and a terminal probability of 0.05. Dense direct solves are run up to $N = 2{,}048$. At $N = 256$, reduced direct takes 0.504 ms against 5.138 ms for factorized GMRES; at $N = 32{,}768$, GMRES takes 29.83 ms against 39.50 ms for Bellman iteration. Table G.11 defines the timers of Figure G.11.

Table G.11.: Recorded scaling-backend timer scopes, with policy probabilities supplied. Graph generation, loading and correctness evaluation are outside these timers.

Backend	Timed work
Dense / sparse direct	Numerical solve; system-matrix construction and format conversion precede the timer.
State-reduced direct	Reduced-system assembly, direct solve and pair-value reconstruction.
Factorized GMRES	Iterative solve using the factored operator and configured tolerance.
Factorized Bellman	Fixed-point iterations and their stopping checks using the factored operator.

GMRES uses relative tolerance 10^{-13}, restart 200 and at most 2,000 iterations; Bellman iteration uses tolerance 10^{-12} and at most 5,000 iterations. On the 42 real systems, the largest error is 3.57×10^{-13} and the largest residual 2.23×10^{-13}. Each worker has a 120 s limit, which sparse direct reaches at $N \geq 8{,}192$ and state-reduced direct at $N = 32{,}768$ (marked in Figure G.11).

Bibliography

Alver, S.; Rahimi-Kalahroudi, A.; and Precup, D., 2025. Partial models for building adaptive model-based reinforcement learning agents. In *Proceedings of the 3rd Conference on Lifelong Learning Agents*, vol. 274 of *Proceedings of Machine Learning Research*, 959–977. https://proceedings.mlr.press/v274/alver25a.html. [Cited on page 11.]

Anthony, T.; Tian, Z.; and Barber, D., 2017. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, vol. 30, 5360–5370. [Cited on page 10.]

Barreto, A.; Dabney, W.; Munos, R.; Hunt, J. J.; Schaul, T.; van Hasselt, H.; and Silver, D., 2017. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems*, vol. 30, 4055–4065. https://proceedings.neurips.cc/paper_files/paper/2017/hash/350db081a661525235354dd3e19b8c05-Abstract.html. [Cited on page 11.]

Barto, A. G.; Sutton, R. S.; and Anderson, C. W., 1983. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13, 5 (1983), 834–846. doi:10.1109/TSMC.1983.6313077. https://ieeexplore.ieee.org/document/6313077/. [Cited on page 5.]

Bellemare, M. G.; Ostrovski, G.; Guez, A.; Thomas, P. S.; and Munos, R., 2016. Increasing the action gap: New operators for reinforcement learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 1476–1483. https://ojs.aaai.org/index.php/AAAI/article/view/10303. [Cited on page 10.]

Bellman, R., 1957. A Markovian decision process. *Journal of Mathematics and Mechanics*, 6, 4 (1957), 679–684. doi:10.1512/iumj.1957.6.56038. https://iumj.org/article/1116/. [Cited on page 4.]

Bradtke, S. J. and Barto, A. G., 1996. Linear least-squares algorithms for temporal difference learning. *Machine Learning*, 22 (1996), 33–57. doi:10.1007/BF00114723. https://link.springer.com/article/10.1007/BF00114723. [Cited on page 9.]

Chevalier-Boisvert, M.; Bahdanau, D.; Lahlou, S.; Willems, L.; Saharia, C.; Nguyen, T. H.; and Bengio, Y., 2019. BabyAI: A platform to study the sample efficiency of grounded language learning. In *International Conference on Learning Representations*. https://arxiv.org/abs/1810.08272. [Cited on pages 29 and 68.]

Dayan, P., 1993. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5, 4 (1993), 613–624. doi:10.1162/neco.1993.5.4.613. [Cited on page 10.]

Espeholt, L.; Soyer, H.; Munos, R.; Simonyan, K.; Mnih, V.; Ward, T.; Doron, Y.; Firoiu, V.; Harley, T.; Dunning, I.; Legg, S.; and Kavukcuoglu, K., 2018. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning*, vol. 80 of *Proceedings of Machine Learning Research*, 1407–1416. https://proceedings.mlr.press/v80/espeholt18a.html. [Cited on page 9.]

Farahmand, A.-M.; Barreto, A.; and Nikovski, D., 2017. Value-aware loss function for model-based reinforcement learning. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, vol. 54 of *Proceedings of Machine Learning Research*, 1486–1494. PMLR. https://proceedings.mlr.press/v54/farahmand17a.html. [Cited on page 10.]

Farama Foundation, 2026. Key Corridor: MiniGrid documentation. https://minigrid.farama.org/environments/babyai/KeyCorridor/. Accessed September 11, 2026. [Cited on page 68.]

Fujimoto, S.; Meger, D.; Precup, D.; Nachum, O.; and Gu, S. S., 2022. Why should I trust you, Bellman? The Bellman error is a poor replacement for value error. In *Proceedings of the 39th International Conference on Machine Learning*, vol. 162 of *Proceedings of Machine Learning Research*, 6918–6943. https://proceedings.mlr.press/v162/fujimoto22a.html. [Cited on pages 10 and 55.]

Fujimoto, S.; van Hoof, H.; and Meger, D., 2018. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning*, vol. 80 of *Proceedings of Machine Learning Research*, 1587–1596. https://proceedings.mlr.press/v80/fujimoto18a.html. [Cited on page 9.]

Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S., 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, vol. 80 of *Proceedings of Machine Learning Research*, 1861–1870. https://proceedings.mlr.press/v80/haarnoja18b.html. [Cited on pages 6, 7, and 69.]

Ilyas, A.; Engstrom, L.; Santurkar, S.; Tsipras, D.; Janoos, F.; Rudolph, L.; and Madry, A., 2020. A closer look at deep policy gradients. In *International Conference on Learning Representations*. https://openreview.net/forum?id=ryxdEkHtPS. [Cited on page 10.]

Kakade, S. and Langford, J., 2002. Approximately optimal approximate reinforcement learning. In *Proceedings of the 19th International Conference on Machine Learning*, 267–274. [Cited on pages 9 and 53.]

Konda, V. R. and Tsitsiklis, J. N., 1999. Actor-critic algorithms. In *Advances in Neural Information Processing Systems*, vol. 12, 1008–1014. https://papers.neurips.cc/paper_files/paper/1999/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html. [Cited on pages 5, 10, 55, and 57.]

Kostrikov, I.; Nair, A.; and Levine, S., 2022. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*. https://arxiv.org/abs/2110.06169. [Cited on pages 9 and 30.]

Kumar, A.; Gupta, A.; and Levine, S., 2020a. DisCor: Corrective feedback in reinforcement learning via distribution correction. In *Advances in Neural Information Processing Systems*, vol. 33, 18560–18572. https://proceedings.neurips.cc/paper/2020/file/d7f426ccbc6db7e235c57958c21c5dfa-Paper.pdf. [Cited on pages 8, 30, 70, and 81.]

Kumar, A.; Zhou, A.; Tucker, G.; and Levine, S., 2020b. Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, vol. 33, 1179–1191. https://proceedings.neurips.cc/paper/2020/hash/0d2b2061826a5df3221116a5085a6052-Abstract.html. [Cited on pages 9 and 30.]

Lagoudakis, M. G. and Parr, R., 2003. Least-squares policy iteration. *Journal of Machine Learning Research*, 4 (2003), 1107–1149. https://jmlr.org/papers/v4/lagoudakis03a.html. [Cited on page 9.]

Lambert, N.; Amos, B.; Yadan, O.; and Calandra, R., 2020. Objective mismatch in model-based reinforcement learning. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, vol. 120 of *Proceedings of Machine Learning Research*, 761–770. https://proceedings.mlr.press/v120/lambert20a.html. [Cited on pages 10 and 55.]

Lin, L.-J., 1992. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8 (1992), 293–321. doi:10.1007/BF00992699. https://link.springer.com/article/10.1007/BF00992699. [Cited on page 6.]

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D., 2015. Human-level control through deep reinforcement learning. *Nature*, 518 (2015), 529–533. doi:10.1038/nature14236. https://www.nature.com/articles/nature14236. [Cited on page 6.]

Moore, A. W. and Atkeson, C. G., 1993. Prioritized sweeping: Reinforcement learning with less data and less time. *Machine Learning*, 13 (1993), 103–130. doi:10.1007/BF00993104. https://link.springer.com/article/10.1007/BF00993104. [Cited on pages 10, 12, and 30.]

Munos, R.; Stepleton, T.; Harutyunyan, A.; and Bellemare, M. G., 2016. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems*, vol. 29, 1054–1062. https://proceedings.neurips.cc/paper_files/paper/2016/file/c3992e9a68c5ae12bd18488bc579b30d-Paper.pdf. [Cited on pages 9 and 12.]

Nair, A.; Gupta, A.; Dalal, M.; and Levine, S., 2020. AWAC: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, (2020). https://arxiv.org/abs/2006.09359. [Cited on pages 6, 9, 15, and 69.]

Osband, I.; Doron, Y.; Hessel, M.; Aslanides, J.; Sezener, E.; Saraiva, A.; McKinney, K.; Lattimore, T.; Szepesvári, C.; Singh, S.; Van Roy, B.; Sutton, R.; Silver, D.; and van Hasselt, H., 2020. Behaviour suite for reinforcement learning. In *International Conference on Learning Representations*. https://openreview.net/forum?id=rygf-kSYwH. [Cited on pages 29 and 69.]

Pan, H.-R.; Gürtler, N.; Neitz, A.; and Schölkopf, B., 2022. Direct advantage estimation. In *Advances in Neural Information Processing Systems*, vol. 35, 11869–11880. https://proceedings.neurips.cc/paper_files/paper/2022/hash/4d893f766ab60e5337659b9e71883af4-Abstract-Conference.html. [Cited on pages 9 and 11.]

Pan, H.-R. and Schölkopf, B., 2024. Skill or luck? Return decomposition via advantage functions. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/file/277628cff838927d869cd1f671328ce0-Paper-Conference.pdf. [Cited on page 9.]

Peng, X. B.; Kumar, A.; Zhang, G.; and Levine, S., 2019. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, (2019). https://arxiv.org/abs/1910.00177v3. [Cited on page 9.]

Rahimi-Kalahroudi, A.; Rajendran, J.; Momennejad, I.; van Seijen, H.; and Chandar, S., 2023. Replay buffer with local forgetting for adapting to local environment changes in deep model-based reinforcement learning. In *Proceedings of the 2nd Conference on Lifelong Learning Agents*, vol. 232 of *Proceedings of Machine Learning Research*, 21–42. https://proceedings.mlr.press/v232/rahimi-kalahroudi23a.html. [Cited on pages 11, 29, and 68.]

Russek, E. M.; Momennejad, I.; Botvinick, M. M.; Gershman, S. J.; and Daw, N. D., 2017. Predictive representations can link model-based reinforcement learning to model-free mechanisms. *PLOS Computational Biology*, 13, 9 (2017), e1005768. doi: 10.1371/journal.pcbi.1005768. https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1005768. [Cited on pages 7 and 10.]

Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D., 2016. Prioritized experience replay. In *International Conference on Learning Representations*. https://arxiv.org/abs/1511.05952. [Cited on pages 8, 30, 70, and 81.]

Schrittwieser, J.; Hubert, T.; Mandhane, A.; Barekatain, M.; Antonoglou, I.; and Silver, D., 2021. Online and offline reinforcement learning by planning with a learned model. In *Advances in Neural Information Processing Systems*, vol. 34, 27580–27591. https://papers.nips.cc/paper_files/paper/2021/hash/e8258e5140317ff36c7f8225a3bf9590-Abstract.html. [Cited on page 10.]

Schulman, J.; Moritz, P.; Levine, S.; Jordan, M. I.; and Abbeel, P., 2016. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*. https://arxiv.org/abs/1506.02438. [Cited on pages 9 and 54.]

Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; Lillicrap, T.; Simonyan, K.; and Hassabis, D., 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362, 6419 (2018), 1140–1144. doi:10.1126/science.aar6404. [Cited on page 10.]

Sutton, R. S., 1988. Learning to predict by the methods of temporal differences. *Machine Learning*, 3 (1988), 9–44. doi:10.1007/BF00115009. https://link.springer.com/article/10.1007/BF00115009. [Cited on page 5.]

Sutton, R. S., 1991. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2, 4 (1991), 160–163. doi:10.1145/122344.122377. https://doi.org/10.1145/122344.122377. [Cited on pages 6 and 11.]

Sutton, R. S. and Barto, A. G., 2018. *Reinforcement Learning: An Introduction*. MIT Press, 2 edn. ISBN 9780262039246. https://mitpress.mit.edu/9780262039246/reinforcement-learning/. [Cited on pages 1, 4, 12, and 57.]

Sutton, R. S.; McAllester, D.; Singh, S.; and Mansour, Y., 1999a. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, vol. 12, 1057–1063. https://papers.nips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html. [Cited on pages 5, 10, and 55.]

Sutton, R. S.; Precup, D.; and Singh, S., 1999b. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112, 1–2 (1999), 181–211. doi:10.1016/S0004-3702(99)00052-1. [Cited on page 57.]

Tang, Y.; Valko, M.; and Munos, R., 2020. Taylor expansion policy optimization. In *Proceedings of the 37th International Conference on Machine Learning*, vol. 119 of *Proceedings of Machine Learning Research*, 9397–9406. https://proceedings.mlr.press/v119/tang20d.html. [Cited on page 9.]

van Seijen, H.; Nekoei, H.; Racah, E.; and Chandar, S., 2020. The LoCA regret: A consistent metric to evaluate model-based behavior in reinforcement learning. In *Advances in Neural Information Processing Systems*, vol. 33, 6562–6572. https://proceedings.neurips.cc/paper/2020/hash/48db71587df6c7c442e5b76cc723169a-Abstract.html. [Cited on pages 11, 29, and 68.]

van Seijen, H. and Sutton, R. S., 2013. Planning by prioritized sweeping with small backups. In *Proceedings of the 30th International Conference on Machine Learning*, vol. 28(3) of *Proceedings of Machine Learning Research*, 361–369. https://proceedings.mlr.press/v28/vanseijen13.html. [Cited on page 10.]

van Seijen, H. and Sutton, R. S., 2015. A deeper look at planning as learning from replay. In *Proceedings of the 32nd International Conference on Machine Learning*, vol. 37 of *Proceedings of Machine Learning Research*, 2314–2322. https://proceedings.mlr.press/v37/vanseijen15.html. [Cited on page 10.]

Wan, Y.; Rahimi-Kalahroudi, A.; Rajendran, J.; Momennejad, I.; Chandar, S.; and van Seijen, H., 2022. Towards evaluating adaptivity of model-based reinforcement learning methods. In *Proceedings of the 39th International Conference on Machine Learning*, vol. 162 of *Proceedings of Machine Learning Research*, 22536–22561. https://proceedings.mlr.press/v162/wan22d.html. [Cited on page 11.]

Wang, Z.; Schaul, T.; Hessel, M.; van Hasselt, H.; Lanctot, M.; and de Freitas, N., 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, vol. 48 of *Proceedings of Machine Learning Research*, 1995–2003. PMLR. https://proceedings.mlr.press/v48/wangf16.html. [Cited on page 9.]

Zhao, G.; Xie, Z.; Zheng, B.; Lu, J.; Gong, E.; and Chen, Z., 2026. BRACE: Anchored Bellman-residual correction for stale critics in asynchronous RL. *arXiv preprint arXiv:2609.09783*, (2026). https://arxiv.org/abs/2609.09783v1. [Cited on page 9.]

List of Figures

1.1. The argument of the thesis 2

4.1. The two routes in SecureCourier 13
4.2. Gate success and shortcut probability after gate practice 14
4.3. Exact junction returns before and after gate practice 15
4.4. Two journeys after the same 22 actions 16
4.5. The gate sequence 16
4.6. How the next-state estimate enters an earlier TD target 17
4.7. Shortcut choice after 1,024 learner cycles 18
4.8. Exact weights improve route choice and delivery return 18

5.1. One AWAC actor update with PDR 27

6.1. Standard TD and PDR rollouts across three benchmarks 33
6.2. KeyCorridor success, all eight methods 33
6.3. MiniGridLoCA Task-B return, all eight methods 34
6.4. DeepSea expected return 35
6.5. SecureCourier: PDR versus Standard TD 36

7.1. Value correction in the controlled task 38
7.2. Correct action ranking at three propagation distances 38
7.3. Value recovery at $N = 50$ 39
7.4. Depth truncation at $N = 50$ 40
7.5. One-update actor response at $N = 50$ 41

8.1. Common-only control in MiniGridLoCA 44
8.2. SecureCourier fixed-actor TD repair 45
8.3. MiniGridLoCA actor-input intervention 46
8.4. Actor-signal alignment and best-action probability change 47

9.1. Removing one component of the revalued signal 50

A.1. Graph construction and reference alignment 59
A.2. A complete state/pair/batch mapping 61
A.3. Streaming for the policy-only source 62

B.1. DeepSea continuations through 8,192 actor updates 68

B.2. Public benchmark layouts 69

C.1. Shortcut weights during actor–critic continuation 73
C.2. Shortcut Q during fixed-actor TD repair 74
C.3. Stale junction estimates during fixed-actor TD training 74
C.4. Initial advantage and weight error on old shortcut records 75
C.5. Using the shortcut 76
C.6. Expected discounted delivery return from the depot 77
C.7. Continued training through cycle 2,048 78
C.8. SecureCourier comparisons with critic baselines 78
C.9. Early mean weight MAE on old shortcut records 80
C.10. Cosine similarity to the exact-weight actor gradient 80
C.11. Paired early gains over Standard TD 81
C.12. Full SecureCourier continuation 82
C.13. Gate use and delivery in the 2D view 83

D.1. Synchronous TD support at $N = 50$ 84
D.2. Size and threshold checks 85
D.3. Additional actor-mediation measurements at $N = 50$ 86
D.4. Retrospective long-distance continuation 87
D.5. Reads to 90% value recovery under two TD orders 88

E.1. LoCA–AWAC: gains between exact and learned references 91
E.2. Q error as graph coverage increases 93
E.3. Early gains over standard TD as graph coverage increases 94
E.4. Learning curves under partial graph coverage 95

G.1. Learning performance against time 100
G.2. Paired benchmark component times 104
G.3. Mean score versus cumulative training time 105
G.4. Native and optimized PDR time 106
G.5. The retained neural-reference LoCA QAC pair 109
G.6. Time and success on exact-reference KeyCorridor 109
G.7. Exact-reference KeyCorridor over checkpoints 111
G.8. Paired SecureCourier query and solve timings 113
G.9. SecureCourier cost accounting and sparse-change scaling 113
G.10. Additional numerical-path timings 115
G.11. Stochastic scaling systems 115

List of Tables

3.1. Prior work grouped by the quantity it changes . . . 8

4.1. Shortcut training weights during continuation . . . 15
4.2. Junction shortcut Q with the actor fixed . . . 17

5.1. Actor inputs built from one reference and one solved change . . . 25

6.1. Experiments in this thesis . . . 30
6.2. Continuation budgets, early windows and paired seeds . . . 31
6.3. Early and final performance on the public benchmarks . . . 32

7.1. Distance slopes of reads to 90% value recovery . . . 40
7.2. Value-input ablation . . . 42

8.1. Accuracy and behaviour rank interventions differently . . . 48

9.1. Reference and change weighting with learned references . . . 51

A.1. Reuse conditions . . . 60
A.2. Core evaluation costs . . . 63
A.3. Recorded timer boundaries in the training adapter . . . 64

B.1. Current PDR training settings . . . 65
B.2. PDR configurations . . . 66
B.3. Settings searched per method . . . 66
B.4. All selected methods: early means . . . 67
B.5. All selected methods: final means . . . 67
B.6. What changes between learning conditions . . . 70
B.7. Learning measures and budgets . . . 71
B.8. Transferred critic-baseline settings . . . 72

C.1. SecureCourier early means . . . 79
C.2. SecureCourier early paired effects . . . 82
C.3. SecureCourier outcomes at cycle 2,048 . . . 83

D.1. Distance slopes relative to replay TD . . . 88

E.1. Common-only ablation in MiniGridLoCA . . . 89

E.2. SecureCourier fixed-actor TD repair: error and sign 90

F.1. Full revaluation, change only and statewise gates 97
F.2. Mixtures of full-revaluation and change-only weights 97

G.1. Cost of the five methods on RTX 3090 99
G.2. Optimized PDR time relative to each baseline 101
G.3. Mean task scores at a shared time 101
G.4. Paired native and optimized PDR on RTX 3090 103
G.5. Native–optimized implementation differences 104
G.6. Task scores in paired native–optimized runs 108
G.7. Recorded costs in the learning runs 110
G.8. Paired SecureCourier PDR implementations on CPU 112
G.9. Native–optimized agreement on SecureCourier 112
G.10. PDR source discovery from policy snapshots 114
G.11. Scaling-backend timer scopes 116